Βελτιστοποίηση στο ευρετήριο R-tree με χρήση εντολών SIMD

ΕΛΕΥΘΕΡΙΟΣ ΜΑΡΙΟΣ ΜΑΝΙΚΑΣ

Διπλωματική Εργασία

Επιβλέπων: ΝΙΚΟΛΑΟΣ ΜΑΜΟΥΛΗΣ

Ιωάννινα, Οκτώβριος, 2025



ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY OF IOANNINA

Περίληψη

Η αυξανόμενη πολυπλοκότητα και ποσότητα πολυδιάστατων και χωρικών δεδομένων δημιουργεί την ανάγκη για αποδοτική επεξεργασία ερωτημάτων σε βάσεις δεδομένων. Ερωτήματα τύπου spatial select και spatial join, τα οποία βασίζονται σε δομές όπως το R-tree, απαιτούν σημαντική υπολογιστική ισχύ, ειδικά σε in-memory συνθήκες. Παράλληλα, οι σύγχρονες CPU παρέχουν προηγμένες δυνατότητες παραλληλισμού σε επίπεδο δεδομένων μέσω SIMD (Single Instruction Multiple Data) εντολών όπως AVX256 και AVX512.Η παρούσα εργασία μελετά τη χρήση SIMD επιτάχυνσης σε ερωτήματα τύπου spatial select πάνω σε R-tree indexes. Υλοποιήθηκαν και αξιολογήθηκαν τρία διαφορετικά layout κόμβων (D0, D1, D2), καθώς και scalar και vectorized εκδόσεις του αλγορίθμου σε γλώσσα C++. Οι διανυσματοποιημένες (vectorized) εκδόσεις αξιοποιούν εντολές AVX και μη αναδρομική προσπέλαση μέσω ουρών. Τα πειραματικά αποτελέσματα δείχνουν σημαντική επιτάχυνση σε σχέση με τις συμβατικές υλοποιήσεις, αποδεικνύοντας ότι η προσεκτική σχεδίαση της μνήμης και η χρήση SIMD μπορούν να βελτιώσουν δραστικά την απόδοση ερωτημάτων σε πολυδιάστατα δεδομένα.

Λέξεις Κλειδιά: SIMD, R-tree, Χωρικά Δεδομένα, Spatial Select, MBR

Abstract

The increasing complexity and volume of multidimensional and spatial data necessitate efficient query processing in modern database systems. Spatial queries, such as spatial select and spatial join, commonly rely on hierarchical indexing structures like the R-tree. These queries can become computationally intensive, particularly in main-memory settings. At the same time, modern CPUs offer advanced data-level parallelism capabilities through SIMD (Single Instruction Multiple Data) instructions, such as AVX256 and AVX512. This thesis investigates the use of SIMD acceleration for spatial queries on R-tree indexes. Three node storage layouts (D0, D1, D2) were implemented and evaluated, alongside both scalar and vectorized versions of the spatial select algorithm in C++. The vectorized implementations leverage AVX instructions and non-recursive traversal using queues. Experimental results demonstrate significant performance improvements over traditional scalar implementations, with the best SIMD variant achieving significant speedups. These findings highlight that careful memory layout design and SIMD-aware algorithm engineering can lead to highly efficient query execution on spatial data.

Keywords: SIMD, R-tree, Spatial Data, Spatial Select, MBR

Πίνακας περιεχομένων

Κεφάλαιο 1. ΕΙΣΑΓΩΓΗ 2

1.1 Η Εξέλιξη της Διαχείρισης Δεδομένων 2

1.2 Πολυδιάστατη Πρόκληση 3

1.3 Χωρικά Δεδομένα 4

1.3.1 Μοντέλα Αναπαράστασης Χωρικών Δεδομένων 4

1.3.2 Αποθήκευση Χωρικών Δεδομένων 6

1.3.3 Ευρετήρια και Βελτιστοποίηση Απόδοσης 7

1.3.4 Λογικά Χωρικά Ερωτήματα (Spatial Queries) 9

1.4 Τεχνικές Παραλληλισμού 12

1.5 Πρόβλημα, Σκοπός και Δομή της Εργασίας 15

Κεφάλαιο 2. Related Work 17

2.1 Εισαγωγή 17

2.2 Χωρικά Ευρετήρια 17

2.2.1 Point Access Methods (PAMs) 18

2.2.2 Spatial Access Methods (SAMs) 18

2.2.3 Σύνοψη 19

2.3 Αξιοποίηση του SIMD σε Θεμελιώδεις Τελεστές Βάσεων Δεδομένων 19

2.4 SIMD σε Δομές Ευρετηρίων Γενικού Σκοπού 20

2.5 SIMD σε Χωρικά Ευρετήρια 20

2.6 Σύνοψη και Τοποθέτηση της Παρούσας Εργασίας 21

Κεφάλαιο 3. Το Ευρετήριο R-tree 23

3.1 Εισαγωγή στα Χωρικά Ευρετήρια 23

3.2 Δομή του R-tree 23

3.2.1 Κόμβοι και Εγγραφές 24

3.2.2 Το Ελάχιστο Περιβάλλον Ορθογώνιο (MBR) 25

3.2.3 Κριτήρια Ποιότητας ενός R-tree 25

3.3 Λειτουργίες στο R-tree 26

3.3.1 Αναζήτηση (Search) 26

3.3.2 Εισαγωγή (Insertion) ... *27*
3.3.3 Διαγραφή (Deletion) ... *28*
3.4 Μαζική Φόρτωση: Ο Αλγόριθμος STR ... 29
3.5 Παραλλαγές του R-tree ... 31
3.6 Σύνοψη Κεφαλαίου ... 33
Κεφάλαιο 4. Σχεδιασμός & Υλοποίηση ... 34
4.1 Η ανάγκη διάταξης δεδομένων στη μνήμη ... 34
4.2 Γενική Αρχιτεκτονική Υλοποίησης ... 35
4.2.1 Κατασκευή του R-tree ... *35*
4.2.2 Κατασκευή Μεθόδου Αναζήτησης (Spatial Select) ... *37*
4.2.3 Βασικές Εντολές SIMD ... *37*
4.3 Διαφορετικά Layouts ... 39
4.3.1 Διάταξη D0 ... *39*
4.3.2 Διάταξη D1 ... *40*
4.3.3 Διάταξη D2 ... *41*
4.4 Μέθοδοι Αναζήτησης (spatial select) ... 42
4.4.1 Spatial Select D0 ... *42*
4.4.2 Spatial Select D1 ... *43*
4.4.3 Spatial Select D2 ... *44*
Κεφάλαιο 5. Πειραματική Αξιολόγηση ... 47
5.1 Πειραματικό Περιβάλλον ... 47
5.2 Σύνολα Δεδομένων ... 47
5.3 Σύνολα Ερωτημάτων ... 48
5.4 Μετρικές Αποδόσεις ... 49
5.5 Πειραματική ανάλυση ... 49
5.5.1 Σύγκριση Απόδοσης ως προς το Fanout ... *49*
5.5.2 Επίδραση της Επιλεκτικότητας στην Απόδοση ... *52*
5.5.3 Απόδοση SIMD στα Φύλλα ... *57*
5.5.4 Πείραμα Αξιολόγησης Απόδοσης ... *61*
Κεφάλαιο 6. Συμπεράσματα ... 64
Βιβλιογραφία ... 66

Κεφάλαιο 1. ΕΙΣΑΓΩΓΗ

1.1 Η Εξέλιξη της Διαχείρισης Δεδομένων

Η ικανότητα συλλογής, αποθήκευσης και επεξεργασίας δεδομένων αποτελεί θεμέλιο της τεχνολογικής και επιστημονικής προόδου. Στις πρώτες ημέρες των υπολογιστών, η διαχείριση δεδομένων στηριζόταν σε απλά συστήματα αρχείων, τα οποία, αν και λειτουργικά για μικρές, μεμονωμένες εφαρμογές, αποδείχθηκαν γρήγορα ανεπαρκή. Η έλλειψη δομής, η αδυναμία ταυτόχρονης πρόσβασης και η απουσία μηχανισμών για τη διασφάλιση της συνέπειας οδήγησαν στην ανάγκη για πιο εξελιγμένα συστήματα.

Οι πρώτες προσπάθειες για τη δόμηση αυτής της πολυπλοκότητας οδήγησαν στα ιεραρχικά και δικτυακά μοντέλα βάσεων δεδομένων κατά τις δεκαετίες του 1960 και 1970. Αυτά τα μοντέλα οργάνωναν τα δεδομένα σε δεντρικές δομές, αντίστοιχα, αντικατοπτρίζοντας τις φυσικές σχέσεις γονέα-παιδιού ή πιο γενικευμένες συνδέσεις. Παρά την καινοτομία τους, η πλοήγηση στα δεδομένα απαιτούσε πολύπλοκο κώδικα χαμηλού επιπέδου, καθιστώντας τα συστήματα δύσκαμπτα σε αλλαγές και δύσκολα στη χρήση.

Η πραγματική επανάσταση ήρθε το 1970 με τη δημοσίευση του Edgar F. Codd, "A Relational Model of Data for Large Shared Data Banks". Ο Codd πρότεινε το σχεσιακό μοντέλο, μια ριζικά διαφορετική προσέγγιση όπου τα δεδομένα οργανώνονται σε απλούς, δισδιάστατους πίνακες (σχέσεις), ανεξάρτητα από τη φυσική τους αποθήκευση. Η ισχύς του μοντέλου αυτού βρισκόταν στην απλότητά του και στη χρήση της μαθηματικής θεωρίας των συνόλων. Αποτέλεσε τη βάση για τη δημιουργία των Συστημάτων Διαχείρισης Σχεσιακών Βάσεων Δεδομένων (RDBMS) και της δηλωτικής γλώσσας SQL (Structured Query Language). Η SQL επέτρεψε στους χρήστες να περιγράφουν *τι* δεδομένα ήθελαν, αφήνοντας στο DBMS να βρει τον βέλτιστο τρόπο (query optimization) για να τα ανακτήσει. Αυτή η αφαίρεση απελευθέρωσε τους προγραμματιστές από τις πολυπλοκότητες της φυσικής αποθήκευσης και έθεσε τα θεμέλια για τη σύγχρονη εποχή των βάσεων δεδομένων.

1.2 Πολυδιάστατη Πρόκληση

Για δεκαετίες, το σχεσιακό μοντέλο κυριαρχούσε απόλυτα. Ωστόσο, στις αρχές του 21ου αιώνα υπήρξε μια εκθετική αύξηση στον όγκο, την ταχύτητα και την ποικιλία των παραγόμενων δεδομένων, ένα φαινόμενο που ονομάστηκε Big Data. Οι τρεις βασικές διαστάσεις του (3Vs - Volume, Velocity, Variety) αποκάλυψαν τους εγγενείς περιορισμούς των παραδοσιακών RDBMS:

- Όγκος (Volume): Η ανάγκη αποθήκευσης terabytes και petabytes δεδομένων κατέστησε τις παραδοσιακές αρχιτεκτονικές μονού server ανεπαρκείς, οδηγώντας σε κατανεμημένα συστήματα.
- Ταχύτητα (Velocity): Η συνεχής ροή δεδομένων από αισθητήρες, κοινωνικά δίκτυα και χρηματοοικονομικές συναλλαγές απαιτούσε συστήματα ικανά για επεξεργασία σε πραγματικό χρόνο.
- Ποικιλία (Variety): Τα δεδομένα δεν ήταν πλέον μόνο δομημένα (αριθμοί, κείμενο σε πίνακες) αλλά και ημι-δομημένα (JSON, XML) ή αδόμητα (εικόνες, βίντεο, φυσική γλώσσα).



Εικόνα 1 Διαστάσεις Big Data [PrCL]

Ως απάντηση, εμφανίστηκε το κίνημα NoSQL (Not only SQL), το οποίο εισήγαγε εναλλακτικά μοντέλα δεδομένων, όπως βάσεις εγγράφων (document stores), στήλης (column-family stores) και γράφων (graph databases), δίνοντας προτεραιότητα στην ευελιξία και την οριζόντια κλιμάκωση.

Πέρα από την ποικιλία των τύπων, μια άλλη, πιο θεμελιώδης πολυπλοκότητα προέκυψε: η πολλαπλότητα των διαστάσεων, Σύγχρονες εγγραφές δεδομένων συχνά περιγράφονται από δεκάδες ή εκατοντάδες χαρακτηριστικά (attributes). Για παράδειγμα, ένα ιατρικό αρχείο μπορεί να περιλαμβάνει δημογραφικά, βιομετρικές μετρήσεις και γονιδιωματικά δεδομένα, αναπαριστώντας κάθε ασθενή ως ένα σημείο σε έναν χώρο εκατοντάδων διαστάσεων. Αυτό το φαινόμενο, γνωστό ως η "Κατάρα της

Διαστατικότητας" (Curse of Dimensionality), καθιστά τις παραδοσιακές μεθόδους ευρετηρίασης, που λειτουργούν καλά σε μία ή δύο διαστάσεις, εντελώς αναποτελεσματικές. Σε χώρους υψηλών διαστάσεων, η έννοια της "γειτονιάς" χάνεται, καθώς όλα τα σημεία τείνουν να απέχουν περίπου το ίδιο μεταξύ τους, και τα ευρετήρια αναγκάζονται να εξετάσουν σχεδόν ολόκληρο το σύνολο δεδομένων.

1.3 Χωρικά Δεδομένα

Τα χωρικά δεδομένα (geospatial data) αποτελούν τη θεμέλιο λίθο της Γεωγραφικής Επιστήμης των Πληροφοριών (GIScience) και των εφαρμογών της. Η μοναδικότητά τους έγκειται στην ικανότητά τους να περιγράφουν όχι μόνο *τι* είναι ένα αντικείμενο, αλλά και *πού* βρίσκεται στον χώρο, παρέχοντας ένα πλαίσιο για την κατανόηση των σχέσεων και των προτύπων που διέπουν τον κόσμο μας. Στον πυρήνα τους, τα χωρικά δεδομένα είναι πληροφορίες που συνδέονται άρρηκτα με μια συγκεκριμένη γεωγραφική θέση. Μια ολοκληρωμένη χωρική οντότητα αποτελείται από τρία βασικά στοιχεία:

1. Γεωμετρική Πληροφορία: Το σχήμα και η θέση της οντότητας στον χώρο, όπως αναπαρίσταται από ένα σημείο, μια γραμμή, ένα πολύγωνο ή ένα κελί πλέγματος (raster).
2. Περιγραφικά Γνωρίσματα (Attributes): Οι μη-χωρικές πληροφορίες που περιγράφουν τα χαρακτηριστικά της οντότητας (π.χ., το όνομα ενός δρόμου, ο πληθυσμός μιας πόλης, ο τύπος μιας χρήσης γης).
3. Σύστημα Αναφοράς Συντεταγμένων (CRS): Το γεωδαιτικό πλαίσιο που επιτρέπει στις συντεταγμένες της γεωμετρίας να αντιστοιχηθούν σε μια πραγματική, μετρήσιμη θέση στην επιφάνεια της Γης.

1.3.1 Μοντέλα Αναπαράστασης Χωρικών Δεδομένων

Η αναπαράσταση των χωρικών δεδομενων γίνεται κυρίως με δύο θεμελιώδη μοντέλα: το διανυσματικό και το ψηφιδωτό (κελιδοστοιχία).

Διανυσματικό Μοντέλο (Vector)

Το διανυσματικό μοντέλο αναπαριστά τον κόσμο ως ένα σύνολο από διακριτές οντότητες με σαφώς καθορισμένα όρια, που ορίζονται από συντεταγμένες. Είναι ιδανικό για την περιγραφή αντικειμένων και φαινομένων που έχουν σαφή γεωμετρία.

- Σημεία (Points): Μεμονωμένες συντεταγμένες (x, y) που αναπαριστούν οντότητες χωρίς έκταση, όπως αισθητήρες, ΑΤΜ ή κτήρια όταν εξετάζονται σε μεγάλη κλίμακα.
- Γραμμές (Lines/Polylines): Σειρές συνδεδεμένων συντεταγμένων που σχηματίζουν γραμμικά χαρακτηριστικά, όπως δρόμοι, ποτάμια και δίκτυα κοινής ωφέλειας.
- Πολύγωνα (Polygons): Κλειστές ακολουθίες συντεταγμένων που ορίζουν περιοχές με εμβαδόν και περίμετρο, όπως οικόπεδα, λίμνες, χρήσεις γης και διοικητικές ενότητες.

Το μεγάλο πλεονέκτημα του διανυσματικού μοντέλου είναι η υψηλή του ακρίβεια στην αναπαράσταση ορίων και η εγγενής του ικανότητα να διαχειρίζεται τοπολογικές σχέσεις (γειτνίαση, συνδεσιμότητα, περιοχή).

Κελιδοστοιχία (Raster)

Το μοντέλο της κελιδοστοιχίας αναπαριστά τον χώρο ως ένα συνεχές πλέγμα (grid) από κελιά (pixels). Κάθε κελί κατέχει μια αριθμητική τιμή που αντιπροσωπεύει το μέγεθος ενός φαινομένου σε εκείνη τη θέση (π.χ., υψόμετρο, θερμοκρασία, δείκτης βλάστησης). Είναι το ιδανικό μοντέλο για τη μελέτη συνεχόμενων φαινομένων που μεταβάλλονται ομαλά στον χώρο, όπως τα δεδομένα τηλεπισκόπησης και τα ψηφιακά μοντέλα εδάφους.

Θεμελιώδεις Γεωμετρικές Έννοιες:

- Συντεταγμένες: Η θέση κάθε κορυφής σε μια διανυσματική γεωμετρία ορίζεται από συντεταγμένες 2D (x,y) ή 3D (x,y,z). Ορισμένα συστήματα υποστηρίζουν και μια τέταρτη διάσταση m (measure) για μετρήσεις κατά μήκος γραμμικών στοιχείων (π.χ., χιλιομετρικές θέσεις σε αυτοκινητόδρομο).
- Συστήματα Αναφοράς Συντεταγμένων (CRS): Ένα CRS είναι απαραίτητο για να αποκτήσουν οι συντεταγμένες νόημα στον πραγματικό κόσμο. Διακρίνονται σε:
    - Γεωγραφικά CRS: Ορίζουν θέσεις σε μια σφαιροειδή/ελλειψοειδή επιφάνεια μέσω γεωγραφικού πλάτους και μήκους (π.χ., WGS84 / EPSG:4326).
    - Προβολικά CRS: Μετασχηματίζουν τις γεωγραφικές συντεταγμένες σε ένα επίπεδο καρτεσιανό πλέγμα (x,y) για ακριβείς μετρήσεις αποστάσεων και εμβαδών (π.χ., ΕΓΣΑ87 / EPSG:2100).

- Ακρίβεια (Accuracy) vs. Ανάλυση (Precision): Είναι κρίσιμο να διακρίνουμε την ακρίβεια (πόσο κοντά είναι μια μέτρηση στην πραγματική τιμή) από την ανάλυση/λεπτομέρεια (το πλήθος των δεκαδικών ψηφίων μιας συντεταγμένης ή το μέγεθος του pixel σε ένα raster).

Τοπολογία & Χωρικές Σχέσεις

Η τοπολογία μελετά τις ιδιότητες των γεωμετρικών σχημάτων που παραμένουν αμετάβλητες υπό παραμόρφωση. Στα GIS, η τοπολογία επιτρέπει τον ορισμό και τον έλεγχο των χωρικών σχέσεων.

- DE-9IM (Dimensionally Extended 9-Intersection Model): Είναι το θεωρητικό μαθηματικό μοντέλο που τυποποιεί τις χωρικές σχέσεις μεταξύ δύο γεωμετριών. Εξετάζει τις τομές μεταξύ του Εσωτερικού (Interior), του Ορίου (Boundary) και του Εξωτερικού (Exterior) κάθε γεωμετρίας, ορίζοντας με ακρίβεια σχέσεις όπως intersects, within, contains, touches, overlaps και crosses.
- Κανόνες Τοπολογίας: Σε προηγμένα γεωγραφικά σύνολα δεδομένων (geodatabases), μπορούν να οριστούν κανόνες για τη διασφάλιση της λογικής συνέπειας των δεδομένων (π.χ., «τα πολύγωνα των δήμων δεν πρέπει να επικαλύπτονται», «οι γραμμές των οδών πρέπει να συνδέονται σε κόμβους»).

1.3.2 Αποθήκευση Χωρικών Δεδομένων

Τα χωρικά δεδομένα αποθηκεύονται σε διάφορες μορφές, ανάλογα με την εφαρμογή:

- Αρχεία: Shapefile (παλαιότερο αλλά διαδεδομένο), GeoPackage (σύγχρονο, βασισμένο σε SQLite), GeoJSON (για web εφαρμογές), TIFF/Cloud-Optimized GeoTIFF (για raster).
- Χωρικές Βάσεις Δεδομένων: Συστήματα που επεκτείνουν τις κλασικές βάσεις δεδομένων με δυνατότητες αποθήκευσης, ευρετηρίασης και ανάλυσης χωρικών δεδομένων. Κορυφαία παραδείγματα είναι το PostGIS (επέκταση της PostgreSQL), το SpatiaLite και οι λύσεις της Esri.
- Υπηρεσίες Web: Πρότυπα του OGC για τη διάθεση δεδομένων μέσω διαδικτύου, όπως WMS/WMTS (εικόνες χαρτών), WFS (διανυσματικές οντότητες) και WCS (raster δεδομένα).

1.3.3 Ευρετήρια και Βελτιστοποίηση Απόδοσης

Η αξιοποίηση χωρικών δεδομένων προϋποθέτει την ικανότητα εκτέλεσης σύνθετων ερωτημάτων με υψηλή ταχύτητα και αποδοτικότητα. Η επεξεργασία τέτοιων ερωτημάτων σε μεγάλα σύνολα δεδομένων εγείρει μια θεμελιώδη υπολογιστική πρόκληση, η οποία καθιστά αναγκαία τη χρήση εξειδικευμένων δομών δεικτοδότησης.

Η γραμμική σάρωση (linear scan) είναι η απλούστερη στρατηγική εκτέλεσης ερωτημάτων: ο διαδοχικός έλεγχος κάθε εγγραφής σε ένα σύνολο δεδομένων, με χρονική πολυπλοκότητα O(N) ανά ερώτημα.

Για παράδειγμα:

«Εντόπισε όλα τα φαρμακεία εντός των ορίων του Δήμου Θεσσαλονίκης» σε μια βάση που περιέχει εκατομμύρια επιχειρήσεις πανελλαδικά. Με γραμμική σάρωση, το σύστημα ελέγχει διαδοχικά κάθε επιχείρηση (σε όλη την Ελλάδα)ως προς το αν οι συντεταγμένες της εμπίπτουν στο πολύγωνο του δήμου.Παρότι ορθό για μικρά μεγέθη, το κόστος O(N) καθίσταται απαγορευτικό σε κλίμακες εκατομμυρίων ή δισεκατομμυρίων οντοτήτων, ειδικά όταν τα ερωτήματα είναι διαδραστικά ή συχνά.

Επιπλέον, το συνολικό κόστος επιβαρύνεται από:

- τον αριθμό κορυφών του πολυγώνου (κόστος point-in-polygon),
- τη χαμηλή τοπικότητα μνήμης,
- και την απουσία δυνατότητας «πρόωρου αποκλεισμού» άσχετων περιοχών.

Το αποτέλεσμα είναι χρόνοι απόκρισης ασύμβατοι με απαιτήσεις πραγματικού χρόνου (π.χ. πλοήγηση, υπηρεσίες εντοπισμού).

Η γραμμική σάρωση παραμένει χρήσιμη όταν:

- το σύνολο δεδομένων είναι μικρό ή μεσαίου μεγέθους,
- το ερώτημα εκτελείται σπάνια ή εκτός κρίσιμου μονοπατιού,
- απαιτείται πλήρης διάσχιση για λόγους ελέγχου/επαλήθευσης,
- ή όταν το κόστος συντήρησης ευρετηρίων υπερβαίνει το όφελος (π.χ. εξαιρετικά υψηλός ρυθμός εισαγωγών/διαγραφών με σποραδική αναζήτηση).

Για κλιμακώσιμη απόδοση σε μεγάλα χωρικά σύνολα, η προσέγγιση αντικαθίσταται από χωρική δεικτοδότηση (spatial indexing), η οποία περιορίζει τον χώρο αναζήτησης σε μικρό σύνολο υποψηφίων και επιτυγχάνει υπο-γραμμική συμπεριφορά για συνήθη

ερωτήματα.Η κλιμάκωση επιτυγχάνεται μέσω χωρικών ευρετηρίων (spatial indexes).Τα χωρικά ευρετήρια είναι εξειδικευμένες δομές δεδομένων που οργανώνουν γεωμετρικά αντικείμενα στο χώρο, συνήθως ομαδοποιώντας όσα είναι γεωγραφικά γειτονικά. Αναλογικά με το ευρετήριο βιβλίου, επιτρέπουν στη μηχανή να παρακάμπτει άμεσα μεγάλες, άσχετες περιοχές και να εστιάζει μόνο σε υποψήφιες ζώνες ενδιαφέροντος. Τυπικά, επιτυγχάνουν υπο-γραμμική απόδοση για συνήθη ερωτήματα, μειώνοντας δραστικά τον απαιτούμενο αριθμό ελέγχων.

Ενδεικτικές δομές.

- R-tree και παραλλαγές (R*-tree, R+-tree): κυρίαρχες για διανυσματικά δεδομένα, βασισμένες σε ιεραρχίες ελάχιστων περιβαλλόντων ορθογωνίων (MBR).
- Quadtree: ευρέως χρησιμοποιούμενο σε ιεραρχική αποσύνθεση χώρου, ιδίως για raster αλλά και για χωρική οργάνωση σημείων/διανυσματικών χαρακτηριστικών.

Η υιοθέτηση χωρικών ευρετηρίων οδηγεί σε βελτίωση απόδοσης κατά πολλές τάξεις μεγέθους, επιτρέποντας:

- Ταχύτητα: χρόνοι απόκρισης σε επίπεδο χιλιοστών του δευτερολέπτου αντί λεπτών.
- Κλιμάκωση (scalability): αποτελεσματική διαχείριση πολύ μεγάλων όγκων δεδομένων.
- Εφικτότητα (feasibility): καθίστανται πρακτικά υλοποιήσιμες προηγμένες αναλυτικές διεργασίες.

Ιδίως, επιταχύνονται θεμελιώδεις τελεστές χωρικών βάσεων:

- Αναζήτηση περιοχής (spatial select query),
- Εύρεση πλησιέστερων γειτόνων (k-NN),
- Χωρική σύζευξη (spatial join).

Η γραμμική σάρωση συνιστά εγγενή δομική ανεπάρκεια για την κλιμακούμενη εκτέλεση χωρικών ερωτημάτων, εισάγοντας υπολογιστικό κόστος ανάλογο του όγκου των δεδομένων. Αντιθέτως, οι δομές χωρικής ευρετηρίασης, όπως οι οικογένειες R-tree και Quadtree, προσφέρουν τη θεμελιώδη αρχιτεκτονική για την υπολογιστικά αποδοτική και επεκτάσιμη επεξεργασία γεω-χωρικών δεδομένων. Πλέον, το ζητούμενο δεν είναι απλώς να κάνουμε γρήγορες τις μεμονωμένες εντολές, αλλά να βρίσκουμε τον πιο έξυπνο τρόπο

να συνδυάζουμε πολλές εντολές μαζί για να απαντάμε σε σύνθετα, πραγματικά ερωτήματα.

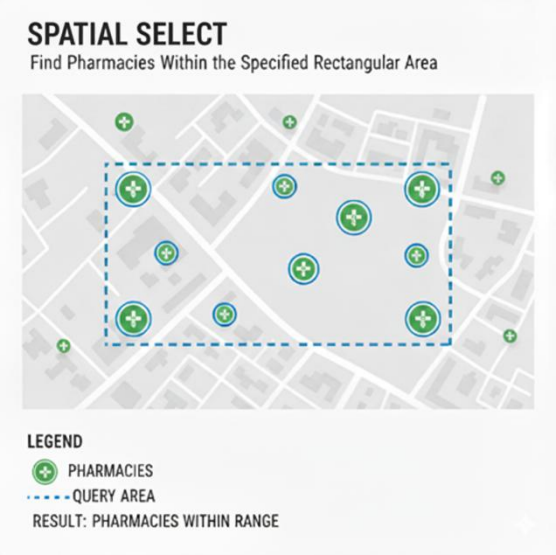
1.3.4 Λογικά Χωρικά Ερωτήματα (Spatial Queries)

Τα χωρικά ερωτήματα (spatial queries) αποτελούν τη βάση της ανάκτησης πληροφοριών από γεωγραφικά συστήματα πληροφοριών (GIS) και χωρικές βάσεις δεδομένων. Μας επιτρέπουν να αναζητούμε και να αναλύουμε δεδομένα με βάση τη γεωγραφική τους θέση και τις χωρικές τους σχέσεις. Τρεις από τις πιο θεμελιώδεις και συχνά χρησιμοποιούμενες κατηγορίες χωρικών ερωτημάτων είναι τα Spatial Select Queries, τα k-Nearest Neighbors (kNN) Queries και τα Spatial Joins.

Ερώτημα Αναζήτηση περιοχής (Spatial Select Query)

Ένα ερώτημα αναζήτηση περιοχής ανακτά όλα τα γεωγραφικά αντικείμενα που τέμνονται, περιέχονται πλήρως ή εφάπτονται με μια προκαθορισμένη γεωμετρική περιοχή αναζήτησης, συνήθως ένα ορθογώνιο ή έναν κύκλο. Είναι ιδανικό για ερωτήματα του τύπου "βρες όλα τα αντικείμενα μέσα σε αυτή την περιοχή".

- Λειτουργία: Το σύστημα ελέγχει ποια από τα γεωμετρικά αντικείμενα της βάσης δεδομένων (π.χ. σημεία, γραμμές, πολύγωνα) έχουν κάποια χωρική σχέση με το γεωμετρικό εύρος του ερωτήματος. Η απόδοση αυτής της διαδικασίας βελτιστοποιείται σημαντικά με τη χρήση χωρικών ευρετηρίων, όπως τα R-trees, που επιτρέπουν την γρήγορη απόρριψη αντικειμένων που βρίσκονται εκτός της περιοχής ενδιαφέροντος.
- Παράδειγμα: Έστω ότι ένας χρήστης θέλει να βρει όλα τα εστιατόρια σε ακτίνα 1 χιλιομέτρου από το κέντρο της πόλης. Η περιοχή αναζήτησης είναι ένας κύκλος (το εύρος) και το αποτέλεσμα του ερωτήματος θα είναι το σύνολο των σημείων (εστιατόρια) που βρίσκονται μέσα σε αυτόν τον κύκλο.

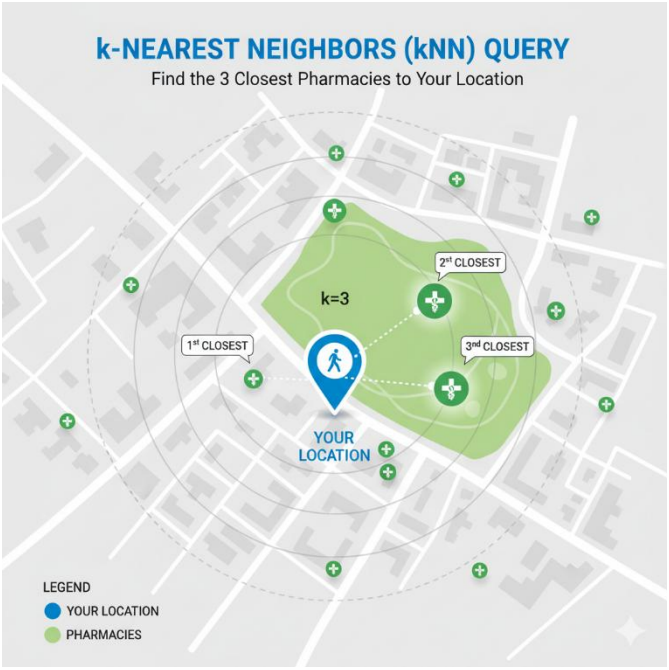


Εικόνα 2: Παράδειγμα για spatial select

Ερώτημα k-Πλησιέστερων Γειτόνων (k-Nearest Neighbors - kNN)

Ένα ερώτημα k-πλησιέστερων γειτόνων (kNN) εντοπίζει τα k γεωγραφικά αντικείμενα που βρίσκονται στην πλησιέστερη απόσταση από ένα συγκεκριμένο σημείο αναφοράς. Σε αντίθεση με το spatial select query, εδώ ο αριθμός των αποτελεσμάτων (k) είναι προκαθορισμένος, όχι η περιοχή αναζήτησης.

- Λειτουργία: Το ερώτημα ξεκινά από ένα σημείο αναφοράς και "επεκτείνεται" στον χώρο, αναζητώντας τα αντικείμενα με τη μικρότερη ευκλείδεια απόσταση. Αλγόριθμοι όπως οι Best-First Search σε δομές R-tree χρησιμοποιούνται για να βρουν αποτελεσματικά τους k γείτονες χωρίς να χρειάζεται να υπολογίσουν την απόσταση από όλα τα αντικείμενα της βάσης.
- Παράδειγμα: Ένας τουρίστας χρειάζεται να βρει τα 3 πλησιέστερα φαρμακεία στη τρέχουσα τοποθεσία του. Το σημείο αναφοράς είναι η τοποθεσία του τουρίστα, το k είναι ίσο με 3, και το αποτέλεσμα θα είναι τα τρία σημεία-φαρμακεία που έχουν τη μικρότερη απόσταση από αυτόν.

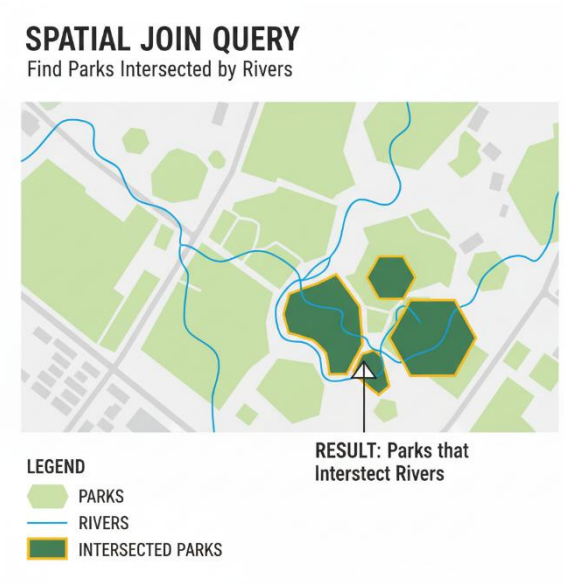


Εικόνα 3: Παράδειγμα για k-NN query

Χωρική Σύζευξη (Spatial Join)

Η χωρική σύζευξη είναι μια από τις πιο ισχυρές λειτουργίες στις χωρικές βάσεις δεδομένων. Συνδυάζει τις πληροφορίες δύο διαφορετικών συνόλων δεδομένων (π.χ. δύο πινάκων) βάσει μιας χωρικής σχέσης (spatial predicate) μεταξύ των γεωμετριών τους. Ενώνει, δηλαδή, εγγραφές από δύο πίνακες όταν τα γεωγραφικά τους αντικείμενα ικανοποιούν μια συνθήκη, όπως "τέμνονται", "περιέχουν" ή "βρίσκονται σε απόσταση μικρότερη από Χ".

- Λειτουργία: Η διαδικασία περιλαμβάνει δύο στάδια: το φιλτράρισμα (filter step), όπου με τη βοήθεια χωρικών ευρετηρίων εντοπίζονται τα ζεύγη αντικειμένων που *πιθανόν* να ικανοποιούν τη συνθήκη, και το στάδιο βελτίωσης (refinement step), όπου γίνονται ακριβείς γεωμετρικοί υπολογισμοί μόνο σε αυτά τα ζεύγη για να επιβεβαιωθεί η σχέση.
- Παράδειγμα: Μια δημοτική αρχή θέλει να βρει όλα τα πάρκα που διασχίζονται από ποτάμια για να ελέγξει πιθανές πλημμυρικές ζώνες. Εδώ, πραγματοποιείται μια χωρική σύζευξη μεταξύ του συνόλου δεδομένων των πάρκων (πολύγωνα) και του συνόλου δεδομένων των ποταμών (γραμμές). Η συνθήκη είναι η "τέμνεται" (intersects). Το αποτέλεσμα θα είναι ένας νέος πίνακας που θα συνδυάζει τις πληροφορίες των πάρκων και των ποταμών για κάθε ζεύγος που τέμνεται.



Εικόνα 4: Παράδειγμα για spatial join query

1.4 Τεχνικές Παραλληλισμού

Για δεκαετίες, η αύξηση της απόδοσης των επεξεργαστών (CPU) ακολουθούσε τον Νόμο του Moore, με τον διπλασιασμό των τρανζίστορ να οδηγεί σε υψηλότερες ταχύτητες ρολογιού. Ωστόσο, φυσικοί περιορισμοί (κατανάλωση ενέργειας, έκλυση θερμότητας) επιβράδυναν δραματικά αυτή την τάση. Η βιομηχανία στράφηκε προς τον παραλληλισμό ως τον κύριο δρόμο για την αύξηση της υπολογιστικής ισχύος. Η παράλληλη επεξεργασία αποτελεί τη θεμελιώδη στρατηγική της σύγχρονης υπολογιστικής για την επίτευξη υψηλών επιδόσεων. Η κεντρική ιδέα είναι η ταυτόχρονη εκτέλεση πολλαπλών υπολογισμών για την ταχύτερη επίλυση ενός προβλήματος. Οι τεχνικές για την επίτευξη του παραλληλισμού διαφέρουν ως προς την κλίμακα εφαρμογής από τις λειτουργίες εντός ενός πυρήνα CPU έως τη συνεργασία χιλιάδων υπολογιστών και το μοντέλο μνήμης που υιοθετούν.

1. Παραλληλισμός Δεδομένων σε Επίπεδο Επεξεργαστή (SIMD)

Ο παραλληλισμός Single Instruction, Multiple Data (SIMD) είναι μια μορφή παραλληλισμού σε επίπεδο δεδομένων (Data-Level Parallelism), η οποία υλοποιείται εντός ενός μεμονωμένου πυρήνα επεξεργαστή.

- Θεμελιώδης Αρχή: Η αρχή του SIMD συνίσταται στην ταυτόχρονη εφαρμογή μιας μοναδικής εντολής σε πολλαπλά, ανεξάρτητα στοιχεία δεδομένων που είναι οργανωμένα σε διανυσματικές δομές. Αντί η επεξεργασία να εκτελείται σειριακά σε κάθε στοιχείο ενός συνόλου δεδομένων, η ίδια λειτουργία εφαρμόζεται

ταυτόχρονα σε ολόκληρο το διάνυσμα δεδομένων, πολλαπλασιάζοντας την υπολογιστική απόδοση (throughput) για αλγορίθμους με υψηλό βαθμό παραλληλισμού δεδομένων.

- Τρόπος Λειτουργίας: Οι σύγχρονοι επεξεργαστές ενσωματώνουν διανυσματικούς καταχωρητές (vector registers) μεγάλου πλάτους (π.χ., 128, 256 ή 512 bits) και εξειδικευμένα σύνολα εντολών (SSE, AVX). Ένας τέτοιος καταχωρητής μπορεί να "πακετάρει" πολλαπλά στοιχεία δεδομένων. Μια εντολή AVX εκτελεί την πράξη (π.χ., πρόσθεση) σε όλα τα ζεύγη ταυτόχρονα, προσφέροντας πολλαπλάσια απόδοση.
- Κύρια Χρήση: Ιδανική για αλγορίθμους με επαναλαμβανόμενες πράξεις σε μεγάλους όγκους δεδομένων, όπως στην επεξεργασία πολυμέσων, σε επιστημονικές προσομοιώσεις και στη βελτιστοποίηση πράξεων σε βάσεις δεδομένων, όπως ο υπολογισμός γεωμετρικών σχέσεων σε ευρετήρια R-tree.

2. Παραλληλισμός σε Επίπεδο Νημάτων (Multithreading)

Το multithreading είναι η κυρίαρχη τεχνική για την αξιοποίηση πολυπύρηνων επεξεργαστών (multi-core CPUs) σε ένα σύστημα. Αποτελεί μια υλοποίηση παραλληλισμού σε επίπεδο νημάτων (thread-level parallelism) και βασίζεται στο υπολογιστικό μοντέλο της κοινόχρηστης μνήμης (shared memory).

- Θεμελιώδης Αρχή: Η κεντρική αρχή είναι η διαίρεση μιας διαδικασίας σε πολλαπλές, ταυτόχρονα εκτελούμενες ροές ελέγχου, γνωστές ως νήματα (threads). Τα νήματα αυτά μοιράζονται τον ίδιο χώρο διευθύνσεων (address space), επιτρέποντας την άμεση και ταχεία επικοινωνία μέσω κοινών μεταβλητών στη μνήμη. Κάθε νήμα αποτελεί μια ανεξάρτητη μονάδα εκτέλεσης που μπορεί να ανατεθεί από το λειτουργικό σύστημα σε έναν διαθέσιμο πυρήνα.
- Τρόπος Λειτουργίας: Κάθε πυρήνας του επεξεργαστή αναλαμβάνει την εκτέλεση ενός ή περισσότερων νημάτων. Η άμεση πρόσβαση στην κοινόχρηστη μνήμη καθιστά την ανταλλαγή πληροφοριών αποδοτική, αλλά εισάγει την πρόκληση του συγχρονισμού. Απαιτείται η χρήση πρωτόγονων συγχρονισμού (π.χ., mutexes, semaphores) για τη διασφάλιση της ακεραιότητας των δεδομένων και την αποφυγή συνθηκών ανταγωνισμού (race conditions).
- Κύρια Χρήση: Εφαρμογές γραφείου, διακομιστές (web servers) που εξυπηρετούν ταυτόχρονα χιλιάδες συνδέσεις, και γενικότερα, κάθε παράλληλος αλγόριθμος που εκτελείται σε έναν υπολογιστή.

3. Μαζικός Παραλληλισμός σε Μονάδες Επεξεργασίας Γραφικών (GPGPU)

Η τεχνική General-Purpose computing on Graphics Processing Units (GPGPU) αξιοποιεί την αρχιτεκτονική μαζικού παραλληλισμού των GPUs για την επιτάχυνση υπολογισμών γενικού σκοπού.

- Θεμελιώδης Αρχή: Η αρχιτεκτονική των GPU χαρακτηρίζεται από έναν πολύ μεγάλο αριθμό απλούστερων επεξεργαστικών πυρήνων, βελτιστοποιημένων για την ταυτόχρονη εκτέλεση της ίδιας συνάρτησης (kernel) σε τεράστια σύνολα δεδομένων. Η τεχνική αυτή αποτελεί μια επέκταση του μοντέλου SIMD, γνωστή ως Single Instruction, Multiple Threads (SIMT), και είναι εξαιρετικά αποτελεσματική για προβλήματα με υψηλή αριθμητική ένταση και εγγενή παραλληλισμό δεδομένων.
- Τρόπος Λειτουργίας: Ο προγραμματιστής, χρησιμοποιώντας πλατφόρμες όπως η CUDA (NVIDIA) ή η OpenCL, ορίζει συναρτήσεις-πυρήνες (kernels) που εκτελούνται από χιλιάδες νήματα στην GPU. Τα δεδομένα μεταφέρονται από τη μνήμη του συστήματος (RAM) στην αποκλειστική μνήμη της GPU, όπου πραγματοποιείται η μαζικά παράλληλη επεξεργασία, και τα αποτελέσματα επιστρέφονται.
- Κύρια Χρήση: Εκπαίδευση νευρωνικών δικτύων, επιστημονικές προσομοιώσεις μεγάλης κλίμακας, ανάλυση εικόνας και εξόρυξη κρυπτονομισμάτων.

4. Παραλληλισμός Εργασιών και Διοχέτευση (Task Parallelism / Pipelining)

Σε αντιδιαστολή με τον παραλληλισμό δεδομένων, ο παραλληλισμός εργασιών εστιάζει στην ταυτόχρονη εκτέλεση λειτουργικά διακριτών και ενδεχομένως ετερογενών υπολογιστικών εργασιών (tasks).

- Θεμελιώδης Αρχή: Ένα υπολογιστικό πρόβλημα αναλύεται σε ένα σύνολο από ανεξάρτητες ή μερικώς εξαρτημένες εργασίες, οι οποίες μπορούν να εκτελεστούν ταυτόχρονα. Όταν οι εργασίες αυτές είναι διατεταγμένες σε μια σειρά, όπου η έξοδος μιας εργασίας αποτελεί την είσοδο για την επόμενη, η αρχιτεκτονική αυτή ονομάζεται διοχέτευση (pipelining). Η διοχέτευση επιτρέπει την επικάλυψη της εκτέλεσης των σταδίων, αυξάνοντας τη συνολική απόδοση (throughput) του συστήματος.
- Τρόπος Λειτουργίας: Οι διαφορετικές εργασίες ανατίθενται σε διαφορετικούς πυρήνες ή νήματα. Η διαχείριση των εξαρτήσεων μεταξύ των εργασιών είναι

κρίσιμη και συνήθως υλοποιείται με χρήση γραφημάτων εργασιών (task graphs) και μηχανισμών συγχρονισμού.

- Κύρια Χρήση: Συστήματα επεξεργασίας σήματος και εικόνας όπου εφαρμόζονται διαδοχικά φίλτρα, μεταγλωττιστές (compilers), και σύνθετες εφαρμογές όπου ανεξάρτητες λειτουργίες (π.χ., γραφικά, ήχος, τεχνητή νοημοσύνη σε ένα παιχνίδι) εκτελούνται παράλληλα.

5. Κατανεμημένος Παραλληλισμός (Distributed Computing)

Ο κατανεμημένος παραλληλισμός αντιπροσωπεύει την επέκταση της παράλληλης επεξεργασίας σε ένα σύνολο από αυτόνομους, δικτυωμένους υπολογιστές (κόμβους). Βασίζεται στο μοντέλο της κατανεμημένης μνήμης (distributed memory).

- Θεμελιώδης Αρχή: Η υπολογιστική ισχύς και η μνήμη πολλαπλών υπολογιστών συνδυάζονται για την επίλυση ενός ενιαίου προβλήματος. Κάθε κόμβος λειτουργεί με τον δικό του ιδιωτικό χώρο διευθύνσεων (αρχιτεκτονική "shared-nothing"). Ως εκ τούτου, η συνεργασία και ο συντονισμός μεταξύ των κόμβων δεν μπορεί να επιτευχθεί μέσω κοινής μνήμης, αλλά απαιτεί την ρητή ανταλλαγή μηνυμάτων (explicit message passing) μέσω του δικτύου επικοινωνίας.
- Τρόπος Λειτουργίας: Το πρόβλημα και τα δεδομένα του κατανέμονται στους διαθέσιμους κόμβους. Οι διαδικασίες που εκτελούνται σε κάθε κόμβο επεξεργάζονται το τοπικό τους τμήμα δεδομένων και επικοινωνούν με άλλες διαδικασίες για την ανταλλαγή πληροφοριών. Το Message Passing Interface (MPI) είναι το πρότυπο API που καθορίζει τις λειτουργίες για την αποστολή και λήψη μηνυμάτων σε τέτοια συστήματα.
- Κύρια Χρήση: Επεξεργασία Μεγάλων Δεδομένων (Big Data) με πλατφόρμες όπως το Apache Spark, υπολογισμοί υψηλών επιδόσεων (HPC) σε υπερ-υπολογιστές (supercomputers) για επιστημονικές προσομοιώσεις, και προβλήματα που υπερβαίνουν τους υπολογιστικούς πόρους ενός μεμονωμένου υπολογιστή.

1.5 Πρόβλημα, Σκοπός και Δομή της Εργασίας

Το Πρόβλημα: Ενώ τα R-trees παρέχουν ένα αποτελεσματικό αλγοριθμικό πλαίσιο για τη μείωση του χώρου αναζήτησης και η τεχνολογία SIMD προσφέρει τεράστια υπολογιστική ισχύ σε επίπεδο υλικού, υπάρχει ένα σημαντικό ερευνητικό κενό στη σχεδίαση δομών δεδομένων και αλγορίθμων που εκμεταλλεύονται πλήρως αυτόν τον

παραλληλισμό. Οι κλασικές υλοποιήσεις R-tree οργανώνουν τα δεδομένα στη μνήμη με τρόπο που δεν είναι φιλικός προς τις διανυσματικές λειτουργίες, εμποδίζοντας την αποτελεσματική εφαρμογή του SIMD.

Σκοπός: Κεντρικός σκοπός της παρούσας διπλωματικής εργασίας είναι η διερεύνηση, ο σχεδιασμός, η υλοποίηση και η αξιολόγηση βελτιστοποιημένων μεθόδων για την εκτέλεση ερωτημάτων περιοχής (spatial select queries) σε R-trees, αξιοποιώντας τις δυνατότητες των σύγχρονων εντολών SIMD. Η έρευνα επικεντρώνεται στο πώς η κατάλληλη οργάνωση των δεδομένων στη μνήμη (memory layout) και ο ανασχεδιασμός των αλγορίθμων αναζήτησης μπορούν να ξεκλειδώσουν την πλήρη ισχύ του παραλληλισμού σε επίπεδο δεδομένων.

Κεφάλαιο 2. Related Work

2.1 Εισαγωγή

Η βελτιστοποίηση της απόδοσης των συστημάτων βάσεων δεδομένων αποτελεί σταθερά αντικείμενο ερευνητικού ενδιαφέροντος. Παραδοσιακά, η έμφαση δινόταν στη μείωση των προσβάσεων στον δίσκο (I/O). Ωστόσο, με την επικράτηση των in-memory βάσεων, το επίκεντρο έχει μετατοπιστεί στην αποδοτική αξιοποίηση της CPU και της ιεραρχίας της μνήμης. Σε αυτό το πλαίσιο, η τεχνολογία SIMD (Single Instruction, Multiple Data) έχει αναδειχθεί ως ένα σημαντικό εργαλείο για την εκμετάλλευση παραλληλισμού σε επίπεδο δεδομένων. Το παρόν κεφάλαιο εξετάζει τη σχετική βιβλιογραφία, αναδεικνύοντας την εξέλιξη της έρευνας και το πλαίσιο μέσα στο οποίο εντάσσεται η παρούσα μελέτη. Αρχικά, παρουσιάζουμε τα βασικά χωρικά ευρετήρια που χρησιμοποιούνται σε βάσεις δεδομένων, αναδεικνύοντας τη σημασία τους για εφαρμογές που χειρίζονται χωρικά δεδομένα. Στη συνέχεια, επισκοπούμε τις εφαρμογές του SIMD σε θεμελιώδεις τελεστές βάσεων δεδομένων (π.χ. σάρωση, συνενώσεις, ταξινομήσεις), όπου η τεχνολογία αυτή έχει ήδη αποδείξει την αξία της. Έπειτα, εξετάζουμε την πιο απαιτητική εφαρμογή του SIMD σε δομές ευρετηρίων γενικού σκοπού, όπως τα B+ Trees και τα Adaptive Radix Trees, εστιάζοντας στις βελτιστοποιήσεις εντός κόμβου. Τέλος, εστιάζουμε στο πιο πρόσφατο και λιγότερο διερευνημένο πεδίο, που αφορά τη χρήση του SIMD σε χωρικά ευρετήρια, με ιδιαίτερη έμφαση στο R-tree και τις σύγχρονες προσεγγίσεις επιτάχυνσης των βασικών τελεστών του.

2.2 Χωρικά Ευρετήρια

Η αποδοτική διαχείριση χωρικών δεδομένων προϋποθέτει τη χρήση εξειδικευμένων ευρετηρίων, τα οποία είναι γνωστά ως χωρικά ευρετήρια. Σε αντίθεση με τις κλασικές δομές ευρετηρίων (π.χ. B+-Trees), που υποστηρίζουν μόνο μονοδιάστατα κλειδιά, τα χωρικά ευρετήρια στοχεύουν στην υποστήριξη πολυδιάστατων και γεωμετρικών δεδομένων (σημεία, γραμμές, πολύγωνα) και στην αποδοτική εκτέλεση χωρικών ερωτημάτων, όπως τα ερωτήματα αναζήτησης περιοχής (spatial select queries), οι πλησιέστεροι γείτονες (k-NN queries) και οι χωρικές συζεύξεις (spatial joins) [RSV01].

Στη βιβλιογραφία, τα χωρικά ευρετήρια κατατάσσονται συνήθως σε δύο μεγάλες κατηγορίες: Point Access Methods (PAMs) και Spatial Access Methods (SAMs) [GG98].

2.2.1 Point Access Methods (PAMs)

Τα PAMs έχουν σχεδιαστεί κυρίως για σημειακά δεδομένα ή για αντικείμενα που μπορούν να περιγραφούν μέσω ενός μοναδικού σημείου αναφοράς (π.χ. το κέντρο βάρους). Η βασική τους αρχή είναι ο αναδρομικός διαχωρισμός του χώρου σε υποπεριοχές, δημιουργώντας ιεραρχικές δομές.

- **k-d Trees**: Χωρίζουν τον χώρο επαναληπτικά κατά μήκος ενός άξονα κάθε φορά, παράγοντας ορθογώνιες περιοχές. Είναι ιδιαίτερα αποτελεσματικά για σημειακά δεδομένα χαμηλών διαστάσεων [BEN75].
- **Quadtrees**: Διασπούν τον δισδιάστατο χώρο σε τέσσερις ίσες περιοχές σε κάθε βήμα. Χρησιμοποιούνται ευρέως σε εφαρμογές GIS για τη διαχείριση γεωγραφικών δεδομένων [FB74].

Αν και αποδοτικά για σημεία, τα PAMs δεν είναι κατάλληλα για δεδομένα με έκταση, καθώς η αναπαράσταση ενός αντικειμένου με ένα μόνο σημείο οδηγεί σε απώλεια χωρικής πληροφορίας.

2.2.2 Spatial Access Methods (SAMs)

Τα SAMs σχεδιάστηκαν για δεδομένα με χωρική έκταση. Σε αντίθεση με τα PAMs, που διαμερίζουν τον χώρο, τα SAMs οργανώνουν τα ίδια τα δεδομένα σε χωρικά περιβλήματα (bounding regions), επιτρέποντας την αποδοτική επεξεργασία χωρικών ερωτημάτων.

- **R-tree** [Gut84]: Η πιο διαδεδομένη δομή, που οργανώνει αντικείμενα σε Minimum Bounding Rectangles (MBRs). Τα MBRs αποθηκεύονται σε ιεραρχικούς κόμβους, επιτρέποντας την αποτελεσματική εκτέλεση spatial select queries και spatial joins.
- **R*-tree** [BKS+90]: Βελτιώνει την απόδοση του R-tree ελαχιστοποιώντας την επικάλυψη και το μέγεθος των MBRs με πιο εξελιγμένες στρατηγικές εισαγωγής.
- **Hilbert R-tree** [KF94]: Χρησιμοποιεί καμπύλες χωρικής διαμέρισης, όπως η Hilbert curve, για να διατηρεί καλύτερα τη χωρική εγγύτητα των αντικειμένων.
- **Άλλες δομές**: Grid Files, X-tree, UB-tree, που αναπτύχθηκαν για εξειδικευμένα σενάρια, όπως δεδομένα υψηλών διαστάσεων [BKK96].

2.2.3 Σύνοψη

Συμπερασματικά, τα χωρικά ευρετήρια χωρίζονται σε PAMs, που είναι αποδοτικά για σημειακά δεδομένα, και σε SAMs, που αποτελούν την κυρίαρχη επιλογή για δεδομένα με έκταση. Ανάμεσά τους, το R-tree και οι παραλλαγές του έχουν καθιερωθεί ως οι πιο σημαντικές δομές ευρετηρίασης γενικού σκοπού, και θα αποτελέσουν το βασικό αντικείμενο ανάλυσης στο επόμενο κεφάλαιο.

2.3 Αξιοποίηση του SIMD σε Θεμελιώδεις Τελεστές Βάσεων Δεδομένων

Η ισχύς του SIMD δοκιμάστηκε και αποδείχθηκε αρχικά σε θεμελιώδεις τελεστές (operators) που παρουσιάζουν εγγενώς υψηλό παραλληλισμό, καθώς επεξεργάζονται μεγάλους όγκους δεδομένων με επαναλαμβανόμενες λειτουργίες.

- **Σάρωση και Φιλτράρισμα (Scan & Filtering):** Η σάρωση πινάκων για την εφαρμογή κατηγορημάτων (predicates) ήταν από τα πρώτα πεδία εφαρμογής. Οι Willhalm et al. στο "SIMD-Scan" [WPB+09] παρουσίασαν τεχνικές για τη μαζική σύγκριση τιμών σε στήλες, επιτυγχάνοντας δραματικές επιταχύνσεις. Παράλληλα, οι Li and Patel στο "BitWeaving" [LP13],πρότειναν την κάθετη οργάνωση των bits για την εκτέλεση λογικών πράξεων σε πολλαπλές εγγραφές ταυτόχρονα.
- **Συνένωση (Join):** Οι αλγόριθμοι hash join, ειδικά σε in-memory περιβάλλοντα, ωφελήθηκαν σημαντικά. Οι Blanas et al. [BLP11] και οι Balkesen et al. [BTA+13] διερεύνησαν σε βάθος τη χρήση SIMD για την επιτάχυνση των κρισιμότερων φάσεων του hash join, όπως ο υπολογισμός των hash values (hashing) και η αναζήτηση στις αλυσίδες σύγκρουσης (probing).
- **Ταξινόμηση και Συναθροίσεις (Sorting & Aggregations):** Οι Satish et al. ανέπτυξαν τον "bandwidth oblivious SIMD sort" [KCS+10a], έναν αλγόριθμο ταξινόμησης που εκμεταλλεύεται το SIMD για μαζικές συγκρίσεις και εναλλαγές. Αντίστοιχα, η υλοποίηση συναθροιστικών συναρτήσεων (π.χ., SUM, AVG) με SIMD έχει γίνει πλέον κοινή πρακτική στις σύγχρονες μηχανές βάσεων δεδομένων.

Αυτή η εκτεταμένη βιβλιογραφία αποδεικνύει ότι το SIMD αποτελεί πλέον μια καθιερωμένη και απαραίτητη τεχνολογία για την επίτευξη υψηλής απόδοσης στους βασικούς τελεστές των σύγχρονων DBMS.

2.4 SIMD σε Δομές Ευρετηρίων Γενικού Σκοπού

Η εφαρμογή του SIMD σε δενδρικές δομές ευρετηρίων, όπως τα B+ Trees, είναι πιο απαιτητική. Η διάσχιση ενός δέντρου περιλαμβάνει "pointer chasing" (ακολουθία δεικτών), μια εγγενώς σειριακή λειτουργία. Ως εκ τούτου, η έρευνα επικεντρώθηκε στη βελτιστοποίηση των λειτουργιών εντός ενός κόμβου (intra-node).

- **B+ Trees:** Ο Kenneth A. Ross [Ros04] ήταν ένας από τους πρωτοπόρους που διερεύνησαν πώς οι συνθήκες επιλογής μπορούν να υλοποιηθούν αποδοτικά στη μνήμη, αναλύοντας τη μετατροπή λογικών τελεστών σε bitwise πράξεις για τη μείωση των branch mispredictions, μια τεχνική που αποτελεί προάγγελο των σύγχρονων SIMD αλγορίθμων. Πιο πρόσφατα, οι Schlegel et al. στο "Fast B-Tree Search using SIMD Instructions" [SWL11] έδειξαν πώς η δυαδική αναζήτηση μέσα σε έναν ταξινομημένο κόμβο μπορεί να γίνει πλήρως παράλληλη με SIMD.
- **Adaptive Radix Tree (ART):** Το ART, όπως παρουσιάστηκε από τους Leis et al. [LKN13], είναι μια δενδρική δομή ειδικά σχεδιασμένη για in-memory βάσεις που είναι εκ φύσεως φιλική προς το SIMD. Η δομή των κόμβων του (που μπορεί να έχουν διαφορετικούς τύπους ανάλογα με τον αριθμό των παιδιών) επιτρέπει τη μαζική σύγκριση τμημάτων ενός κλειδιού αναζήτησης με τις εγγραφές του κόμβου, καθιστώντας το εξαιρετικά αποδοτικό.
- **Αρχιτεκτονικά Ευαίσθητες Δομές (Architecture-Sensitive Structures):** Εργασίες όπως το **FAST** [KCS+10] και το **VAST** [YOHY12] εισήγαγαν την ιδέα της συν-σχεδίασης της δομής του ευρετηρίου και των αλγορίθμων του, λαμβάνοντας υπόψη τις ιδιαιτερότητες της CPU, συμπεριλαμβανομένων των καταχωρητών SIMD και της ιεραρχίας της cache.

  Συνολικά, οι έρευνες αυτές στα B+ Trees κατέδειξαν ότι η κύρια ευκαιρία για βελτιστοποίηση με SIMD δεν βρίσκεται στη διάσχιση του δέντρου (pointer chasing), αλλά στην επιτάχυνση της αναζήτησης *εντός* ενός κόμβου, μια αρχή που είναι άμεσα μεταφέρσιμη και στα R-trees.

2.5 SIMD σε Χωρικά Ευρετήρια

Σε αντίθεση με την εκτενή έρευνα που προηγήθηκε, η εφαρμογή του SIMD σε χωρικά ευρετήρια αποτελεί ένα πολύ πιο σύγχρονο και λιγότερο εξερευνημένο πεδίο. Η πολυπλοκότητα των γεωμετρικών ελέγχων (π.χ., έλεγχος επικάλυψης ορθογωνίων) και η δομή των χωρικών ευρετηρίων παρουσίαζαν σημαντικές προκλήσεις.

Οι Rayhan and Aref [RA23], στο paper που αποτελεί τη βάση της παρούσας εργασίας, τονίζουν ρητά ότι η βιβλιογραφία των χωρικών βάσεων δεδομένων είχε σε μεγάλο βαθμό παραβλέψει τη διανυσματοποίηση των σχετικών αλγορίθμων. Ενώ υπήρχαν κάποιες αρχικές αναφορές στη χρήση SIMD για αναζήτηση σε Quad-trees σε μερικώς διανυσματοποιημένη μορφή από τον [Ros04], η έρευνά τους είναι η πρώτη που παρουσιάζει μια ολοκληρωμένη προσέγγιση για το R-tree. Οι βασικές τους συνεισφορές περιλαμβάνουν:

1. Σχεδιασμό SIMD-friendly Layouts : Προτείνουν νέους τρόπους οργάνωσης των MBRs για να επιτρέπουν μαζική φόρτωση και επεξεργασία.
2. Πλήρως Διανυσματοποιημένους Αλγόριθμους: Αναπτύσσουν πλήρως διανυσματοποιημένες εκδόσεις για το spatial select και το spatial join.
3. Βελτιστοποιήσεις Hardware-Aware: Εισάγουν τεχνικές όπως το software prefetching για την απόκρυψη της λανθάνουσας κατάστασης της μνήμης (memory latency).

Η έρευνα αυτή αποδεικνύει ότι υπάρχει τεράστιο περιθώριο βελτίωσης, με αναφερόμενες επιταχύνσεις που φτάνουν το 4x για το spatial select και το 9x για το spatial join.

2.6 Σύνοψη και Τοποθέτηση της Παρούσας Εργασίας

Η βιβλιογραφική επισκόπηση καταδεικνύει μια σαφή πορεία: από τη γενική εφαρμογή του SIMD σε γραμμικούς τελεστές, στην πιο πολύπλοκη αξιοποίησή του σε δενδρικά ευρετήρια γενικού σκοπού, και τέλος, στην πρωτοποριακή και πρόσφατη εφαρμογή του σε χωρικά ευρετήρια όπως το R-tree. Είναι σαφές ότι, ενώ η κοινότητα των βάσεων δεδομένων έχει αγκαλιάσει το SIMD, η εξειδικευμένη υπο-περιοχή των χωρικών ευρετηρίων μόλις τώρα αρχίζει να εξερευνά συστηματικά τις δυνατότητές του.

Η παρούσα διπλωματική εργασία εντάσσεται άμεσα σε αυτή την τελευταία, σύγχρονη ερευνητική κατεύθυνση. Υλοποιώντας και αξιολογώντας πειραματικά τις μεθόδους που προτάθηκαν από τους Rayhan and Aref, η εργασία αυτή στοχεύει να επιβεβαιώσει, να αναλύσει και να επεκτείνει τα ευρήματά τους. Εστιάζοντας ειδικά στον τελεστή spatial select, διερευνά έμπρακτα τον αντίκτυπο των διαφορετικών διατάξεων μνήμης και των βελτιστοποιήσεων στην απόδοση, παρέχοντας μια βαθύτερη κατανόηση του πώς οι

χωρικές δομές δεδομένων μπορούν να επανασχεδιαστούν για να εκμεταλλευτούν πλήρως την ισχύ των σύγχρονων επεξεργαστών.

Κεφάλαιο 3. Το Ευρετήριο R-tree

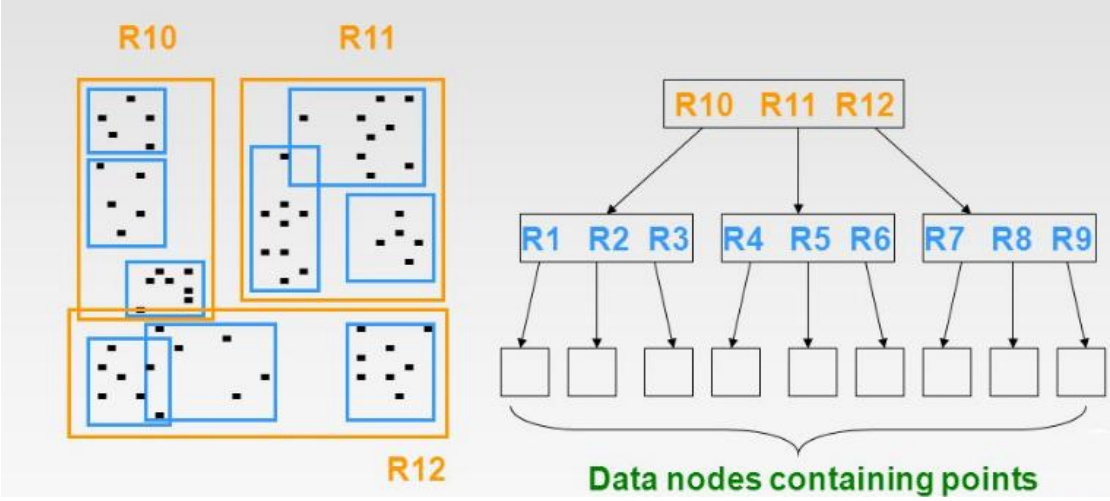
3.1 Εισαγωγή στα Χωρικά Ευρετήρια

Σε σύγχρονες εφαρμογές που διαχειρίζονται γεωγραφικά ή πολυδιάστατα δεδομένα, όπως συστήματα γεωγραφικών πληροφοριών (GIS), σχεδιασμός με υπολογιστή (CAD) και βάσεις δεδομένων πολυμέσων, η αποδοτική ανάκτηση δεδομένων βάσει της θέσης τους στον χώρο είναι κρίσιμη. Οι κλασικές δομές ευρετηρίασης, όπως τα B-trees, είναι σχεδιασμένες για μονοδιάστατα δεδομένα και αποδεικνύονται ανεπαρκείς για την απάντηση πολυδιάστατων ερωτημάτων (π.χ. "βρες όλα τα κτίρια που βρίσκονται εντός της καθορισμένης πλημμυρικής ζώνης").

Για την αντιμετώπιση αυτής της πρόκλησης, αναπτύχθηκαν οι χωρικές δομές δεδομένων, με το R-tree (Rectangle-tree) να αποτελεί μία από τις πιο θεμελιώδεις και ευρέως χρησιμοποιούμενες. Το R-tree, που προτάθηκε από τον Antonin Guttman το 1984 [Gut84], είναι μια δενδρική δομή δεδομένων, επέκταση του B-tree, σχεδιασμένη για την ευρετηρίαση πολυδιάστατων αντικειμένων, όπως σημεία, γραμμές και πολύγωνα. Η κεντρική ιδέα είναι η ιεραρχική ομαδοποίηση κοντινών αντικειμένων, όπου κάθε ομάδα αναπαρίσταται στον επόμενο, υψηλότερο επίπεδο του δέντρου από το Ελάχιστο Περιβάλλον Ορθογώνιο (Minimum Bounding Rectangle - MBR) που την περικλείει.

3.2 Δομή του R-tree

Το R-tree είναι ένα ισοζυγισμένο ως προς το ύψος δένδρο, παρόμοια με το B-tree, το οποίο διασφαλίζει ότι όλες οι διαδρομές από τη ρίζα προς οποιοδήποτε φύλλο έχουν το ίδιο μήκος. Αυτή η ιδιότητα εγγυάται ότι οι επιδόσεις των λειτουργιών δεν υποβαθμίζονται σε έναν συγκεκριμένο κλάδο του δέντρου, παρέχοντας προβλέψιμο χρόνο αναζήτησης.



Εικόνα 5: Δομή R-tree [GSDQ]

3.2.1 Κόμβοι και Εγγραφές

Το δένδρο αποτελείται από δύο τύπους κόμβων: τους εσωτερικούς κόμβους και τους κόμβους-φύλλα.

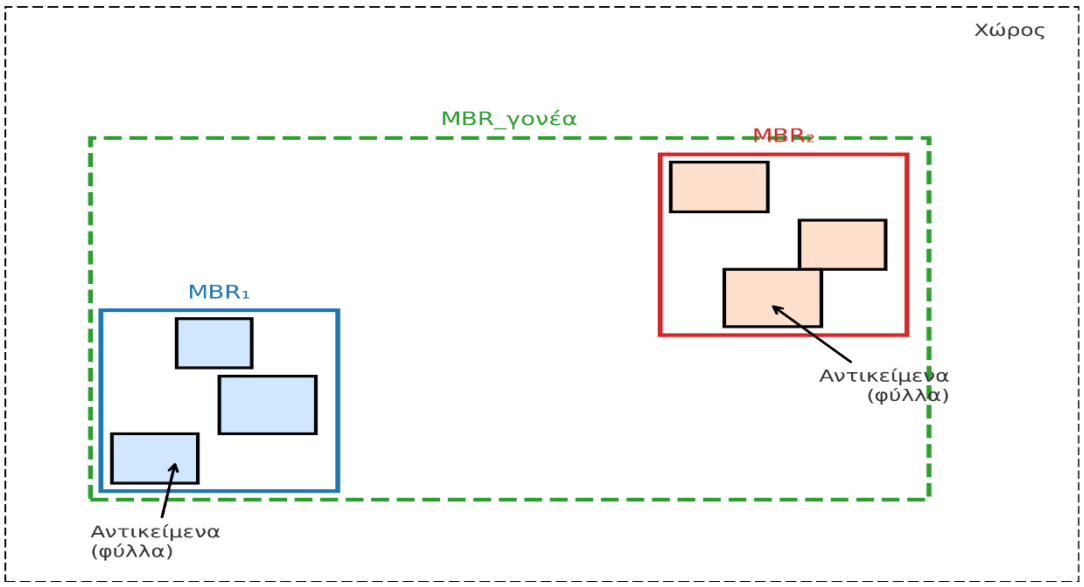
- Κόμβοι-Φύλλα (Leaf Nodes): Βρίσκονται στο χαμηλότερο επίπεδο του δέντρου και περιέχουν εγγραφές της μορφής (MBR, object_id) ή (Point,object_id). Το MBR είναι το ελάχιστο περιβάλλον ορθογώνιο του αντικειμένου και το object_id είναι ένας δείκτης προς τα πλήρη δεδομένα του αντικειμένου, τα οποία είναι αποθηκευμένα στη βάση δεδομένων. Η άλλη εκδοχή είναι στα φύλλα να είναι αποθηκευμένα τα σημεία(Point) μαζί με ένα id αντίστοιχο με την προηγούμενη περίπτωση.

- Εσωτερικοί Κόμβοι (Internal Nodes): Κάθε εσωτερικός κόμβος περιέχει εγγραφές της μορφής (MBR, child_ptr). Το child_ptr είναι ένας δείκτης προς έναν κόμβο-παιδί στο αμέσως κατώτερο επίπεδο, και το MBR είναι το ελάχιστο ορθογώνιο που περικλείει όλα τα MBR των εγγραφών του συγκεκριμένου κόμβου-παιδιού.

Κάθε κόμβος έχει μια προκαθορισμένη χωρητικότητα, δηλαδή μπορεί να περιέχει έναν μεταβλητό αριθμό εγγραφών, από m έως M. Η παράμετρος M ορίζει τη μέγιστη χωρητικότητα, ενώ η m (όπου $m \leq M/2$) ορίζει την ελάχιστη, εξασφαλίζοντας έτσι την ισορροπία και την αποδοτική χρήση του χώρου. Η ρίζα αποτελεί εξαίρεση, καθώς μπορεί να έχει λιγότερες από m εγγραφές.

3.2.2 Το Ελάχιστο Περιβάλλον Ορθογώνιο (MBR)

Το MBR είναι το δομικό στοιχείο του R-tree. Είναι το μικρότερο δυνατό ορθογώνιο, με πλευρές παράλληλες προς τους άξονες, το οποίο περικλείει πλήρως ένα χωρικό αντικείμενο ή μια συλλογή από MBRs. Η χρήση του απλοποιεί δραστικά τα χωρικά ερωτήματα: αντί για πολύπλοκους υπολογισμούς επικάλυψης μεταξύ ακανόνιστων πολυγώνων, το σύστημα εκτελεί ταχύτατους ελέγχους επικάλυψης μεταξύ ορθογωνίων.



Εικόνα 6: MBR

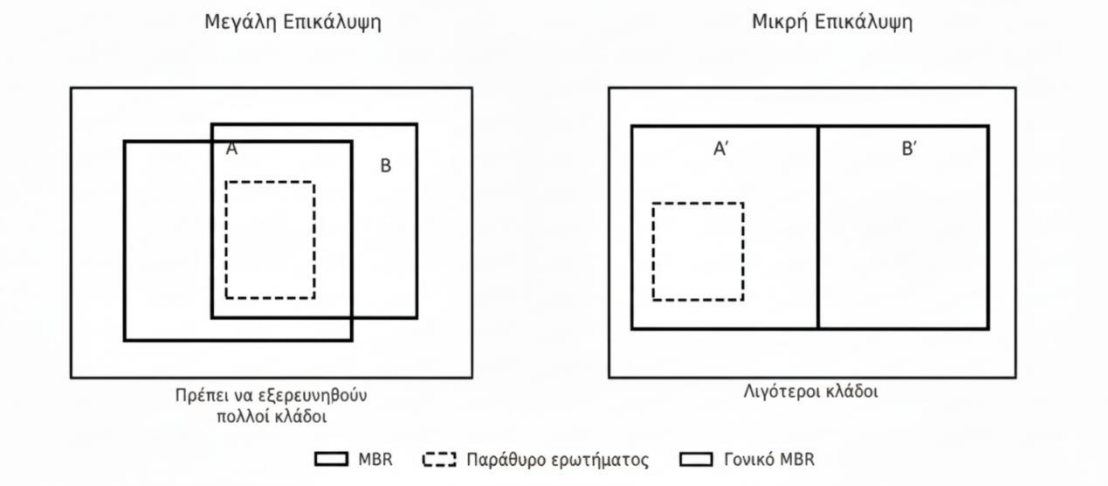
3.2.3 Κριτήρια Ποιότητας ενός R-tree

Η απόδοση ενός R-tree εξαρτάται άμεσα από την ποιότητα της ομαδοποίησης των ορθογωνίων του. Δύο βασικά κριτήρια καθορίζουν αυτή την ποιότητα:

1. Κάλυψη (Coverage): Η συνολική περιοχή που καλύπτεται από όλα τα MBRs σε έναν κόμβο. Στόχος είναι η ελαχιστοποίηση της κάλυψης (ή "νεκρής περιοχής"), δηλαδή της περιοχής εντός ενός MBR που δεν καταλαμβάνεται από τα MBRs των παιδιών του. Μικρότερη κάλυψη σημαίνει ότι τα MBRs είναι πιο "σφιχτά", μειώνοντας την πιθανότητα να ακολουθήσει μια αναζήτηση ένα μονοπάτι χωρίς λόγο.

2. Επικάλυψη (Overlap): Η περιοχή που μοιράζονται δύο ή περισσότερα MBRs στον ίδιο κόμβο. Η ελαχιστοποίηση της επικάλυψης είναι κρίσιμη, καθώς μεγάλη

επικάλυψη αναγκάζει ένα ερώτημα αναζήτησης να εξερευνήσει πολλαπλούς κλάδους του δέντρου, αυξάνοντας δραματικά τον χρόνο απόκρισης.



Εικόνα 7: Περιπτώσεις Overlap

Ένα βέλτιστο R-tree, λοιπόν, είναι αυτό που ελαχιστοποιεί ταυτόχρονα την κάλυψη και την επικάλυψη.

3.3 Λειτουργίες στο R-tree

3.3.1 Αναζήτηση (Search)

Αλγόριθμος 1: Ψευδοκώδικας για τον Αλγόριθμο Αναζήτησης

Input: R-tree T, search rectangle R

Output: set of id (S) of objects intersecting R

```
1   S ← ∅
2       PROCEDURE RANGEQUERY (N, R)
3           IF leaf(N) THEN
4               FOR EACH (MBR, id) IN entries(N) DO
5                   IF MBR INTERSECTS R THEN ADD id TO S
6           ELSE
7               FOR EACH (MBR, ptr) IN entries(N) DO
8                   IF MBR INTERSECTS R THEN
                        RANGEQUERY (ptr, R)
9       RANGEQUERY (root(T), R)
10      RETURN S
```

Ο ψευδοκώδικας αποτυπώνει ένα αλγόριθμο ο οποίος βρίσκει τα αντικείμενα που τέμνουν ένα ορθογώνιο R διασχίζοντας το R-tree από τη ρίζα προς τα κάτω. Σε κάθε κόμβο εξετάζει μόνο τις εγγραφές των οποίων το ελάχιστο περιβάλλον ορθογώνιο (MBR) τέμνεται με το R, ενώ αγνοεί όλα τα υπόλοιπα υποδέντρα, έτσι μειώνει δραστικά τον χώρο αναζήτησης. Όταν φτάνει σε φύλλο, ελέγχει με ακρίβεια τα αντικείμενα και προσθέτει στο αποτέλεσμα τα αναγνωριστικά όσων πράγματι τέμνουν το R. Η ορθότητα διασφαλίζεται επειδή τυχόν «ψευδώς θετικές» περιπτώσεις από τα MBR απορρίπτονται στο επίπεδο των φύλλων. Η ταχύτητα προκύπτει από την απόρριψη των μη συμβατών υποδεντρων: το κόστος εξαρτάται από τους λίγους κόμβους που όντως τέμνονται με το R, όχι από όλα τα δεδομένα. Η ίδια ιδέα μπορεί να γραφτεί και επαναληπτικά με χρήση στοίβας.

3.3.2 Εισαγωγή (Insertion)

Η εισαγωγή ενός νέου αντικειμένου περιλαμβάνει δύο βασικά στάδια:

1. Επιλογή Φύλλου (ChooseLeaf): Η διαδικασία ξεκινά από τη ρίζα και σε κάθε επίπεδο επιλέγεται ο υποκόμβος του οποίου το MBR απαιτεί τη μικρότερη αύξηση εμβαδού (area enlargement) για να συμπεριλάβει το νέο αντικείμενο. Αυτή η ευριστική προσπαθεί να διατηρήσει τα MBRs όσο το δυνατόν πιο μικρά. Η διαδικασία συνεχίζεται μέχρι να φτάσει σε έναν κόμβο-φύλλο.

2. Διαίρεση Κόμβου (Node Splitting): Το νέο αντικείμενο εισάγεται στο επιλεγμένο φύλλο. Αν το φύλλο δεν είναι γεμάτο (έχει λιγότερες από M εγγραφές), η διαδικασία ολοκληρώνεται. Αν ο κόμβος υπερχειλίσει (πλέον περιέχει M+1 εγγραφές), πρέπει να διαιρεθεί σε δύο νέους κόμβους. Η διαίρεση είναι το πιο κρίσιμο σημείο της εισαγωγής. Ο στόχος είναι να διαμοιραστούν οι M+1 εγγραφές στους δύο νέους κόμβους με τρόπο που να ελαχιστοποιεί την επικάλυψη και το συνολικό εμβαδόν. Οι πιο γνωστοί αλγόριθμοι διαίρεσης είναι:

    - Linear Split (Γραμμικός), γρήγορος, λιγότερο «βέλτιστος»
        - Επιλογή άξονα: για κάθε άξονα (x, y) υπολόγισε τη μέγιστη κανονικοποιημένη απόσταση ανάμεσα στα όρια των MBRs και διάλεξε τον άξονα με τη μεγαλύτερη διάκριση.

- Σπόροι: επί του επιλεγμένου άξονα, διάλεξε το ζεύγος MBRs που απέχει περισσότερο (μεγαλύτερη απόσταση μεταξύ πλησιέστερων ακμών) αυτά γίνονται οι δύο «σπόροι».
- Κατανομή υπολοίπων: κάθε υπόλοιπη εγγραφή πηγαίνει στον κόμβο που απαιτεί τη μικρότερη αύξηση εμβαδού του MBR του. Σε ισοβαθμία προτιμάται το μικρότερο τελικό εμβαδό (ή/και περίμετρο).Πολυπλοκότητα περίπου O(M).

- Quadratic Split (Τετραγωνικός), ακριβέστερος, ακριβότερος υπολογιστικά
  - Σπόροι: επίλεξε το ζεύγος MBRs που, αν μπει στο ίδιο παιδί, θα δημιουργούσε τη μεγαλύτερη «σπατάλη» χώρου (δηλ. τη μεγαλύτερη αύξηση εμβαδού του ελάχιστου περιβάλλοντος ορθογωνίου σε σχέση με το άθροισμα των εμβαδών τους). Αυτό το ζεύγος γίνεται οι δύο «σπόροι».
  - Κατανομή υπολοίπων: για κάθε υπόλοιπη εγγραφή, επέλεξε τον κόμβο που ελαχιστοποιεί την αύξηση εμβαδού· αν υπάρξει ισοβαθμία, προτίμησε μικρότερη επικάλυψη και κατόπιν μικρότερο εμβαδό. Πολυπλοκότητα περίπου O(M^2).

Μετά τη διαίρεση, τα MBRs των γονικών κόμβων πρέπει να ενημερωθούν αναδρομικά προς τα πάνω, μέχρι τη ρίζα. Αν η διαίρεση φτάσει μέχρι τη ρίζα, τότε δημιουργείται μια νέα ρίζα και το ύψος του δέντρου αυξάνεται κατά ένα.

3.3.3 Διαγραφή (Deletion)

Η διαγραφή ενός αντικειμένου περιλαμβάνει την εύρεση και αφαίρεσή του, καθώς και τη διατήρηση της δομής του δέντρου.

1. Εύρεση Κόμβου (FindLeaf): Το δέντρο διασχίζεται από τη ρίζα για να εντοπιστεί ο κόμβος-φύλλο που περιέχει το προς διαγραφή αντικείμενο.

2. Αφαίρεση: Η εγγραφή αφαιρείται από τον κόμβο-φύλλο.

3. Διαχείριση Υποχείλισης (Handling Underflow): Αν μετά τη διαγραφή ο κόμβος περιέχει λιγότερες από m εγγραφές (υποχείλιση), πρέπει να γίνει αναδιοργάνωση. Οι συνήθεις στρατηγικές είναι:

   - Διαγραφή και Επανεισαγωγή (Delete and Re-insert): Ο κόμβος διαγράφεται και όλες οι εναπομείνασες εγγραφές του επανεισάγονται στο δέντρο από την αρχή. Αυτή η μέθοδος μπορεί να βελτιώσει δυναμικά την ποιότητα του δέντρου.

- Συγχώνευση (Merge): Ο υπογεμισμένος κόμβος συγχωνεύεται με έναν γειτονικό (αδελφό) κόμβο.

4. Ενημέρωση Προγόνων: Όπως και στην εισαγωγή, οι αλλαγές στα MBRs προπαγίδευονται προς τα πάνω, συρρικνώνοντας τα ορθογώνια των γονικών κόμβων όπου χρειάζεται.

3.4 Μαζική Φόρτωση: Ο Αλγόριθμος STR

Η κατασκευή ενός R-tree με διαδοχικές εισαγωγές μπορεί να είναι χρονοβόρα σε μεγάλα σύνολα δεδομένων και να καταλήγει σε δείκτη χαμηλής ποιότητας (υψηλή επικάλυψη, χαμηλή πληρότητα κόμβων). Οι αλγόριθμοι μαζικής φόρτωσης (bulk loading) αντιμετωπίζουν το πρόβλημα χτίζοντας το δέντρο εξ αρχής, με τρόπο που επιβάλλει καλή χωρική ομαδοποίηση και υψηλή πληρότητα φύλλων. Ένας από τους αποδοτικότερους είναι ο STR (Sort-Tile-Recursive), ο οποίος κατασκευάζει ένα «packed» R-tree από κάτω προς τα πάνω (bottom-up): τα φύλλα γεμίζουν συστηματικά, η επικάλυψη μεταξύ κόμβων μειώνεται σε σχέση με ένα βασικό R-tree και, ως συνέπεια της συσκευασμένης κατασκευής, το δέντρο θεωρείται στατικό (μετά την κατασκευή δεν υποστηρίζονται δυναμικές προσθήκες/διαγραφές χωρίς αναδόμηση). Δοθέντων N αντικειμένων με MBRs και χωρητικότητας κόμβου C, ο STR:

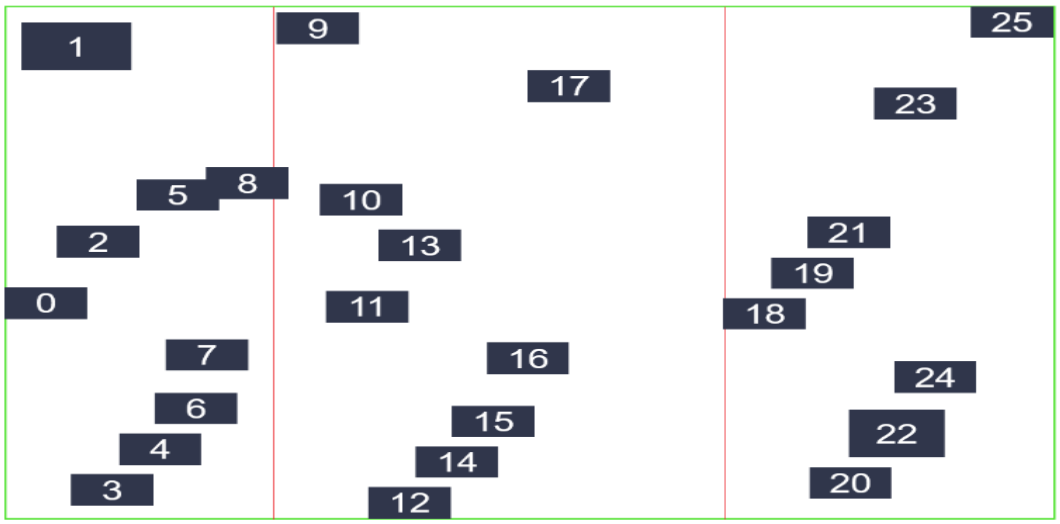
1. Sort: ταξινομεί τα αντικείμενα κατά x του κέντρου,

2. Tile: «κόβει» τη (ήδη ταξινομημένη) ακολουθία σε κατακόρυφες λωρίδες και, μέσα σε κάθε λωρίδα, ταξινομεί κατά y,

3. Pack/Recursive: ομαδοποιεί σε ομάδες μεγέθους C για να σχηματίσει φύλλα στη συνέχεια επαναλαμβάνει αναδρομικά στα MBRs των νέων κόμβων έως ότου παραχθεί η ρίζα.

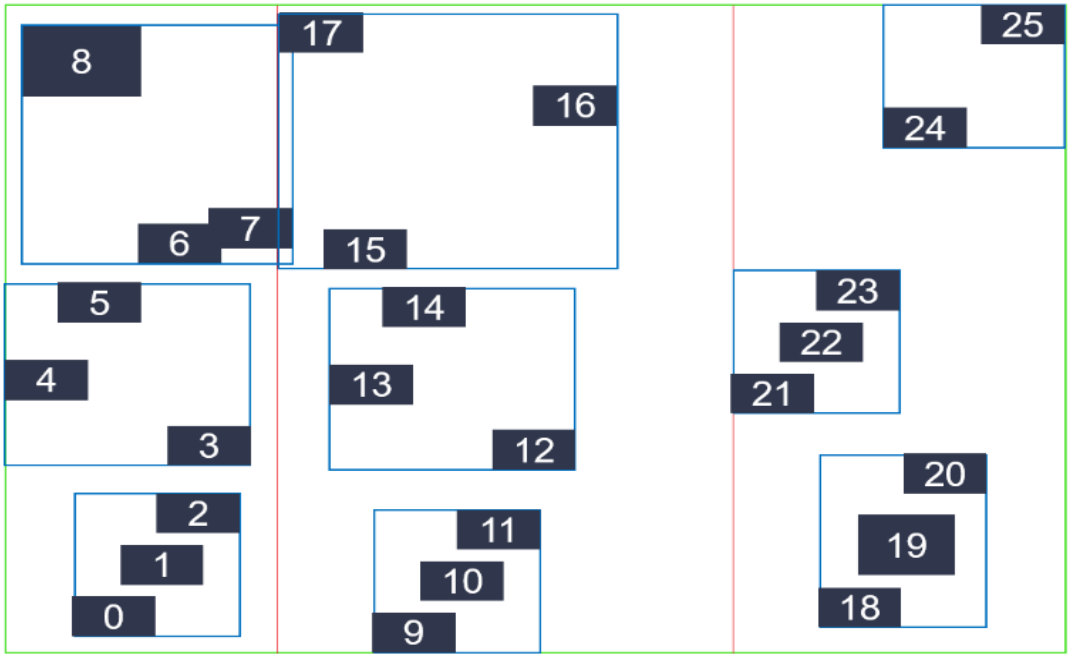
Σε μια πρακτική εκδοχή για δισδιάστατα σημεία, η διάσπαση μπορεί να χρησιμοποιεί περίπου $\sqrt{(N/r)}$ λωρίδες (με r τον μέγιστο αριθμό παιδιών ανά γονέα):

(i)



Εικόνα 8: Ταξινόμηση όλων ως προς x [Xilinx]

(ii)



Εικόνα 9: Διαμερίση σε √(N/r) μέρη και ταξινόμηση κάθε μέρους ως προς y [Xilinx]

(iii) συσκευασία ανά r στοιχεία σε γονείς και

(iv) επανάληψη μέχρι να απομείνει ένας γονέας (η ρίζα).

Σε ροές πολύ μεγάλης κλίμακας, η κρίσιμη φάση είναι η ταξινόμηση. Μια υλοποίηση περιγράφει δύο στάδια για αποδοτική ταξινόμηση όταν το σύνολο υπερβαίνει περίπου 16 εκατομμύρια εγγραφές: blockSort (διαίρεση σε blocks και τοπική ταξινόμηση) και mergeTreeSort (συγχώνευση των ήδη ταξινομημένων blocks με σταθερή ροή εξόδου).

Αυτές οι τεχνικές στοχεύουν υψηλή διαμεταγωγή και καλή αξιοποίηση μνήμης/πόρων κατά τη μαζική φόρτωση του δείκτη.

Ο STR τυπικά παράγει R-trees με εξαιρετική χωρική ομαδοποίηση και μικρή επικάλυψη μεταξύ MBRs γειτονικών κόμβων, βελτιστοποιώντας την πληρότητα των φύλλων και επιταχύνοντας αισθητά ερωτήματα τύπου range, k-NN και spatial join σε σχέση με δέντρα που έχουν χτιστεί με απλές σειριακές εισαγωγές. Η βελτίωση αυτή αντανακλάται και σε πρακτικές υλοποιήσεις, όπου η καλή «συσκευασία» του χώρου στα χαμηλά επίπεδα του δέντρου μειώνει δραστικά τον αριθμό κόμβων που πρέπει να επισκεφθεί ο εκτελεστής ερωτημάτων. Η ανάλυση του STR αλγορίθμου βασίστηκε

3.5 Παραλλαγές του R-tree

Η αρχική δομή του R-tree [Gut84] αποτέλεσε τη βάση για πολλές βελτιωμένες παραλλαγές. Οι πιο σημαντικές από αυτές είναι τα R*-tree, R^+-tree και Hilbert R-tree.

R*-tree (R-star tree)

Το R*-tree προτάθηκε από τους Beckmann, Kriegel, Schneider και Seeger [BKS+90]. Εισάγει πιο εξελιγμένες ευριστικές για την εισαγωγή και τη διαίρεση κόμβων, με στόχο τη δημιουργία ενός δέντρου υψηλότερης ποιότητας.

- Βελτιωμένη επιλογή υποδέντρου: Κατά την εισαγωγή, αντί να επιλέγεται απλώς το υποδέντρο με τη μικρότερη αύξηση εμβαδού, το R*-tree εξετάζει και άλλους παράγοντες, όπως η αύξηση της επικάλυψης, ειδικά για τους εσωτερικούς κόμβους.

- Forced Re-insertion: Όταν ένας κόμβος πρόκειται να διαιρεθεί, το R*-tree προσπαθεί πρώτα να αποφύγει τη διαίρεση. Επανεισάγει ένα ποσοστό (π.χ. 30%) των εγγραφών του κόμβου στο δέντρο. Αυτό βοηθά στην τοποθέτηση αντικειμένων σε πιο κατάλληλους κόμβους και συχνά οδηγεί σε καλύτερη συνολική δομή και μεγαλύτερη πληρότητα κόμβων.

- Έξυπνη διαίρεση: Ο αλγόριθμος διαίρεσης δεν ελαχιστοποιεί μόνο το εμβαδόν, αλλά και την επικάλυψη μεταξύ των νέων κόμβων, εξετάζοντας πολλαπλούς άξονες και τρόπους διαμοιρασμού.

Οι βελτιώσεις αυτές καθιστούν το R*-tree σημαντικά ταχύτερο στην πράξη για τα περισσότερα σενάρια αναζήτησης, αν και το κόστος κατασκευής του είναι ελαφρώς υψηλότερο.

R^+-tree

Το R^+-tree παρουσιάστηκε από τους Sellis, Roussopoulos και Faloutsos [SRF87]. Η βασική του διαφοροποίηση είναι ότι αποφεύγει εντελώς την επικάλυψη μεταξύ MBRs (Minimum Bounding Rectangles) στους εσωτερικούς κόμβους. Για να το επιτύχει αυτό, επιτρέπει σε ένα αντικείμενο να αποθηκεύεται σε περισσότερα από ένα φύλλα (duplication).

- Πλεονέκτημα: Οι point queries γίνονται πιο αποδοτικές, καθώς δεν χρειάζεται να ακολουθούνται πολλαπλές διαδρομές.
- Μειονέκτημα: Αυξημένο μέγεθος του δέντρου και πιθανή υποβάθμιση της απόδοσης σε ενημερώσεις.

Hilbert R-tree

Το Hilbert R-tree προτάθηκε από τους Kamel και Faloutsos [KF94]. Βασίζεται στη χρήση της καμπύλης Hilbert (Hilbert space-filling curve), η οποία επιβάλλει μια γραμμική διάταξη στα πολυδιάστατα δεδομένα.

- Packed Hilbert R-tree: Προσανατολισμένο σε στατικά δεδομένα, με υψηλή πληρότητα κόμβων (συχνά κοντά στο 100%).
- Dynamic Hilbert R-tree: Υποστηρίζει δυναμικά δεδομένα, χρησιμοποιώντας deferred splitting (π.χ. 2-to-3 splits) και την έννοια των “cooperating siblings” για βελτίωση της απόδοσης και της χωρικής ομαδοποίησης.

Σύμφωνα με συγκριτικές μελέτες [BKK96], η απόδοση των παραλλαγών του R-tree ποικίλει ανάλογα με το είδος των δεδομένων και το είδος του ερωτήματος:

- το Hilbert R-tree υπερέχει σε περιπτώσεις με δυναμικά δεδομένα,
- το R*-tree προσφέρει τη βέλτιστη ισορροπία σε γενικές χρήσεις,
- ενώ το R^+-tree είναι ιδιαίτερα αποδοτικό σε point queries, με κόστος αυξημένης αποθήκευσης.

Συμπερασματικά οι τεχνικές αυτές οδηγούν σε αποδοτικότερη αναζήτηση, ιδιαίτερα για μεγάλα datasets με πολυδιάστατη πληροφορία.

3.6 Σύνοψη Κεφαλαίου

Σε αυτό το κεφάλαιο, αναλύσαμε σε βάθος τη θεμελιώδη χωρική δομή ευρετηρίασης, το R-tree. Εξετάσαμε τη δενδρική δομή του, που βασίζεται στην ιεραρχική ομαδοποίηση αντικειμένων μέσω Ελαχίστων Περιβαλλόντων Ορθογωνίων (MBRs). Αναλύσαμε τις βασικές λειτουργίες της αναζήτησης, της εισαγωγής και της διαγραφής, δίνοντας έμφαση στις προκλήσεις, όπως η διαίρεση κόμβων. Παρουσιάσαμε τη μέθοδο μαζικής φόρτωσης STR ως έναν αποδοτικό τρόπο κατασκευής βελτιστοποιημένων δέντρων και, τέλος, αναφερθήκαμε σε σημαντικές παραλλαγές όπως το R*-tree. Με την επικράτηση των in-memory βάσεων δεδομένων, όπου το κόστος προσπέλασης του δίσκου εκμηδενίζεται, ο παράγοντας που καθορίζει την τελική απόδοση είναι πλέον η υπολογιστική ισχύς της CPU και η ταχύτητα προσπέλασης της μνήμης RAM. Η κατανόηση αυτών των μηχανισμών είναι απαραίτητη, καθώς αποτελεί τη βάση πάνω στην οποία θα εξεταστούν οι τεχνικές βελτιστοποίησης με χρήση εντολών SIMD στο επόμενο κεφάλαιο.

Κεφάλαιο 4. Σχεδιασμός & Υλοποίηση

Στο κεφάλαιο αυτό παρουσιάζεται ο σχεδιασμός και οι τεχνικές λεπτομέρειες της υλοποίησης που αναπτύχθηκε στο πλαίσιο της παρούσας εργασίας. Αρχικά, αναλύεται η σημασία της διάταξης των δεδομένων στη μνήμη για την αποδοτική χρήση εντολών SIMD. Στη συνέχεια, περιγράφεται η γενική αρχιτεκτονική του συστήματος, συμπεριλαμβανομένης της μεθόδου κατασκευής του R-tree και του αλγορίθμου αναζήτησης. Τέλος, εξετάζονται αναλυτικά τρεις διαφορετικές διατάξεις αποθήκευσης που υλοποιήθηκαν, οι σχεδιαστικές επιλογές που έγιναν και οι αποκλίσεις από την πρότυπη υλοποίηση του paper [RA23].

4.1 Η ανάγκη διάταξης δεδομένων στη μνήμη

Η μετάβαση από τις παραδοσιακές βάσεις δεδομένων που περιορίζονται από την ταχύτητα του δίσκου σε συστήματα που λειτουργούν εξ ολοκλήρου στη μνήμη (in-memory) έχει μετατοπίσει το κέντρο βάρους της βελτιστοποίησης από τις λειτουργίες I/O στην αποδοτική χρήση της CPU και της ιεραρχίας της μνήμης. Όπως αναλύθηκε στο Κεφάλαιο 2, οι σύγχρονες αρχιτεκτονικές επεξεργαστών προσφέρουν τεράστια υπολογιστική ισχύ μέσω του παραλληλισμού σε επίπεδο δεδομένων με εντολές SIMD (Single Instruction, Multiple Data). Ωστόσο, η αξιοποίηση αυτής της ισχύος δεν είναι αυτόματη. Οι κλασικές δομές δεδομένων, όπως το R-tree, συχνά οργανώνουν τις πληροφορίες με τρόπους που δεν είναι «φιλικοί» προς τις διανυσματικές λειτουργίες, εμποδίζοντας την αποτελεσματική εφαρμογή του SIMD. Για να μπορέσει μια εντολή SIMD να επεξεργαστεί ταυτόχρονα 8 ή 16 τιμές κινητής υποδιαστολής (AVX-256 ή AVX-512), αυτές οι τιμές πρέπει ιδανικά να βρίσκονται σε συνεχόμενες θέσεις στη μνήμη. Μια συμβατική υλοποίηση R-tree, όπου οι τέσσερις συντεταγμένες ενός MBR αποθηκεύονται μαζί ως ένα αντικείμενο, καθιστά αυτή τη διαδικασία αναποτελεσματική, καθώς απαιτεί πολλαπλές, ασυνεχείς προσβάσεις στη μνήμη για τη συλλογή των δεδομένων που χρειάζονται για μία μόνο διανυσματική πράξη. Για τον λόγο αυτό, ο σχεδιασμός της κατάλληλης διάταξης των δεδομένων στη μνήμη (memory layout) είναι εξίσου κρίσιμος με τον σχεδιασμό του ίδιου του αλγορίθμου. Στο παρόν κεφάλαιο περιγράφεται αναλυτικά η διαδικασία υλοποίησης των δομών και των αλγορίθμων που αποτέλεσαν αντικείμενο μελέτης της παρούσας διπλωματικής. Στόχος είναι να παρουσιαστεί πώς οι

θεωρητικές προσεγγίσεις που αναλύθηκαν στο προηγούμενο κεφάλαιο μετατράπηκαν σε λογισμικό και να αναδειχθούν οι σχεδιαστικές αποφάσεις που ελήφθησαν σε κάθε στάδιο.Αρχικά παρουσιάζεται η γενική αρχιτεκτονική που ακολουθήθηκε και οι κοινές επιλογές που εφαρμόστηκαν σε όλες τις υλοποιήσεις, όπως η μέθοδος κατασκευής του ευρετηρίου μέσω bulk loading με τον αλγόριθμο STR και η χρήση SIMD εντολών για την επιτάχυνση των πράξεων spatial select. Στη συνέχεια, αναλύονται διαδοχικά οι τρεις διαφορετικές διατάξεις μνήμης (D0, D1, D2), οι οποίες αποτελούν παραλλαγές της βασικής δομής R-tree και διαφοροποιούνται στον τρόπο αποθήκευσης των εσωτερικών κόμβων. Για κάθε διάταξη περιγράφονται οι αντίστοιχες δομές δεδομένων, ο τρόπος υλοποίησης της μεθόδου spatial select, καθώς και οι βελτιστοποιήσεις που εφαρμόστηκαν με χρήση SIMD intrinsics.

4.2 Γενική Αρχιτεκτονική Υλοποίησης

Πριν από την παρουσίαση των τριών διαφορετικών διατάξεων μνήμης (D0, D1 και D2), είναι απαραίτητο να παρουσιαστούν οι θεμελιώδεις σχεδιαστικές αποφάσεις που εφαρμόστηκαν οριζόντια σε όλες τις υλοποιήσεις. Οι αποφάσεις αυτές αφορούν τόσο τον τρόπο κατασκευής του ευρετηρίου όσο και τις τεχνικές επιτάχυνσης που χρησιμοποιήθηκαν κατά την εκτέλεση ερωτημάτων. Οι δύο αυτές επιλογές αποτέλεσαν τον κοινό άξονα πάνω στον οποίο βασίστηκαν οι διαφορετικές υλοποιήσεις που αναλύονται στα επόμενα υπο-κεφάλαια.

4.2.1 Κατασκευή του R-tree

Για την αρχική δημιουργία του ευρετηρίου, αντί για διαδοχικές εισαγωγές που μπορεί να οδηγήσουν σε ένα μη βέλτιστο δέντρο, επιλέχθηκε η μέθοδος μαζικής φόρτωσης (bulk loading). Συγκεκριμένα, υλοποιήθηκε ο αλγόριθμος STR (Sort-Tile-Recursive), ο οποίος κατασκευάζει το δέντρο από κάτω προς τα πάνω (bottom-up), διασφαλίζοντας υψηλή ποιότητα και απόδοση, όπως αναλύθηκε στο Κεφάλαιο 3 .Στο πλαίσιο της παρούσας εργασίας, κατασκευάστηκαν τρία διαφορετικά R-trees, ένα για κάθε διάταξη μνήμης όπως αναφέρεται στην προσέγγιση των Rayhan and Aref [RA23]. Ο αλγόριθμος STR αποτέλεσε την κοινή βάση για την κατασκευή και των τριών, διασφαλίζοντας ότι η αρχική χωρική οργάνωση των δεδομένων είναι πανομοιότυπη. Η μόνη διαφοροποίηση εντοπίζεται στη δομή αποθήκευσης των εσωτερικών κόμβων (Internal Nodes) σε κάθε δέντρο.

Η διαδικασία κατασκευής ακολουθεί τα παρακάτω στάδια:

1. **Δημιουργία Επιπέδου Φύλλων (Leaf Level Creation)**:
    - Αρχικά, το σύνολο των δεδομένων (σημεία) ταξινομείται με βάση τη συντεταγμένη x.
    - Στη συνέχεια, τα ταξινομημένα δεδομένα χωρίζονται σε ceil(sqrt(N/C)) κατακόρυφες λωρίδες (strips), όπου N είναι το πλήθος των σημείων και C η χωρητικότητα ενός κόμβου (fanout).
    - Κάθε λωρίδα ταξινομείται εσωτερικά με βάση τη συντεταγμένη y.
    - Τέλος, τα σημεία ομαδοποιούνται σε κόμβους-φύλλα (Leaf Nodes), οι οποίοι έχουν πανομοιότυπη δομή και για τις τρεις υλοποιήσεις. Για κάθε κόμβο-φύλλο που δημιουργείται, υπολογίζεται το Ελάχιστο Περιβάλλον Ορθογώνιο (MBR) που περικλείει όλα τα σημεία του.
2. **Δημιουργία Εσωτερικών Επιπέδων (Internal Levels Creation)**:
    - Αυτό το στάδιο εκτελείται επαναληπτικά, ξεκινώντας από τα MBRs των κόμβων-φύλλων. Σε κάθε επανάληψη, τα MBRs του τρέχοντος επιπέδου ομαδοποιούνται για να δημιουργήσουν τους γονικούς κόμβους του αμέσως ανώτερου επιπέδου.
    - Για τη δημιουργία του MBR ενός γονικού κόμβου, ο αλγόριθμος πρέπει να βρει τα ακραία όρια όλων των MBRs των παιδιών του. Αυτό επιτυγχάνεται μέσω μιας σειράς συγκρίσεων: για κάθε MBR-παιδί, οι συντεταγμένες min_x, min_y, max_x και max_y συγκρίνονται με τις τρέχουσες ακραίες τιμές του υπό διαμόρφωση γονικού MBR. Στο τέλος, το γονικό MBR θα έχει ως min_x τη μικρότερη από όλες τις min_x των παιδιών του, ως max_x τη μεγαλύτερη από όλες τις max_x, και ούτω καθεξής.
    - Σε αυτό ακριβώς το σημείο διαφοροποιείται η υλοποίηση για κάθε διάταξη (layout) και θα αναλυθεί στην ενότητα 4.3

Η διαδικασία αυτή επαναλαμβάνεται, δημιουργώντας διαδοχικά ανώτερα επίπεδα, μέχρις ότου ο αριθμός των κόμβων σε ένα επίπεδο να είναι τόσο μικρός ώστε να χωρούν όλοι σε έναν μοναδικό κόμβο. Αυτός ο τελευταίος κόμβος αποτελεί τη ρίζα (root) του R-tree, ολοκληρώνοντας την κατασκευή.

4.2.2 Κατασκευή Μεθόδου Αναζήτησης (Spatial Select)

Ο αλγόριθμος για την εκτέλεση των ερωτημάτων εύρους (spatial select) υλοποιήθηκε με μια μη αναδρομική διάσχιση κατά πλάτος (Breadth-First Search - BFS), χρησιμοποιώντας μια δομή ουράς (queue). Αυτή η προσέγγιση επιλέχθηκε αντί της αναδρομικής Depth-First Search (DFS) καθώς, όπως επισημαίνεται και στο paper [RA23], είναι πιο φιλική προς βελτιστοποιήσεις όπως το software prefetching και γενικότερα επιτρέπει καλύτερο έλεγχο της ροής εκτέλεσης καθώς επιτρέπει την προσπέλαση των διευθύνσεων των επόμενων κόμβων και την έκδοση εντολών prefetch πριν αυτοί επεξεργαστούν. Η διαδικασία είναι η εξής:

1. Αρχικοποίηση: Μια κενή ουρά δημιουργείται και ο κόμβος-ρίζα του R-tree εισάγεται σε αυτήν.
2. Επανάληψη: Ο αλγόριθμος εκτελείται όσο η ουρά δεν είναι κενή:
    - Ένας κόμβος εξάγεται από την αρχή της ουράς.
    - Αν ο κόμβος είναι φύλλο (leaf node), κάθε σημείο που περιέχει ελέγχεται αν βρίσκεται εντός των ορίων του ερωτήματος εύρους.
    - Αν ο κόμβος είναι εσωτερικός (internal node), ελέγχεται το MBR κάθε παιδιού του. Μόνο οι κόμβοι-παιδιά των οποίων το MBR τέμνεται (intersects) με το εύρος του ερωτήματος εισάγονται στο τέλος της ουράς για περαιτέρω επεξεργασία.

Αυτός ο κοινός σκελετός διάσχισης χρησιμοποιείται σε όλες τις υλοποιήσεις, με τη θεμελιώδη διαφορά να εντοπίζεται στον τρόπο με τον οποίο υλοποιείται ο έλεγχος τομής (intersection test) στον εσωτερικό κόμβο: βαθμωτή (scalar) για τη διάταξη D0 και διανυσματικός (vectorized) για τις διατάξεις D1 και D2.

4.2.3 Βασικές Εντολές SIMD

Η χρήση SIMD intrinsics αποτελεί βασικό εργαλείο για την επιτάχυνση της πράξης spatial select, καθώς επιτρέπει την εκτέλεση πολλαπλών συγκρίσεων ταυτόχρονα μέσα σε έναν κύκλο μηχανής. Οι εντολές αυτές υλοποιούνται στη C++ μέσω ειδικών συναρτήσεων (intrinsics), οι οποίες αντιστοιχούν απευθείας σε εντολές του συνόλου SIMD (π.χ. AVX2, AVX-512) του επεξεργαστή. Στον ακόλουθο πίνακα παρουσιάζονται οι πιο σημαντικές από αυτές, οι οποίες χρησιμοποιήθηκαν στο πλαίσιο της παρούσας εργασίας για την επιτάχυνση των συγκρίσεων MBR–ερωτήματος και τον φιλτράρισμα σημείων στα φύλλα.

Πίνακας 1: Εντολές SIMD AVX2-256

Intrinsic	SIMD Set	Λειτουργία
_mm256_set1_ps	AVX2	Δημιουργεί ένα διάνυσμα 8 floats με την ίδια τιμή (broadcast).
_mm256_set_ps	AVX2	Δημιουργεί ένα διάνυσμα 8 floats από 8 διαφορετικές τιμές (με αντίστροφη σειρά).
_mm256_loadu_ps	AVX2	Φορτώνει 8 floats από μη-ευθυγραμμισμένη θέση μνήμης.
_mm256_cmp_ps	AVX2	Εκτελεί παράλληλες συγκρίσεις μεταξύ 8 ζευγών floats.
_mm256_and_ps	AVX2	Εκτελεί λογικό AND (κατά bit) σε δύο διανύσματα.
_mm256_movemask_ps	AVX2	Μετατρέπει το πιο σημαντικό bit κάθε float σε μια μάσκα 8-bit.

Πίνακας 2: Εντολές SIMD AVX-512

Intrinsic	SIMD Set	Λειτουργία
_mm512_set1_ps	AVX-512	Αντίστοιχο broadcast για 16 floats.
_mm512_set_ps	AVX-512	Δημιουργεί ένα διάνυσμα 16 floats από 16 διαφορετικές τιμές (με αντίστροφη σειρά).
_mm512_loadu_ps	AVX-512	Αντίστοιχη φόρτωση για 16 floats.
_mm512_cmp_ps_mask	AVX-512	Συγκρίνει 16 ζεύγη floats και επιστρέφει απευθείας μάσκα 16-bit.

Το βασικό μοτίβο λειτουργίας στις SIMD υλοποιήσεις είναι η επεξεργασία των MBRs σε ομάδες (blocks) των 8 (για AVX2) ή 16 (για AVX-512), η παράλληλη εκτέλεση των τεσσάρων απαραίτητων συγκρίσεων και ο συνδυασμός των αποτελεσμάτων για τη γρήγορη εύρεση των επιλέξιμων κόμβων-παιδιών.

4.3 Διαφορετικά Layouts

Η κεντρική πειραματική υπόθεση της εργασίας είναι ο αντίκτυπος της διάταξης μνήμης στην απόδοση. Παρακάτω αναλύεται η δομή και η υλοποίηση κάθε διάταξης.

4.3.1 Διάταξη D0

Το D0 layout αποτελεί την κλασική μορφή αποθήκευσης των κόμβων του R-tree, όπως αυτή εισήχθη στην αρχική του πρόταση. Στο συγκεκριμένο layout, κάθε εγγραφή του κόμβου, η οποία περιλαμβάνει το ελάχιστο περιβάλλον ορθογώνιο (Minimum Bounding Rectangle – MBR) και τον δείκτη προς τον αντίστοιχο κόμβο-παιδί, αποθηκεύεται συνεχόμενα στη μνήμη. Ειδικότερα, τα πεδία (*MBR.low*$_x$, MBR.low$_y$, MBR.high$_x$, MBR.high$_y$, ptr) βρίσκονται σε διαδοχικές θέσεις, και οι εγγραφές στοιχίζονται σειριακά στον κόμβο όπως αποτυπώνεται και στην Εικόνα 10. Η διάταξη αυτή πλεονεκτεί ως προς την απλότητα και τη συμβατότητά της με τις παραδοσιακές υλοποιήσεις R-tree. Όλα τα δεδομένα που αντιστοιχούν σε ένα MBR βρίσκονται αποθηκευμένα τοπικά, γεγονός που διευκολύνει την πλήρη ανάκτηση μιας εγγραφής με μία μόνο προσπέλαση στη μνήμη. Επιπλέον, η δομή είναι εύκολη στην υλοποίηση και εναρμονίζεται με την υφιστάμενη βιβλιογραφία, γεγονός που την καθιστά κατάλληλη ως σημείο αναφοράς (baseline) για πειραματικές συγκρίσεις. Ωστόσο, το D0 layout παρουσιάζει σημαντικά μειονεκτήματα στο πλαίσιο της αξιοποίησης SIMD εντολών. Η πακεταρισμένη μορφή των MBRs δεν επιτρέπει την αποδοτική φόρτωση ομοειδών πεδίων (π.χ. όλων των τιμών lowx) σε SIMD registers, καθώς τα δεδομένα δεν είναι αποθηκευμένα σε συνεχόμενες θέσεις μνήμης. Ως αποτέλεσμα, η εφαρμογή SIMD συγκρίσεων απαιτεί πολλαπλά και μη-συνεχόμενες προσπελάσεις στην κύρια μνήμη, περιορίζοντας δραστικά την αποδοτικότητα. Παράλληλα, η υλοποίηση της πράξης spatial select στο D0 layout απαιτεί τέσσερις ξεχωριστές συγκρίσεις (lowx, lowy, highx, highy), οι οποίες αντιστοιχούν σε ισάριθμα conditional branches. Συνοψίζοντας, το D0 layout συνιστά τη βασική υλοποίηση της διάταξης κόμβων σε R-tree και αποτελεί χρήσιμο σημείο αναφοράς για την αξιολόγηση πιο εξελιγμένων προσεγγίσεων. Παρά τα πλεονεκτήματα απλότητας και τοπικότητας δεδομένων, η απόδοσή του περιορίζεται στις βαθμωτές υλοποιήσεις, ενώ κρίνεται

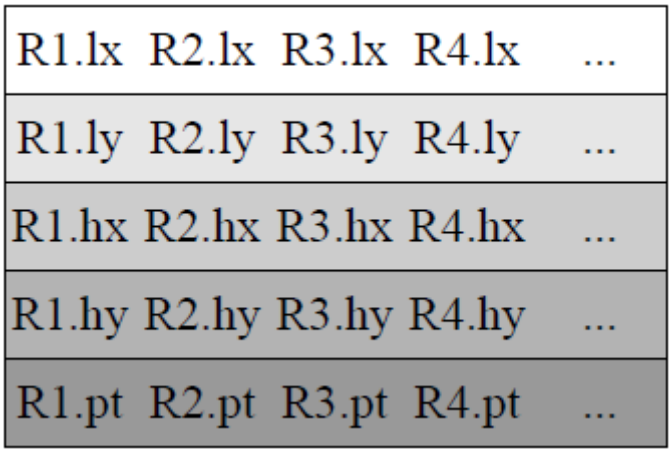
ανεπαρκές για την πλήρη εκμετάλλευση των SIMD δυνατοτήτων των σύγχρονων επεξεργαστών.

R1.lx	R1.ly	R1.hx	R1.hy	R1.pt
R2.lx	R2.ly	R2.hx	R2.hy	R2.pt
R3.lx	R3.ly	R3.hx	R3.hy	R3.pt
R4.lx	R4.ly	R4.hx	R4.hy	R4.pt
…	…	…	…	…

Εικόνα 10: D0 storage layout of index nodes[RA23]

4.3.2 Διάταξη D1

Το D1 layout προτείνει μια διαφορετική οργάνωση των δεδομένων με στόχο την καλύτερη αξιοποίηση των SIMD εντολών. Σε αυτή τη διάταξη, τα πεδία όλων των MBRs οργανώνονται σε ξεχωριστούς πίνακες: πρώτα αποθηκεύονται όλες οι τιμές lowx, κατόπιν όλες οι lowy, ακολουθούν οι highx και highy, και τέλος οι δείκτες (ptr) προς τους child κόμβους, όπως αποτυπώνεται στην Εικόνα 11. Με τον τρόπο αυτό, τα ομοειδή πεδία κάθε MBR βρίσκονται τοποθετημένα σε συνεχόμενες θέσεις μνήμης. Η συγκεκριμένη διάταξη διευκολύνει τη μαζική φόρτωση δεδομένων σε SIMD registers, επιτρέποντας την ταυτόχρονη σύγκριση πολλαπλών τιμών ενός πεδίου (π.χ. όλων των lowx συντεταγμένων). Αυτό οδηγεί σε σημαντική μείωση των απαιτούμενων εντολών και βελτιώνει την αποδοτικότητα σε πράξεις όπως η spatial select. Επιπλέον, η οργάνωση των pointers σε ξεχωριστό πίνακα καθιστά δυνατή την αποδοτική εισαγωγή διευθύνσεων στη δομή ουράς που χρησιμοποιείται για τις SIMD-friendly υλοποιήσεις με breadth-first traversal. Παρά τα πλεονεκτήματα αυτά, το D1 layout εισάγει μεγαλύτερη πολυπλοκότητα σε σχέση με το D0, καθώς απαιτεί μετασχηματισμό της δομής των κόμβων και επιπλέον φόρτο κατά την κατασκευή και συντήρηση του δένδρου. Επίσης, η διάσπαση των δεδομένων σε πολλαπλούς πίνακες μπορεί να επιφέρει μικρή υποβάθμιση στην τοπικότητα δεδομένων, αφού τα στοιχεία ενός MBR δεν βρίσκονται πλέον αποθηκευμένα ενιαία. Συνολικά, το D1 layout επιτυγχάνει σημαντική βελτίωση στην αποδοτικότητα SIMD, καθιστώντας το κατάλληλο για πειραματική αξιολόγηση σε περιβάλλοντα όπου η ταχύτητα των spatial select αποτελεί κρίσιμο ζητούμενο. Τα αποτελέσματα από την πειραματική αξιολόγηση θα σχολιαστούν στο Κεφάλαιο 5.



Εικόνα 11: D1 storage layout of index nodes [RA23]

4.3.3 Διάταξη D2

Το D2 layout συνιστά μια περαιτέρω βελτιστοποίηση σε σχέση με το D1, επιχειρώντας να συνδυάσει την ευκολία SIMD vectorization με καλύτερη τοπικότητα δεδομένων. Στη συγκεκριμένη διάταξη, κάθε εσωτερικός κόμβος αποθηκεύει τις ελάχιστες συντεταγμένες όλων των MBRs σε έναν πίνακα (lo) και τις μέγιστες συντεταγμένες σε έναν δεύτερο πίνακα (hi). Σε κάθε πίνακα οι τιμές οργανώνονται εναλλάξ ως ζεύγη (x,y) για κάθε παιδί κόμβο, όπως αποτυπώνεται στην Εικόνα 12. Η οργάνωση αυτή μειώνει τον κατακερματισμό των δεδομένων στη μνήμη και διευκολύνει τη φόρτωση συνεχόμενων τιμών σε SIMD registers, καθώς κάθε MBR περιγράφεται πλέον με δύο βασικά διανύσματα αντί για τέσσερα ανεξάρτητα πεδία. Η συμπαγής αυτή διάταξη βελτιώνει τη χωρική τοπικότητα σε επίπεδο cache, αφού οι συντεταγμένες όλων των παιδιών βρίσκονται αποθηκευμένες σε συνεχόμενες θέσεις μνήμης. Έτσι μειώνονται τα LLC misses και βελτιώνεται η συνολική αποδοτικότητα σε ερωτήματα χαμηλής επιλεκτικότητας, όπου η πρόσβαση στη μνήμη αποτελεί συχνά τον κύριο περιορισμό. Το βασικό μειονέκτημα είναι ότι η κατασκευή του δένδρου απαιτεί πρόσθετη λογική για τη διάσπαση των MBRs και την τοποθέτησή τους στους δύο πίνακες, γεγονός που αυξάνει ελαφρώς την πολυπλοκότητα της υλοποίησης. Συμπερασματικά, το D2 layout διατηρεί την απλότητα του D1 ως προς τη δυνατότητα διανυσματικών πράξεων, ενώ ταυτόχρονα επιτυγχάνει καλύτερη αποδοτικότητα μνήμης μέσω της διάσπασης των MBRs σε δύο διακριτούς πίνακες. Στην παρούσα εργασία το D2 layout υλοποιήθηκε τόσο με χρήση AVX2 (256-bit) όσο και με AVX-512 (512-bit), με οργάνωση των MBRs στους πίνακες lo και hi. Οι λεπτομέρειες της υλοποίησης της πράξης spatial select και οι αποκλίσεις από τη βιβλιογραφία παρουσιάζονται στο Κεφάλαιο 4.4.

R1.lx	R1.ly	R2.lx	R2.ly	R3.lx	R3.ly	R4.lx	R4.ly	...
R1.hx	R1.hy	R2.hx	R2.hy	R3.hx	R3.hy	R4.hx	R4.hy	...
R1.pt	R2.pt	R3.pt	R4.pt	...	...	...	...	...

Εικόνα 12: D2 storage layout of index nodes [RA23]

4.4 Μέθοδοι Αναζήτησης (spatial select)

4.4.1 Spatial Select D0

Ο αλγόριθμος που αναπτύχθηκε βασίζεται σε μια στρατηγική διάσχισης του δέντρου πρώτα κατά πλάτος (Breadth-First Search - BFS). Για τη διαχείριση των κόμβων που πρόκειται να εξεταστούν, χρησιμοποιείται η δομή δεδομένων της ουράς (queue), διασφαλίζοντας την επισκόπηση του δέντρου επίπεδο προς επίπεδο. Η διαδικασία ξεκινά τοποθετώντας τον κόμβο-ρίζα (root node) του R-tree στην ουρά. Σε κάθε βήμα της επανάληψης, ένας κόμβος αφαιρείται από την αρχή της ουράς και εξετάζεται. Αν πρόκειται για εσωτερικό κόμβο (internal node), ο αλγόριθμος ελέγχει διαδοχικά κάθε ένα από τα Ελάχιστα Περιβάλλοντα Ορθογώνια (Minimum Bounding Rectangles - MBRs) που αυτός περιέχει. Το κρίσιμο βήμα της βελτιστοποίησης είναι το κλάδεμα του χώρου αναζήτησης: μόνο οι κόμβοι-παιδιά των οποίων το MBR παρουσιάζει επικάλυψη (overlap) με την ορθογώνια περιοχή του ερωτήματος προστίθενται στο τέλος της ουράς για μελλοντική επεξεργασία. Με αυτόν τον τρόπο, ολόκληροι κλάδοι του δέντρου που είναι εγγυημένο ότι δεν περιέχουν σχετικά δεδομένα παρακάμπτονται, μειώνοντας δραστικά τον υπολογιστικό φόρτο. Η διάσχιση συνεχίζεται μέχρι η αναζήτηση να φτάσει σε έναν κόμβο-φύλλο (leaf node). Σε αυτό το σημείο, ο αλγόριθμος παύει να εξετάζει MBRs και προχωρά στον έλεγχο των πραγματικών σημείων (points) που είναι αποθηκευμένα στον κόμβο. Κάθε σημείο ελέγχεται ξεχωριστά για να διαπιστωθεί εάν οι συντεταγμένες του (x, y) εμπίπτουν πλήρως εντός της περιοχής που ορίζει το ερώτημα (query).Για κάθε σημείο που ικανοποιεί τα κριτήρια του ερωτήματος, ένας εσωτερικός μετρητής αυξάνεται. Μετά την εξάντληση όλων των πιθανών κλάδων του δέντρου (δηλαδή όταν η ουρά αδειάσει), η συνάρτηση ολοκληρώνεται επιστρέφοντας τη συνολική καταμέτρηση (total count) των σημείων που βρέθηκαν εντός της καθορισμένης περιοχής.

4.4.2 Spatial Select D1

Ο αλγόριθμος αναζήτησης για τη διάταξη D1 αξιοποιεί πλήρως την οργάνωση των δεδομένων σε ξεχωριστούς, συνεχόμενους πίνακες για κάθε συντεταγμένη (min_xs, min_ys, max_xs, max_ys). Αυτή η δομή είναι ιδανική για διανυσματική επεξεργασία, καθώς επιτρέπει τη φόρτωση ομοειδών δεδομένων (π.χ., 8 ή 16 τιμές min_x ταυτόχρονα) σε καταχωρητές SIMD με μία μόνο εντολή. Η διάσχιση του δέντρου ακολουθεί και εδώ τη μη αναδρομική προσέγγιση BFS με χρήση ουράς. Η βασική διαφοροποίηση εντοπίζεται στον τρόπο επεξεργασίας των εσωτερικών κόμβων.

Αλγόριθμος 2: Ψευδοκώδικας για το spatial select D1

```
1   Broadcast the query's minimum and maximum bounds in both x and y directions.
2   For each group of child nodes:
3       Load the children's bounding box coordinates (min_x, min_y, max_x, max_y)
4       Compare each child's bounding box against the query box
5       Combine these four conditions to determine whether a child intersects the
        query
6       Record which children satisfy all conditions
7   For every child that intersects: Add it to the processing queue
8   Continue until all children of the node have been examined
```

Προετοιμασία Ερωτήματος: Οι τέσσερις οριακές τιμές του ορθογωνίου του ερωτήματος (query.min_x, query.min_y, query.max_x, query.max_y) μεταδίδονται (broadcast) σε αντίστοιχους καταχωρητές SIMD. Για παράδειγμα, με AVX2, δημιουργείται ένας καταχωρητής που περιέχει την τιμή query.min_x οκτώ φορές, ένας δεύτερος για το query.min_y κ.ο.κ.

Επεξεργασία Εσωτερικού Κόμβου: Τα MBRs ενός εσωτερικού κόμβου επεξεργάζονται σε ομάδες των 8 (για AVX2) ή 16 (για AVX-512). Για κάθε ομάδα:

- **Φόρτωση Δεδομένων:** Φορτώνονται οι αντίστοιχες συντεταγμένες από τους τέσσερις πίνακες του κόμβου στους καταχωρητές SIMD.
- **Παράλληλες Συγκρίσεις:** Εκτελούνται ταυτόχρονα και οι τέσσερις απαραίτητες συνθήκες τομής για ολόκληρη την ομάδα των MBRs:
    - MBR.max_x >= query.min_x
    - MBR.min_x <= query.max_x
    - MBR.max_y >= query.min_y

- MBR.min_y <= query.max_y

Συνδυασμός Αποτελεσμάτων: Τα αποτελέσματα των τεσσάρων αυτών διανυσματικών συγκρίσεων (που είναι μάσκες bit) συνδυάζονται με μια λογική πράξη AND. Το αποτέλεσμα είναι μια τελική μάσκα όπου κάθε bit αντιστοιχεί σε ένα MBR της ομάδας. Ένα bit είναι 1 μόνο αν και οι τέσσερις συνθήκες για το αντίστοιχο MBR είναι αληθείς.

Εξαγωγή και Προσθήκη στην Ουρά: Η τελική μάσκα μετατρέπεται σε έναν ακέραιο (bitmask). Ο αλγόριθμος διατρέχει τα bits αυτού του ακεραίου και για κάθε bit που είναι 1, προσθέτει τον αντίστοιχο κόμβο-παιδί στην ουρά για περαιτέρω επεξεργασία.

Επεξεργασία Κόμβου-Φύλλου: Όταν η διάσχιση φτάνει σε έναν κόμβο-φύλλο, τα σημεία που περιέχει ελέγχονται. Στο πλαίσιο της υλοποίησης, αναπτύχθηκαν δύο εκδοχές:

- Βαθμωτή (Scalar) Επεξεργασία: Κάθε σημείο ελέγχεται σειριακά ως προς το αν βρίσκεται εντός του ορθογωνίου του ερωτήματος.
- Διανυσματοποιημένη (Vectorized) Επεξεργασία: Για περαιτέρω βελτιστοποίηση, τα σημεία του φύλλου ελέγχονται επίσης σε ομάδες με χρήση SIMD. Οι συντεταγμένες x και y των σημείων φορτώνονται σε καταχωρητές και συγκρίνονται παράλληλα με τα όρια του ερωτήματος, επιταχύνοντας σημαντικά το τελικό στάδιο της καταμέτρησης.

Η διανυσματοποιημένη μέθοδος μειώνει δραστικά τον αριθμό των εντολών και των διακλαδώσεων (branches) σε σχέση με τη βαθμωτή προσέγγιση D0, μετατρέποντας τον υπολογιστικά κοστοβόρο έλεγχο τομής σε μια σειρά από αποδοτικές διανυσματικές λειτουργίες.

4.4.3 Spatial Select D2

Η μέθοδος αναζήτησης για τη διάταξη D2 αποτελεί εξέλιξη της D1, εκμεταλλευόμενη τη βελτιωμένη τοπικότητα δεδομένων που προσφέρει η ομαδοποίηση των συντεταγμένων σε δύο πίνακες: lo (που περιέχει εναλλάξ τις min_x, min_y) και hi (που περιέχει εναλλάξ τις max_x, max_y).Η συνολική αρχιτεκτονική BFS παραμένει ίδια, αλλά ο πυρήνας της διανυσματικής επεξεργασίας εντός των εσωτερικών κόμβων διαφοροποιείται σημαντικά για να ταιριάζει με τη νέα δομή δεδομένων.

Αλγόριθμος 3: Ψευδοκώδικας spatial select D2

1	Prepare two broadcast vectors with the query's minimum and maximum coordinates (interleaved x and y values)
2	For each group of child nodes:
3	Load the interleaved bounding box coordinates
4	Compare each child's high values with the query's minimums, and its low values with the query's maximums
5	Extract the results into a bitmask
6	For each child in the group:
7	If both x and y comparisons are satisfied, αdd that child to the queue
8	Continue until all children have been examined

Προετοιμασία Ερωτήματος: Η προετοιμασία των καταχωρητών του ερωτήματος είναι κρίσιμη. Αντί για τέσσερις ξεχωριστούς καταχωρητές, δημιουργούνται δύο, οι οποίοι αντικατοπτρίζουν την εναλλασσόμενη δομή των πινάκων lo και hi. Ένας καταχωρητής query_mins γεμίζει με εναλλασσόμενες τιμές query.min_x, query.min_y.Ένας καταχωρητής query_maxs γεμίζει με εναλλασσόμενες τιμές query.max_x, query.max_y.

Αυτό διασφαλίζει ότι κατά τη σύγκριση, κάθε συντεταγμένη x του MBR θα συγκριθεί με τη σωστή συντεταγμένη x του ερωτήματος, και αντίστοιχα για τις y.

Επεξεργασία Εσωτερικού Κόμβου: Τα MBRs επεξεργάζονται και πάλι σε ομάδες (των 4 για AVX2, 8 για AVX-512, καθώς κάθε MBR καταλαμβάνει δύο floats σε κάθε πίνακα).

Φόρτωση Δεδομένων: Απαιτούνται μόνο δύο εντολές φόρτωσης ανά ομάδα: μία από τον πίνακα lo και μία από τον hi. Αυτό βελτιώνει την απόδοση της μνήμης σε σχέση με τις τέσσερις φορτώσεις της D1.

Παράλληλες Συγκρίσεις: Ο έλεγχος τομής απλοποιείται σε δύο μόνο διανυσματικές συγκρίσεις:

- hi_vec >= query_mins (ελέγχει ταυτόχρονα τις συνθήκες MBR.max_y >= query.min_y και MBR.max_x >= query.min_x)
- lo_vec <= query_maxs (ελέγχει ταυτόχρονα τις συνθήκες MBR.min_y <= query.max_y και MBR.min_x <= query.max_x)

Συνδυασμός και Αποκωδικοποίηση Μάσκας: Οι μάσκες των δύο παραπάνω συγκρίσεων συνδυάζονται με λογικό AND. Η τελική μάσκα που προκύπτει απαιτεί ειδική αποκωδικοποίηση. Επειδή οι συντεταγμένες είναι εναλλασσόμενες, ένα MBR θεωρείται

ότι τέμνεται με το ερώτημα μόνο αν και τα δύο bits που του αντιστοιχούν στη μάσκα είναι 1 (ένα για τη σύγκριση (x,y) με τα κατώτερα όρια και ένα για τη σύγκριση (x,y) με τα ανώτερα). Για παράδειγμα, στην υλοποίηση AVX-512, το πρώτο MBR αντιστοιχεί στα δύο πρώτα bits. Η συνθήκη (bitmask & 0x3) == 0x3 ελέγχει αν το πρώτο παιδί πρέπει να προστεθεί στην ουρά. Το δεύτερο παιδί ελέγχεται με (bitmask & 0xC) == 0xC κ.ο.κ.

Επεξεργασία Κόμβου-Φύλλου: Η επεξεργασία των φύλλων είναι πανομοιότυπη με αυτή της διάταξης D1, υποστηρίζοντας τόσο τη βαθμωτή όσο και τη βελτιστοποιημένη διανυσματοποιημένη εκδοχή.

Συνολικά, η διάταξη D2 με τη βελτιωμένη τοπικότητα δεδομένων και τον μειωμένο αριθμό προσβάσεων στη μνήμη, σε συνδυασμό με την οργάνωση των διανυσματικών πράξεων, προσφέρει τη θεωρητικά υψηλότερη απόδοση, ειδικά σε σενάρια όπου η ταχύτητα της μνήμης αποτελεί περιοριστικό παράγοντα.

Κεφάλαιο 5. Πειραματική Αξιολόγηση

Στο κεφάλαιο αυτό παρουσιάζεται η πειραματική αξιολόγηση των μεθόδων που υλοποιήθηκαν. Στόχος είναι η ποσοτική μέτρηση της απόδοσης των διαφορετικών διατάξεων μνήμης (D0, D1, D2) και η ανάδειξη των πλεονεκτημάτων που προσφέρει η χρήση εντολών SIMD στην επιτάχυνση των ερωτημάτων εύρους (spatial select) σε R-trees.

5.1 Πειραματικό Περιβάλλον

Όλα τα πειράματα διεξήχθησαν σε ένα σύγχρονο υπολογιστικό σύστημα με τα ακόλουθα τεχνικά χαρακτηριστικά, ώστε να διασφαλιστεί η αξιοπιστία και η επαναληψιμότητα των μετρήσεων.

- Επεξεργαστής (CPU): Intel® Core™ i7-11700K (8 cores at 3.60 GHz)
- Κύρια Μνήμη (RAM): 128 GB
- Λειτουργικό Σύστημα: Ubuntu 22.04
- Μεταγλωττιστής (Compiler): GCC (v13) με τις ακόλουθες ρυθμίσεις βελτιστοποίησης: -O3, -march=native. Η σημαία -O3 ενεργοποιεί το υψηλότερο επίπεδο βελτιστοποιήσεων, ενώ οι σημαίες -mavx=native, ζητούν από τον compiler να παράξει κώδικα που εκμεταλλεύεται συγκεκριμένα χαρακτηριστικά της τρέχουσας CPU όπως το autovectorization (instruction sets, AVX2/AVX-512)

5.2 Σύνολα Δεδομένων

Για τις ανάγκες της πειραματικής αξιολόγησης, χρησιμοποιήθηκαν δύο δημόσια διαθέσιμα, πραγματικά σύνολα γεωχωρικών δεδομένων από το project Spatial Hadoop [SpHa] του Πανεπιστημίου της Μινεσότα. Δεδομένου ότι η παρούσα εργασία εστιάζει στην ευρετηρίαση σημειακών δεδομένων (points), τα αρχικά σύνολα, που περιείχαν πολυγωνικές και γραμμικές γεωμετρίες, έχουν υποστεί προ επεξεργασία. Συγκεκριμένα, από κάθε γεωμετρία (πολύγωνο ή πολύ-γραμμή) εξήχθη το κεντροειδές (centroid) της, δημιουργώντας τα δύο τελικά σύνολα σημείων που χρησιμοποιήθηκαν στα πειράματα.

- EDGES Dataset: Το πρώτο και μεγαλύτερο σύνολο δεδομένων προέρχεται από το dataset EDGES. Μετά τον υπολογισμό των κεντροειδών, δημιουργήθηκε το τελικό αρχείο σημείων που χρησιμοποιήθηκε στην εργασία, αποτελούμενο από 51 εκατομμύρια σημεία.
- ROADS Dataset: Το δεύτερο σύνολο δεδομένων προέρχεται από το dataset ROADS. Μετά τον υπολογισμό των κεντροειδών, δημιουργήθηκε το τελικό αρχείο σημείων που χρησιμοποιήθηκε στην εργασία, αποτελούμενο από 17 εκατομμύρια σημεία.

5.3 Σύνολα Ερωτημάτων

Για την αξιολόγηση της απόδοσης υπό διαφορετικούς φόρτους εργασίας, δημιουργήθηκαν πέντε σύνολα ερωτημάτων αναζήτησης περιοχής (spatial select queries) για κάθε dataset. Τα σύνολα αυτά διαφοροποιούνται ως προς την επιλεκτικότητά τους (selectivity), δηλαδή το ποσοστό των δεδομένων που επιστρέφουν κατά μέσο όρο. Τα τετράγωνα παράθυρα των ερωτημάτων είχαν τυχαία θέση στον χώρο, με τις διαστάσεις τους να ρυθμίζονται ώστε να επιτυγχάνουν τις ακόλουθες πέντε τιμές επιλεκτικότητας:

- **0.01%** (Πολύ χαμηλή επιλεκτικότητα)
- **0.05%** (Χαμηλή επιλεκτικότητα)
- **0.1%** (Μεσαία επιλεκτικότητα)
- **0.5%** (Υψηλή επιλεκτικότητα)
- **1%** (Πολύ υψηλή επιλεκτικότητα)

Η επιλογή αυτού του κλιμακωτού εύρους επιλεκτικοτήτων παρέχει μια σφαιρική εικόνα της συμπεριφοράς των αλγορίθμων υπό διαφορετικό φόρτο εργασίας. Η προσέγγιση αυτή επιτρέπει την αξιολόγηση της απόδοσης τόσο σε σενάρια που περιορίζονται από την καθυστέρηση του συστήματος ,όπως οι εξαιρετικά στοχευμένες αναζητήσεις, όσο και σε σενάρια που είναι υπολογιστικά απαιτητικά, όπως οι χωρικές αναλύσεις ευρείας κάλυψης.

5.4 Μετρικές Αποδόσεις

Η κύρια μετρική για την αξιολόγηση της απόδοσης ήταν ο χρόνος εκτέλεσης (Execution Time) των ερωτημάτων spatial select, μετρούμενος σε χιλιοστά του δευτερολέπτου (ms). Για κάθε πείραμα, εκτελέστηκαν πέντε σύνολα, 10.000 ερωτημάτων με συγκεκριμένη επιλεκτικότητα όπως αναφέρθηκε στην ενότητα 5.3 και καταγράφηκε ο μέσος χρόνος απόκρισης. Επιπλέον, η απόδοση (Throughput) υπολογίστηκε ως ο αριθμός των ερωτημάτων που εκτελούνται ανά δευτερόλεπτο, δηλαδή ο λόγος του συνολικού αριθμού ερωτημάτων προς τον συνολικό χρόνο εκτέλεσης.

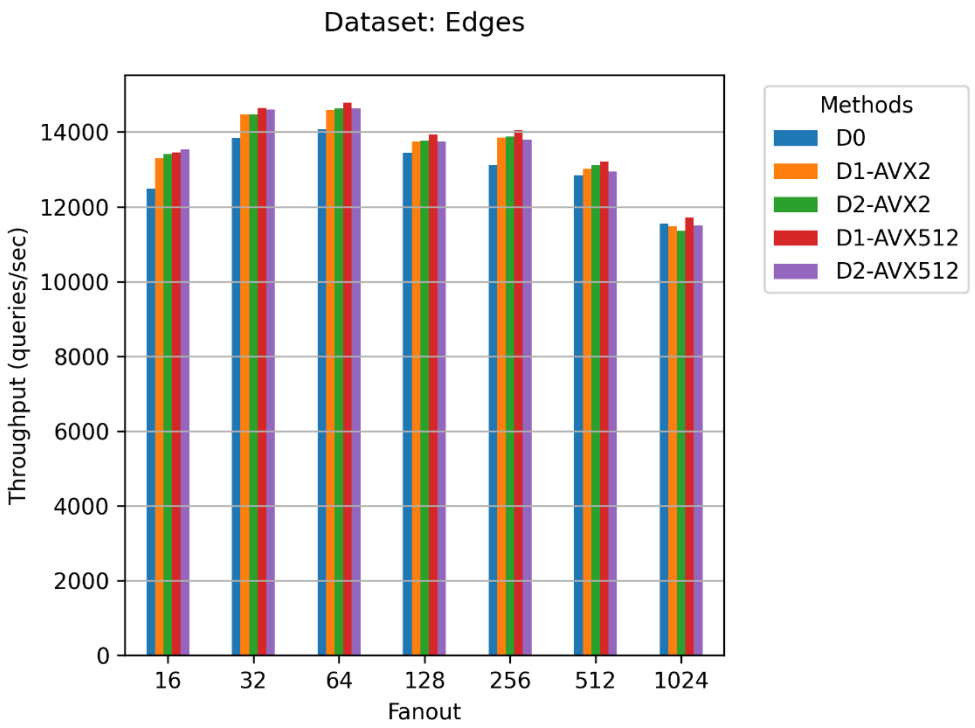
$$Throughput = number_of_queries \,/\, total_execution_time(sec)$$

5.5 Πειραματική ανάλυση

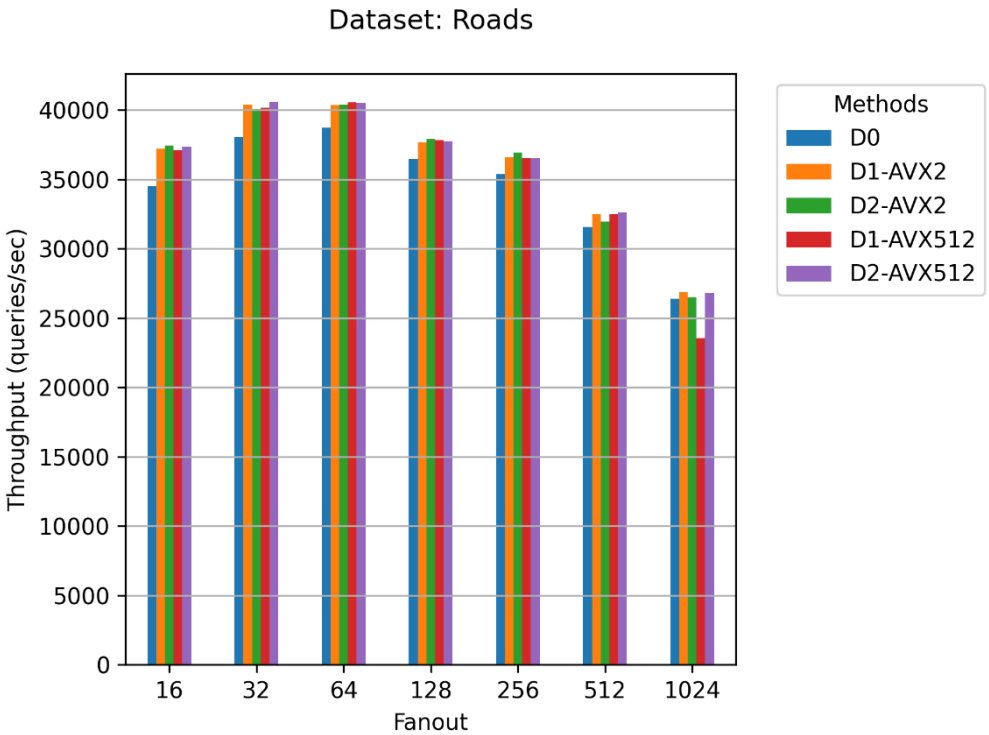
Για την αξιολόγηση της απόδοσης της υλοποίησης που περιεγράφηκε στο Κεφάλαιο 4, πραγματοποιήθηκε μια σειρά από πειράματα. Ακολουθεί η παρουσίαση και η ανάλυση των αποτελεσμάτων τους.

5.5.1 Σύγκριση Απόδοσης ως προς το Fanout

Ένας από τους κρισιμότερους παράγοντες που επηρεάζουν την απόδοση ενός R-tree είναι το fanout, δηλαδή ο μέγιστος αριθμός παιδιών που μπορεί να έχει κάθε εσωτερικός κόμβος. Η επιλογή του fanout επιδρά άμεσα στη δομή του δέντρου: ένα μικρό fanout οδηγεί σε ένα "ψηλό" δέντρο που απαιτεί περισσότερες προσπελάσεις στη μνήμη για τη διάσχισή του, ενώ ένα μεγάλο fanout δημιουργεί ένα "κοντό" δέντρο, μειώνοντας τις προσπελάσεις, αλλά αυξάνοντας το υπολογιστικό κόστος επεξεργασίας εντός κάθε κόμβου. Για τον εντοπισμό της βέλτιστης τιμής, διεξήχθη πείραμα όπου μετρήθηκε η απόδοση (throughput) για fanout από 16 έως 1024. Η Εικόνα 13 και η Εικόνα 14 παρουσιάζουν τη συγκριτική απόδοση όλων των υλοποιήσεων (D0, D1-AVX2, D2-AVX2, D1-AVX512, D2-AVX512) για ένα σύνολο 10.000 ερωτημάτων με επιλεκτικότητα 0.1% πάνω στο σύνολο δεδομένων Edges και στο Roads αντίστοιχα.



Εικόνα 13: Απόδοση (queries/sec) σε συνάρτηση με το fanout για το σύνολο δεδομένων Edges και επιλεκτικότητα 0.1%.



Εικόνα 14: Απόδοση (queries/sec) σε συνάρτηση με το fanout για το σύνολο δεδομένων Roads και επιλεκτικότητα 0.1%.

Από τη συγκριτική ανάλυση των δύο διαγραμμάτων προκύπτουν τα εξής συμπεράσματα:

- Βέλτιστη Περιοχή Fanout: Η συμπεριφορά της απόδοσης είναι παρόμοια και στα δύο σύνολα δεδομένων, παρά τη σημαντική διαφορά στο μέγεθός τους. Και στις δύο περιπτώσεις, η απόδοση αυξάνεται απότομα από fanout 16, φτάνει στη μέγιστη τιμή της στην περιοχή fanout 32-64, και στη συνέχεια αρχίζει να μειώνεται σταδιακά. Αυτό το εύρημα είναι συνεπές με την ανάλυση του paper [RA23], το οποίο επίσης εντοπίζει μια βέλτιστη περιοχή πριν η απόδοση αρχίσει να φθίνει λόγω του αυξημένου υπολογιστικού κόστους επεξεργασίας εντός των μεγαλύτερων κόμβων.
- Ιεραρχία Απόδοσης Μεθόδων: Σε όλο το εύρος των fanout, η ιεραρχία απόδοσης των μεθόδων παραμένει σταθερή. Η υλοποίηση D1_AVX-512 είναι σταθερά η ταχύτερη σε όλα τα σενάρια. Οι υλοποιήσεις με AVX-512 είναι πάντα ταχύτερες από τις αντίστοιχες με AVX2, ενώ η διάταξη D1 υπερτερεί σταθερά της D2 για το ίδιο σύνολο εντολών SIMD. Η βαθμωτή (scalar) υλοποίηση D0 αποτελεί, όπως αναμενόταν, τη βάση σύγκρισης και είναι με διαφορά η πιο αργή σε κάθε περίπτωση.

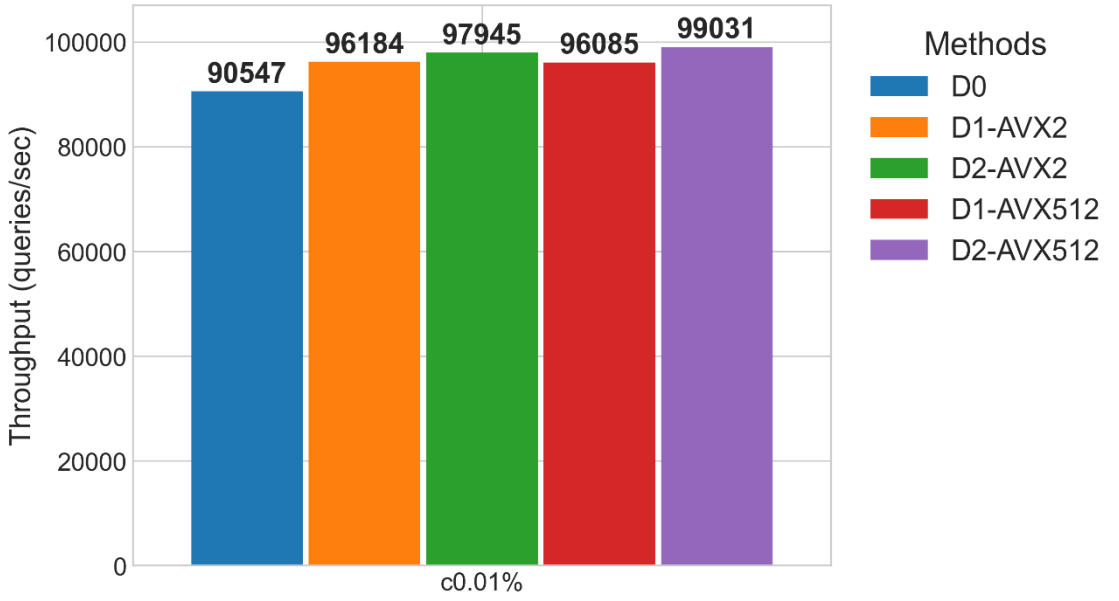
Δεδομένου ότι η βέλτιστη περιοχή λειτουργίας είναι σταθερά η ίδια και στα δύο πειράματα, για τη διεξαγωγή των υπολοίπων συγκρίσεων επιλέχθηκε η τιμή

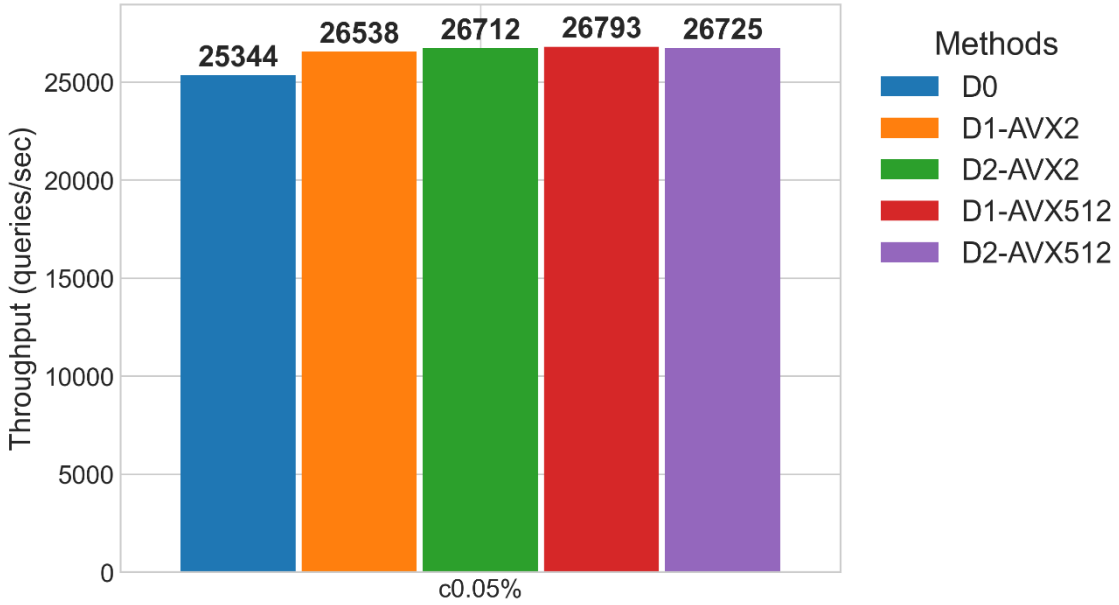
fanout = 64. Η επιλογή αυτή εξασφαλίζει μέγιστη απόδοση και επιτρέπει μια δίκαιη και αντικειμενική σύγκριση της επίδρασης των διαφορετικών διατάξεων μνήμης και των SIMD βελτιστοποιήσεων. Τα πειραματικά αποτελέσματα αποκαλύπτουν μια απόκλιση από τα αποτελέσματα της δημοσίευσης των Rayhan and Aref. Ενώ στην παρούσα μελέτη η διάταξη μνήμης D1 επιδεικνύει συστηματικά ανώτερη απόδοση έναντι της D2 για το ίδιο σύνολο εντολών SIMD, η έρευνα [RA23] καταλήγει στο συμπέρασμα ότι η D2 υπερτερεί οριακά για ερωτήματα τύπου spatial select. Η ερμηνεία αυτής της απόκλισης εντοπίζεται στην εφαρμογή μιας επιπρόσθετης βελτιστοποίησης στο υλικό αναφοράς, η οποία δεν υιοθετήθηκε στην παρούσα υλοποίηση: του software prefetching (O2). Σύμφωνα με την ανάλυση στο [RA23], ο τελεστής spatial select χαρακτηρίζεται ως memory-bound, με την απόδοσή του να περιορίζεται από τη λανθάνουσα κατάσταση της μνήμης, και συγκεκριμένα από τις αστοχίες στην κρυφή μνήμη τελευταίου επιπέδου (LLC misses). Οι συγγραφείς χρησιμοποίησαν εντολές _mm_prefetch για την απόκρυψη αυτής της καθυστέρησης, επιτυγχάνοντας την εκ των προτέρων φόρτωση των δεδομένων των κόμβων. Μετά από αυτές τις βελτιώσεις, η απόδοση άρχισε να εξαρτάται από δευτερεύοντες παράγοντες, όπως οι αστοχίες στην L1-D cache, πεδίο στο οποίο η διάταξη D2 έδειξε οριακά καλύτερη συμπεριφορά. Στην παρούσα διπλωματική εργασία , δεν έχει

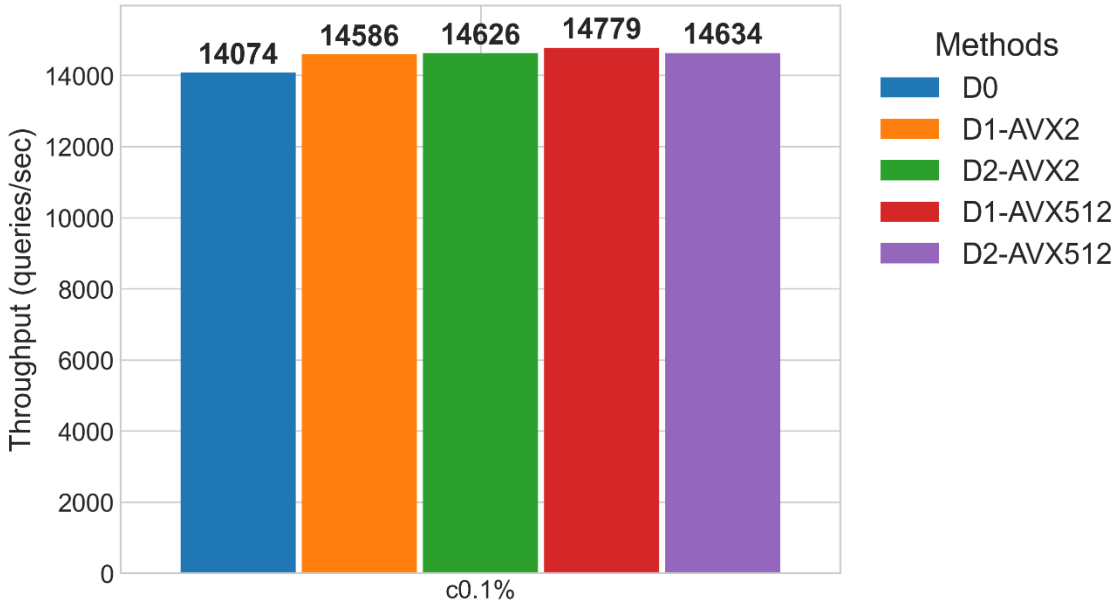
υλοποιηθεί η βελτιστοποίηση software prefetching. Έτσι, ο καθοριστικός παράγοντας απόδοσης παραμένει η αποδοτικότητα της κάθε διάταξης μνήμης σε συνδυασμό με τις διανυσματικές λειτουργίες. Υπό αυτές τις συνθήκες, η διάταξη D1 (Structure of Arrays) έχει καλύτερη αρχιτεκτονική για το συγκεκριμένο πρόβλημα . Η οργάνωσή της επιτρέπει την πλήρως γραμμική φόρτωση δεδομένων (π.χ. όλες οι συντεταγμένες min_x) στους καταχωρητές SIMD, μια διαδικασία που είναι ιδανική για την αρχιτεκτονική των σύγχρονων επεξεργαστών. Αντιθέτως, η πλεκτή (interleaved) δομή της D2, αν και πιο συμπαγής, απαιτεί πιο σύνθετη λογική για τον διαχωρισμό των συντεταγμένων εντός των καταχωρητών, εισάγοντας μια μικρή υπολογιστική επιβάρυνση. Συνεπώς, τα αποτελέσματά μας επιβεβαιώνουν ότι χωρίς την εφαρμογή προηγμένων τεχνικών prefetching, η αρχιτεκτονική καταλληλόλητα της διάταξης D1 για διανυσματικές λειτουργίες είναι αποδοτικότερη.

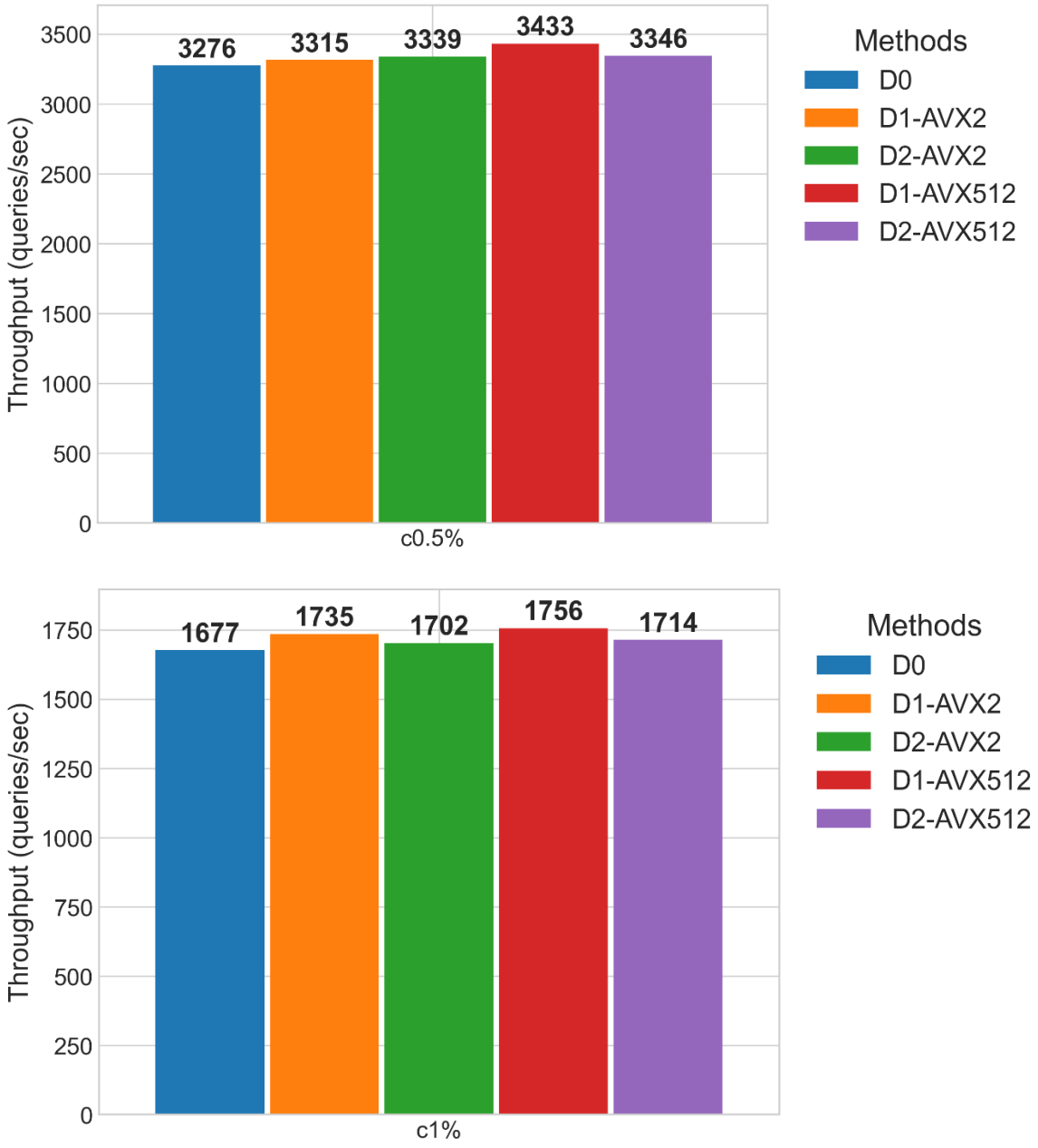
5.5.2 Επίδραση της Επιλεκτικότητας στην Απόδοση

Η επιλεκτικότητα ενός ερωτήματος, δηλαδή το ποσοστό των δεδομένων που επιστρέφονται, αποτελεί καθοριστικό παράγοντα για την απόδοση. Ερωτήματα υψηλής επιλεκτικότητας (π.χ., 0.01%, επιστρέφουν λίγα αποτελέσματα) περιορίζονται κυρίως από την καθυστέρηση της διάσχισης του δέντρου. Αντίθετα, ερωτήματα χαμηλής επιλεκτικότητας (π.χ., 1%, επιστρέφουν πολλά αποτελέσματα) είναι υπολογιστικά έντονα, καθώς ο κύριος φόρτος εργασίας μετατοπίζεται στην επεξεργασία των MBRs και των σημείων εντός των κόμβων. Για να αξιολογηθεί η συμπεριφορά των υλοποιήσεων, μετρήθηκε η απόδοση για πέντε διαφορετικά επίπεδα επιλεκτικότητας (0.01%, 0.05%, 0.1%, 0.5%, 1%) όπως αυτά ορίστηκαν στην ενότητα 5.3, διατηρώντας το fanout σταθερό στην βέλτιστη τιμή των 64.

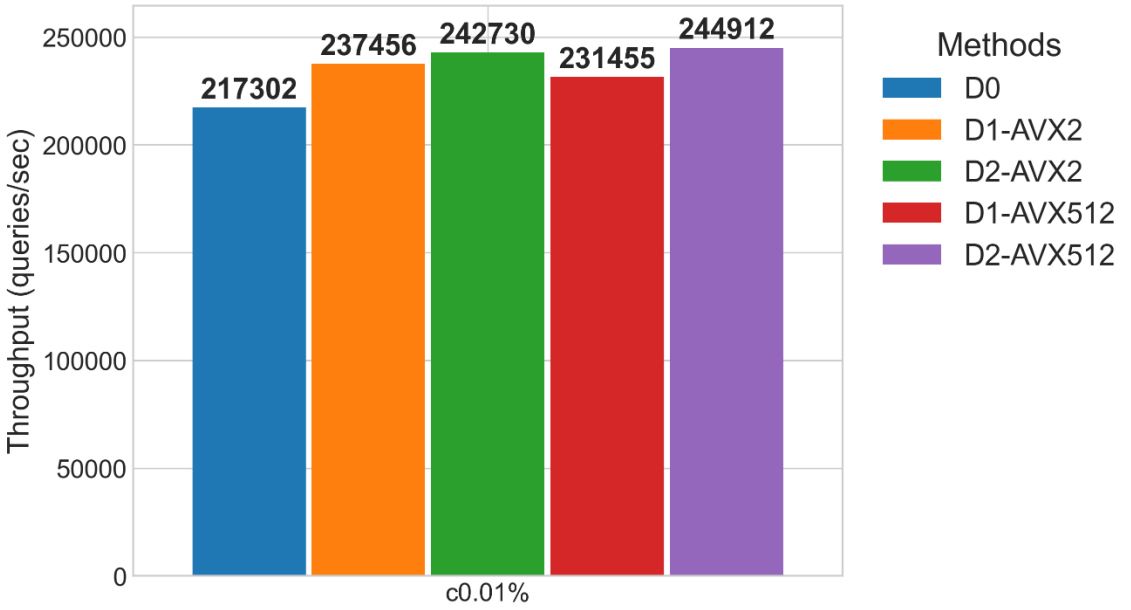

Throughput (queries/sec)
100000
80000
60000
40000
20000
0
90547
96184
97945
96085
99031
c0.01%
Methods
D0
D1-AVX2
D2-AVX2
D1-AVX512
D2-AVX512

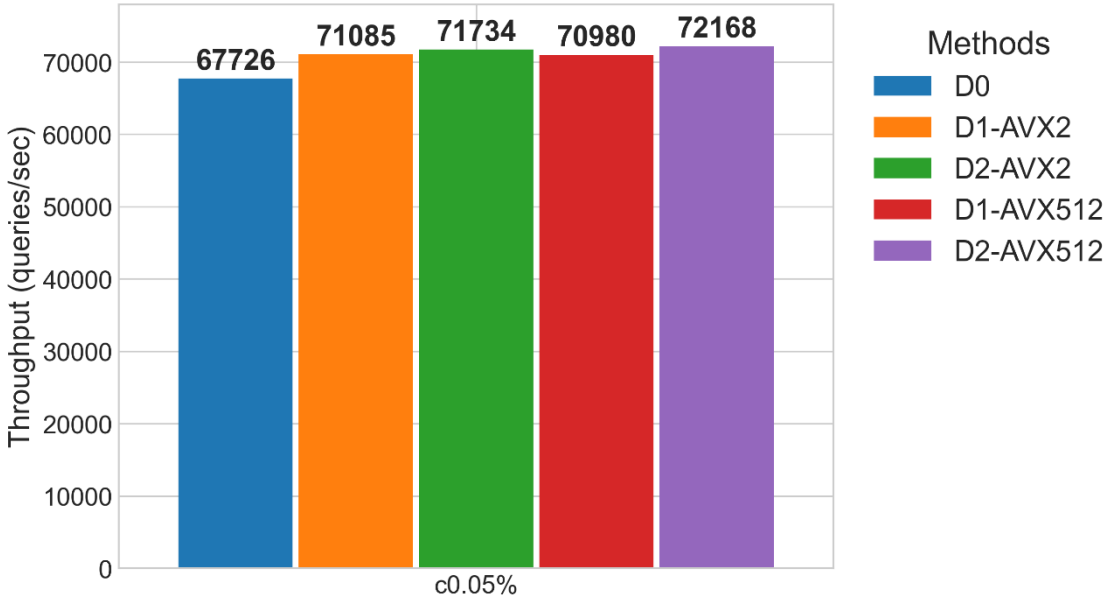

Throughput (queries/sec)
25000
20000
15000
10000
5000
0
25344
26538
26712
26793
26725
c0.05%
Methods
D0
D1-AVX2
D2-AVX2
D1-AVX512
D2-AVX512

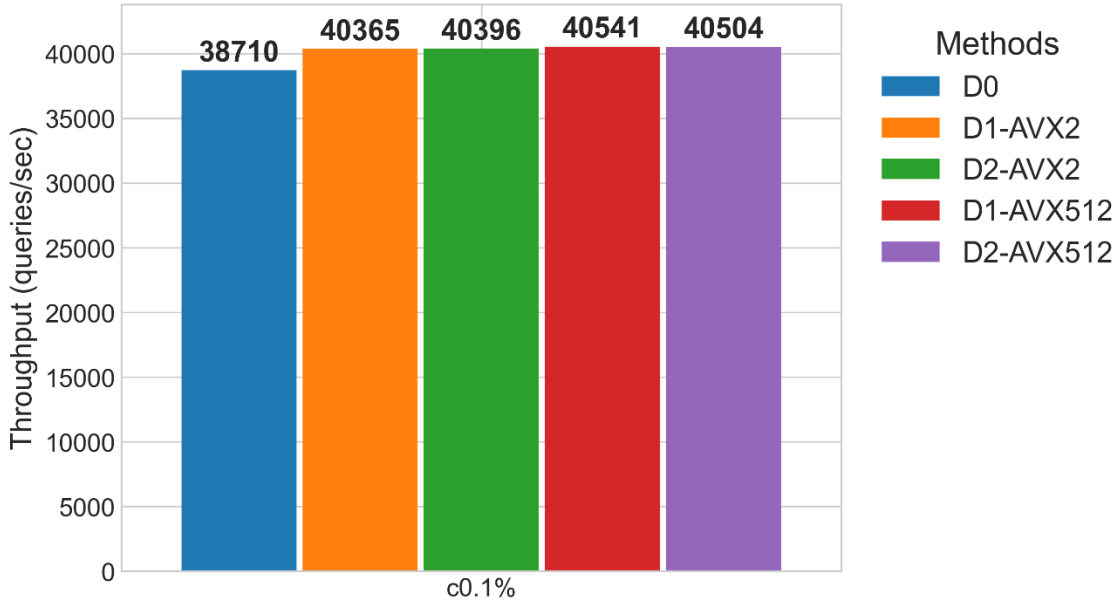

Throughput (queries/sec)
14000
12000
10000
8000
6000
4000
2000
0
14074
14586
14626
14779
14634
c0.1%
Methods
D0
D1-AVX2
D2-AVX2
D1-AVX512
D2-AVX512

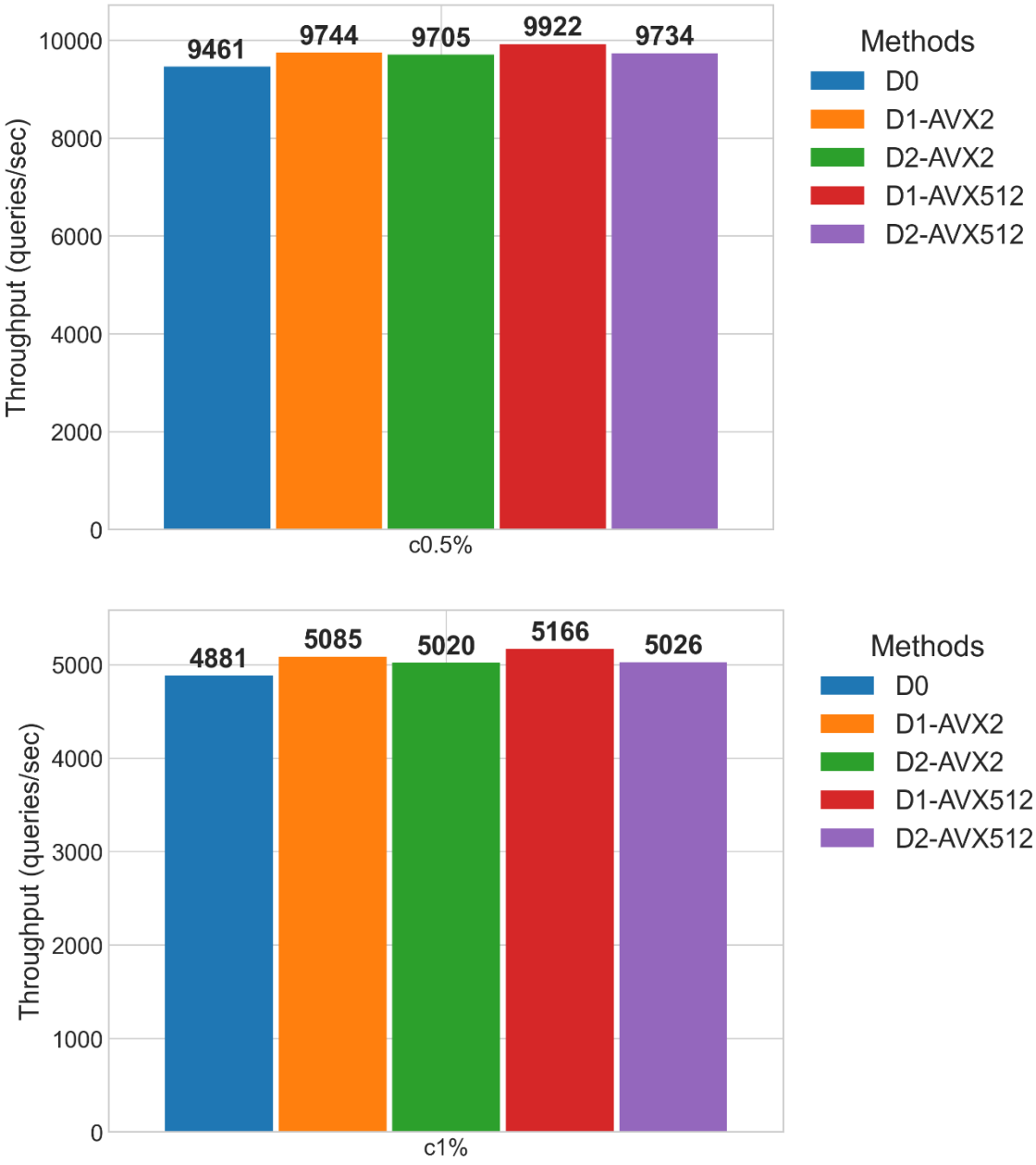


Εικόνα 15: Σύγκριση απόδοσης (queries/sec) για διαφορετικές επιλεκτικότητες στο σύνολο Edges


Throughput (queries/sec)
250000
200000
150000
100000
50000
0
217302
237456
242730
231455
244912
c0.01%
Methods
D0
D1-AVX2
D2-AVX2
D1-AVX512
D2-AVX512


Throughput (queries/sec)
70000
60000
50000
40000
30000
20000
10000
0
67726
71085
71734
70980
72168
c0.05%
Methods
D0
D1-AVX2
D2-AVX2
D1-AVX512
D2-AVX512


Throughput (queries/sec)
40000
35000
30000
25000
20000
15000
10000
5000
0
38710
40365
40396
40541
40504
c0.1%
Methods
D0
D1-AVX2
D2-AVX2
D1-AVX512
D2-AVX512



Εικόνα 16: Σύγκριση απόδοσης (queries/sec) για διαφορετικές επιλεκτικότητες στο σύνολο Roads

Η ερμηνεία των πειραματικών αποτελεσμάτων οδηγεί στα ακόλουθα συμπεράσματα:

- Η ανωτερότητα των διανυσματοποιημένων υλοποιήσεων (D1/D2) έναντι της βαθμωτής (D0) επιβεβαιώνεται σε όλο το εύρος των πειραμάτων. Ωστόσο, παρατηρείται ότι το σχετικό όφελος απόδοσης (speedup) δεν αυξάνεται αναλογικά με τον υπολογιστικό φόρτο. Αυτό οφείλεται στο ότι τα παρόντα πειράματα δεν περιλαμβάνουν διανυσματοποίηση στο επίπεδο των φύλλων. Καθώς η επιλεκτικότητα αυξάνεται, μεγαλύτερο μέρος του χρόνου εκτέλεσης

δαπανάται στην επεξεργασία των σημείων στους κόμβους-φύλλα, η οποία παραμένει βαθμωτή. Αυτό ο περιορισμός απόδοσης λόγω βαθμωτών λειτουργιών περιορίζει το συνολικό όφελος από την επιτάχυνση στους εσωτερικούς κόμβους.

- Η σύγκριση μεταξύ D1 και D2 αποκαλύπτει μια σημαντική εναλλαγή στην απόδοση. Στα ερωτήματα με εξαιρετικά υψηλή επιλεκτικότητα (0.01%), η διάταξη D2 είναι ελαφρώς ταχύτερη, κάτι που αποδίδεται στην καλύτερη χωρική τοπικότητα (cache locality) που προσφέρει. Ωστόσο, για όλες τις υπόλοιπες τιμές επιλεκτικότητας (≥ 0.05%), όπου ο υπολογιστικός φόρτος αυξάνεται, η διάταξη D1 γίνεται η ταχύτερη, αποδεικνύοντας την αρχιτεκτονική της ανωτερότητα για καθαρές διανυσματικές πράξεις.

- Σε όλες τις πειραματικές συνθήκες, οι υλοποιήσεις με AVX-512 επιτυγχάνουν την υψηλότερη απόδοση. Η ικανότητα επεξεργασίας διπλάσιου αριθμού στοιχείων ανά εντολή (16 έναντι 8) προσφέρει ένα σαφές και σταθερό πλεονέκτημα, το οποίο αποτυπώνεται στο υψηλότερο throughput του συστήματος.

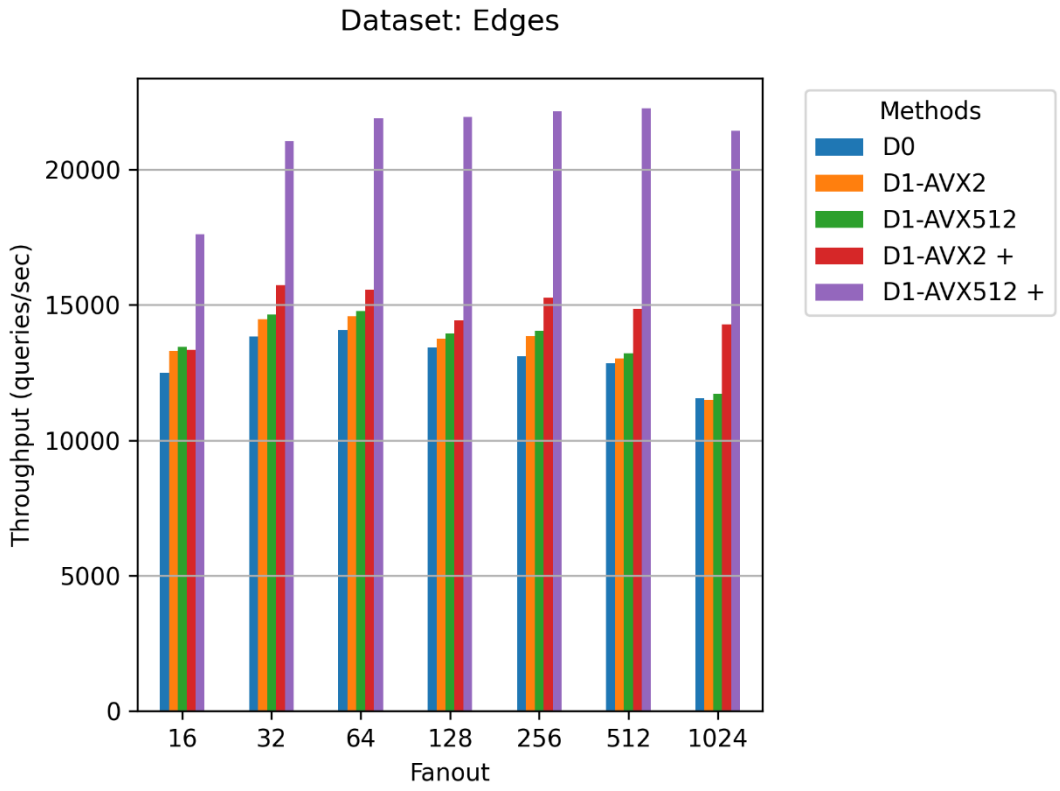
- Η σύγκριση των Εικόνων 15 και 16 αποκαλύπτει ότι, παρότι οι τιμές απόδοσης είναι υψηλότερες για το μικρότερο σύνολο δεδομένων Roads, λόγω του μικρότερου ύψους του R-tree, η σχετική κατάταξη και η συμπεριφορά των υλοποιήσεων παραμένουν αμετάβλητες. Η συνέπεια αυτή υποδεικνύει ότι τα συμπεράσματα μπορούν να γενικευτούν και δεν περιορίζονται από τα χαρακτηριστικά ενός συγκεκριμένου συνόλου δεδομένων.

Συμπερασματικά, η πειραματική αξιολόγηση καταδεικνύει ότι η υιοθέτηση διατάξεων μνήμης φιλικών προς SIMD αποτελεί κρίσιμη βελτιστοποίηση, με την D1 να προσφέρει την καλύτερη ισορροπία για ένα ευρύ φάσμα υπολογιστικών φόρτων.

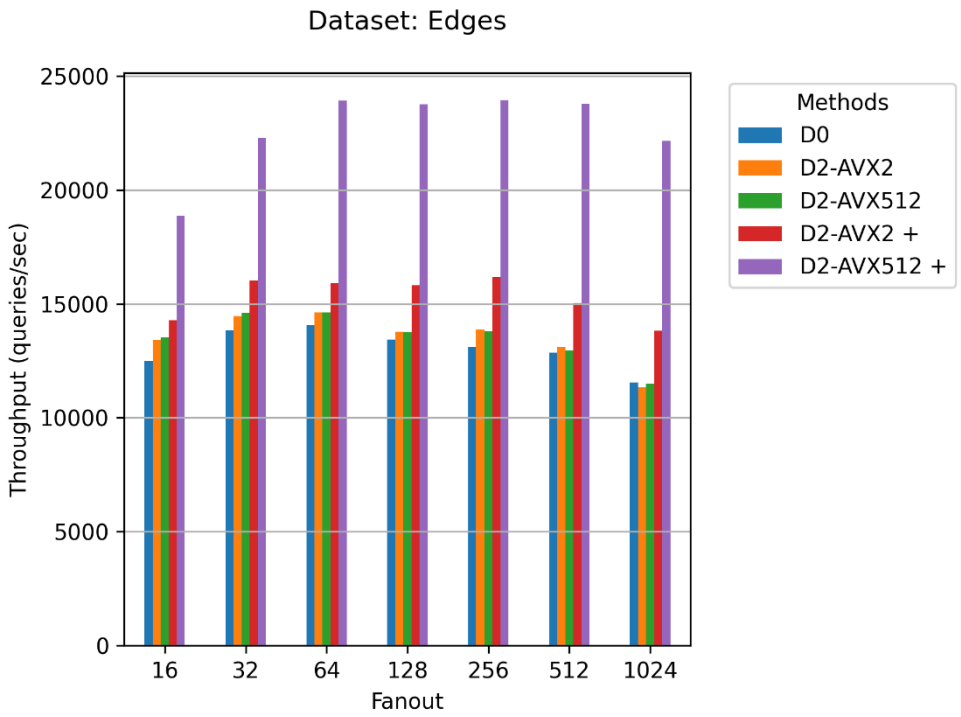
5.5.3 Απόδοση SIMD στα Φύλλα

Η διαδικασία απάντησης ενός ερωτήματος εύρους ολοκληρώνεται στους κόμβους-φύλλα (leaf nodes), όπου κάθε σημείο που περιέχεται σε αυτούς ελέγχεται ως προς το αν εμπίπτει στα όρια του ορθογωνίου του ερωτήματος. Όπως διαπιστώθηκε στην προηγούμενη ενότητα, το στάδιο αυτό, ειδικά σε ερωτήματα που δεν έχουν εξαιρετικά υψηλή επιλεκτικότητα, μπορεί να αποτελέσει σημαντικό μέρος του συνολικού υπολογιστικού κόστους, λειτουργώντας ως "scalar bottleneck". Ως εκ τούτου, η βελτιστοποίηση αυτού του ελέγχου μέσω SIMD εντολών είναι κρίσιμης σημασίας.

Για να μετρηθεί το όφελος αυτής της βελτιστοποίησης, συγκρίθηκαν οι διανυσματοποιημένες υλοποιήσεις (D1/D2) με τις αντίστοιχες εμπλουτισμένες εκδόσεις τους (που συμβολίζονται με "+"), οι οποίες εφαρμόζουν διανυσματοποίηση και κατά τον έλεγχο των σημείων στους κόμβους-φύλλα. Οι συγκρίσεις πραγματοποιήθηκαν στο σύνολο Edges για επιλεκτικότητα 0.1%. Οι Εικόνες 17 και 18 παρουσιάζουν την επίδραση της βελτιστοποίησης για τις διατάξεις D1 και D2 αντίστοιχα, ενώ η Εικόνα 19 παραθέτει την τελική σύγκριση των πλήρως βελτιστοποιημένων μεθόδων.



Εικόνα 17: Επίδραση της διανυσματοποίησης στους κόμβους-φύλλα για τη διάταξη D1 (Edges, 0.1%)



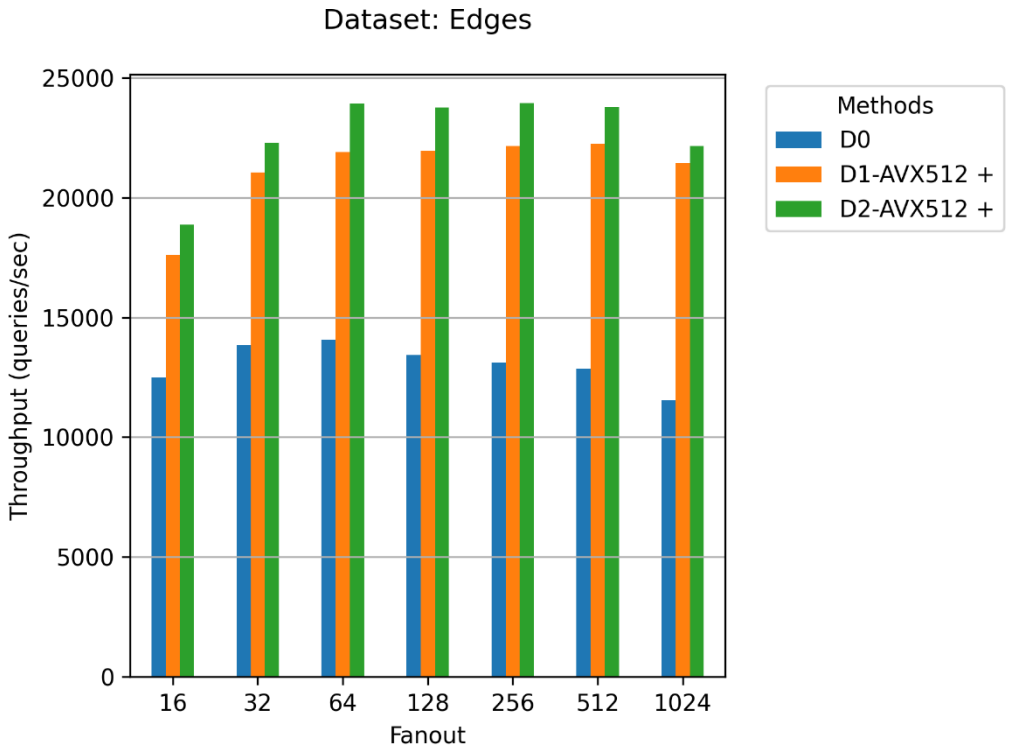
Εικόνα 18: Επίδραση της διανυσματοποίησης στους κόμβους-φύλλα για τη διάταξη D2 (Edges, 0.1%)

Στην Εικόνα 17, που αφορά τη διάταξη D1, παρατηρούνται τα εξής:

- Οι βελτιστοποιημένες μέθοδοι (D1-AVX2+ και D1-AVX512+) επιδεικνύουν υψηλότερη απόδοση (throughput) σε σχέση με τις μη βελτιστοποιημένες εκδόσεις τους σε όλο το εύρος των fanouts.
- Η μέθοδος D1-AVX512+ (κόκκινες ράβδοι) αναδεικνύεται ως η καλύτερη υλοποίηση, επιτυγχάνοντας την υψηλότερη απόδοση. Για παράδειγμα, σε fanout 64, η βελτίωση αγγίζει το 52%. Αυτό υπογραμμίζει ότι η διανυσματοποίηση του ελέγχου στα φύλλα είναι απαραίτητη για την πλήρη αξιοποίηση των δυνατοτήτων των διανυσμάτων 512-bit.
- Είναι αξιοσημείωτο ότι οι μη βελτιστοποιημένες εκδόσεις D1-AVX2 και D1-AVX-512 έχουν σχεδόν πανομοιότυπη απόδοση. Αυτό υποδηλώνει ότι η απλή χρήση ευρύτερων διανυσμάτων (512-bit έναντι 256-bit) χωρίς την εφαρμογή της βελτιστοποίησης στα φύλλα, όπου εντοπίζεται η καθυστέρηση, δεν προσφέρει ουσιαστικό πλεονέκτημα.

Τα αποτελέσματα για τη διάταξη D2, όπως αποτυπώνονται στην Εικόνα 18, ακολουθούν παρόμοια τάση με τη D1:

- Οι βελτιστοποιημένες εκδόσεις "+" είναι και εδώ σαφώς ανώτερες, με την αύξηση της απόδοσης να είναι ακόμα πιο έντονη, φτάνοντας το 65% σε fanout 64.
- Η μέθοδος D2-AVX512+ (κόκκινες ράβδοι) ξεχωρίζει και πάλι ως η ταχύτερη, επιβεβαιώνοντας τα οφέλη του συνδυασμού των εντολών AVX-512 με τον πλήρως βελτιστοποιημένο αλγόριθμο.
- Η σχέση μεταξύ των μη βελτιστοποιημένων εκδόσεων (D2-AVX2 και D2-AVX-512) παραμένει η ίδια, με αμελητέες διαφορές στην απόδοσή τους, ενισχύοντας το συμπέρασμα ότι η υπολογιστική καθυστέρηση βρισκόταν στη βαθμωτή επεξεργασία των φύλλων.



Εικόνα 19: D1-AVX512 + έναντι D2-AVX512 +

Στην Εικόνα 19 συγκρίνονται οι καλύτερες υλοποιήσεις από κάθε κατηγορία (D1-AVX512+ έναντι D2-AVX512+).

- Η διάταξη D2 (πορτοκαλί ράβδοι, D2-AVX512+) είναι σταθερά ταχύτερη από τη διάταξη D1 (μπλε ράβδοι, D1-AVX512+) σε όλα τα fanouts.
- Η βελτίωση που προσφέρει η D2 είναι μετρήσιμη και σημαντική. Αφού και οι δύο μέθοδοι εξάλειψαν το υπολογιστικό bottleneck στα φύλλα, η ανώτερη χωρική τοπικότητα της D2 της δίνει το τελικό προβάδισμα στην απόδοση.

- Βέλτιστο Fanout: Και οι δύο διατάξεις επιτυγχάνουν τη μέγιστη απόδοσή τους για τιμές fanout μεταξύ 64 και 128, με την απόδοση να μειώνεται σε μεγαλύτερες τιμές fanout.

Συμπερασματικά, η ανάλυση αυτή καταδεικνύει ότι:

- Η διανυσματοποίηση του ελέγχου στους κόμβους-φύλλα (που συμβολίζεται με "+") είναι απολύτως κρίσιμη για την επίτευξη υψηλής απόδοσης.
- Η διάταξη δεδομένων D2 είναι αντικειμενικά ανώτερη από τη διάταξη D1 όταν ολόκληρος ο αλγόριθμος είναι βελτιστοποιημένος.
- Ο συνδυασμός της διάταξης D2 με την πλήρως βελτιστοποιημένη υλοποίηση AVX-512 (D2-AVX512+) αποτελεί τη βέλτιστη διαμόρφωση, καθώς απέδωσε την υψηλότερη ταχύτητα από όλες τις μεθόδους που δοκιμάστηκαν

5.5.4 Πείραμα Αξιολόγησης Απόδοσης

Για την πληρέστερη κατανόηση των παραγόντων που καθορίζουν την απόδοση, το τελικό αυτό πείραμα εστιάζει στην ανάλυση των μετρητών απόδοσης υλικού (hardware performance counters). Οι μετρητές αυτοί, που καταγράφονται απευθείας από την CPU, μας επιτρέπουν να προχωρήσουμε πέρα από τον απλό χρόνο εκτέλεσης και να εξετάσουμε πώς κάθε υλοποίηση αλληλοεπιδρά με την αρχιτεκτονική του επεξεργαστή σε επίπεδο εντολών, διακλαδώσεων και προσβάσεων στην κρυφή μνήμη. Οι Πίνακες 3 και 4 παρουσιάζουν τα δεδομένα που συλλέχθηκαν για τις βασικές υλοποιήσεις στα σύνολα δεδομένων Edges και Roads αντίστοιχα, προσφέροντας μια εξήγηση για την ιεραρχία απόδοσης που παρατηρήθηκε στα προηγούμενα πειράματα.

Πίνακας 3: Μετρητές απόδοσης υλικού για το σύνολο δεδομένων Edges (Fanout=64, Selectivity=0.1%).

METHOD/STATS	Cycles	Instructions	Branch-instructions	Branch-misses	L1-references	L1-misses
D0	357K	581K	145k	1865	213K	8259
D1-AVX2 +	322K	476K	39K	173	89K	8139
D2-AVX2 +	315K	483K	38K	131	98K	8077
D1-AVX512 +	218K	329K	27K	206	73K	8167
D2-AVX512 +	211K	324K	25K	128	73K	8054

Πίνακας 4: Μετρητές απόδοσης υλικού για το σύνολο δεδομένων Roads (Fanout=64, Selectivity=0.1%).

METHOD/STATS	Cycles	Instructions	Branch-instructions	Branch-misses	L1-references	L1-misses
D0	131K	207K	52K	930	75K	3083
D1-AVX2 +	114K	169K	15K	84	32K	3018
D2-AVX2 +	109K	172K	14K	64	35K	2982
D1-AVX512 +	79K	118K	11K	102	26K	3025
D2-AVX512 +	78K	116K	9K	59	26K	2966

Η ανάλυση των μετρητών οδηγεί στα ακόλουθα συμπεράσματα:

- Ο καθοριστικότερος δείκτης απόδοσης, οι κύκλοι ρολογιού (Cycles), επιβεβαιώνει ότι η D2-AVX512+ είναι η ταχύτερη μέθοδος. Η υπεροχή της οφείλεται άμεσα στον δραστικά μειωμένο αριθμό εντολών (Instructions) που εκτελεί, περίπου 44% λιγότερες σε σχέση με τη βαθμωτή D0. Αυτό αποδεικνύει εμπράκτως το θεμελιώδες όφελος του SIMD: η εκτέλεση μιας εντολής σε πολλαπλά δεδομένα μειώνει τον συνολικό αριθμό εντολών που απαιτούνται.
- Η πιο εντυπωσιακή διαφορά παρατηρείται στις διακλαδώσεις. Η βέλτιστη υλοποίηση D2-AVX512+ επιτυγχάνει μια μείωση στις αστοχίες πρόβλεψης (Branch-misses) της τάξης του 93-94% σε σύγκριση με τη βαθμωτή D0. Κάθε branch miss αποτελεί ένα εξαιρετικά κοστοβόρο γεγονός για την CPU. Η σχεδόν πλήρης εξάλειψή τους μέσω της χρήσης μασκών bit αντί για εντολές if αποτελεί έναν από τους κύριους λόγους για την επιτάχυνση των διανυσματοποιημένων μεθόδων.
- Η ανάλυση της συμπεριφοράς της L1 cache αποκαλύπτει τις θεμελιώδεις αρχιτεκτονικές διαφορές των διατάξεων. Η βαθμωτή υλοποίηση D0 εκτελεί έναν δραματικά υψηλότερο αριθμό προσπελάσεων στην L1 (L1-references), σχεδόν τριπλάσιο σε σχέση με τις βελτιστοποιημένες εκδόσεις. Αυτή η έντονη κίνηση δεδομένων οφείλεται στην "cache-unfriendly" φύση της διάταξης D0 (Array of Structures), η οποία αναγκάζει τον επεξεργαστή σε πολλαπλές, ασυνεχείς προσβάσεις για τη συλλογή ομοειδών δεδομένων. Αντιθέτως, οι διατάξεις D1 και D2 είναι σχεδιασμένες για γραμμική προσπέλαση, μειώνοντας τις συνολικές αναφορές στη μνήμη. Παρότι η D0 έχει και τις περισσότερες απόλυτες L1-misses,

η ανώτερη χωρική τοπικότητα της D2 επιβεβαιώνεται από το γεγονός ότι καταγράφει σταθερά τον χαμηλότερο αριθμό L1-misses και στα δύο datasets. Αυτή η οριακή αλλά σταθερή αποδοτικότητα είναι ο παράγοντας που της δίνει το τελικό προβάδισμα έναντι της D1.

Συμπερασματικά, η ανάλυση σε επίπεδο υλικού επιβεβαιώνει ότι η κορυφαία απόδοση της υλοποίησης D2-AVX512+ είναι αποτέλεσμα ενός συνδυασμού τριών παραγόντων: μειωμένου αριθμού εντολών λόγω παραλληλισμού SIMD, εξαιρετικής απόδοσης πρόβλεψης διακλαδώσεων λόγω του αλγοριθμικού σχεδιασμού, και βέλτιστης χρήσης της κρυφής μνήμης λόγω της ανώτερης διάταξης δεδομένων. Παρατήρηση: Οι τιμές των μετρητών απόδοσης υλικού στους Πίνακες 3 και 4 έχουν προκύψει μετά από στρογγυλοποίηση.

Κεφάλαιο 6. Συμπεράσματα

Η παρούσα διπλωματική εργασία τοποθετείται στο ερευνητικό πεδίο της βελτιστοποίησης των συστημάτων βάσεων δεδομένων, εστιάζοντας στην επιτάχυνση των χωρικών ερωτημάτων σε in-memory περιβάλλοντα. Το κεντρικό ερευνητικό ερώτημα που διερευνήθηκε αφορά τον τρόπο με τον οποίο οι αρχές του παραλληλισμού σε επίπεδο δεδομένων (Data-Level Parallelism) μπορούν να ενσωματωθούν σε κλασικές δομές ευρετηρίασης, όπως το R-tree, για την πλήρη αξιοποίηση της υπολογιστικής ισχύος των σύγχρονων επεξεργαστών. Για την απάντηση αυτού του ερωτήματος, η μεθοδολογία που ακολουθήθηκε βασίστηκε στον σχεδιασμό και την υλοποίηση τριών διακριτών διατάξεων μνήμης (memory layouts) για τους κόμβους του R-tree: της παραδοσιακής D0, της αρχιτεκτονικά φιλικής προς SIMD D1 (Structure of Arrays), και της βελτιστοποιημένης για χωρική τοπικότητα D2 (interleaved). Πάνω σε αυτές τις δομές αναπτύχθηκαν αλγόριθμοι αναζήτησης που αξιοποιούν εντολές AVX2 και AVX-512. Μέσω μιας σειράς πειραμάτων, αξιολογήθηκε η απόδοση των υλοποιήσεων, οδηγώντας σε μια σειρά από συμπεράσματα που αποδεικνύουν τη συνεισφορά της παρούσας εργασίας.

Η πειραματική ανάλυση οδήγησε στην κατανόηση των παραγόντων που επιδρούν στην απόδοση:

1. Σε θεμελιώδες επίπεδο, επιβεβαιώθηκε ότι η διανυσματοποίηση προσφέρει αύξηση της απόδοσης έναντι της παραδοσιακής, βαθμωτής προσέγγισης (D0). Η ικανότητα επεξεργασίας πολλαπλών MBRs ή σημείων ταυτόχρονα αποτελεί μια θεμελιώδη βελτιστοποίηση, η οποία επικυρώθηκε σε όλο το εύρος των πειραμάτων.
2. Σε ένα βαθύτερο επίπεδο, αποδείχθηκε ότι η διάταξη των δεδομένων στη μνήμη είναι εξίσου κρίσιμη με τον ίδιο τον αλγόριθμο. Η μελέτη ανέδειξε μια δυναμική σχέση μεταξύ της βέλτιστης διάταξης και του εκάστοτε περιοριστικού παράγοντα της απόδοσης. Η διάταξη D1 αποδείχθηκε αρχιτεκτονικά ανώτερη σε σενάρια που περιορίζονταν από την υπολογιστική ισχύ, ενώ η D2 υπερείχε σε σενάρια που περιορίζονταν από την ταχύτητα προσπέλασης της μνήμης.
3. Το πιο εξειδικευμένο συμπέρασμα ήταν ο καθοριστικός ρόλος της διανυσματοποίησης στους κόμβους-φύλλα. Η εφαρμογή SIMD στο τελικό στάδιο του ελέγχου των σημείων εξάλειψε μια μεγάλη καθυστέρηση λόγω της βαθμωτής επεξεργασίας των φύλλων, οδηγώντας σε τεράστια αύξηση της απόδοσης. Η

βελτιστοποίηση αυτή αποδείχθηκε απαραίτητη συνθήκη για να αναδειχθεί το πλήρες δυναμικό των φιλικών για χρήση SIMD διατάξεων και να καθοριστεί η αποδοτικότερη υλοποίηση.

Συγκεφαλαιώνοντας, η παρούσα εργασία καταλήγει στο συμπέρασμα ότι η βέλτιστη απόδοση για ερωτήματα εύρους σε R-tree επιτυγχάνεται μέσω μιας ολιστικής προσέγγισης που συνδυάζει την κατάλληλη αρχιτεκτονική δεδομένων με την πλήρη διανυσματοποίηση του αλγορίθμου. Η βέλτιστη διαμόρφωση που προέκυψε από την πειραματική διαδικασία είναι ο συνδυασμός της διάταξης μνήμης D2 με την πλήρως διανυσματοποιημένη υλοποίηση AVX512 (D2-AVX512+). Η διαμόρφωση αυτή αξιοποιεί ταυτόχρονα την ανώτερη χωρική τοπικότητα της D2, το μέγιστο πλάτος παραλληλισμού του AVX512 και την εξάλειψη των υπολογιστικών καθυστερήσεων σε ολόκληρη τη δομή του δέντρου.

Τα αποτελέσματα αυτά ανοίγουν τον δρόμο για μελλοντικές ερευνητικές επεκτάσεις, οι οποίες θα μπορούσαν να περιλαμβάνουν:

- Εφαρμογή Software Prefetching: Η υλοποίηση της βελτιστοποίησης O2 από τη δημοσίευση [RA23], για την απόκρυψη της καθυστέρησης της μνήμης, θα μπορούσε να προσφέρει περαιτέρω αύξηση της απόδοσης, ειδικά στη διάταξη D2.
- Διανυσματοποίηση άλλων Τελεστών: Η επέκταση της μεθοδολογίας σε άλλους χωρικούς τελεστές, όπως το k-Nearest Neighbors (k-NN) και το Spatial Join.
- Δυναμικές Ενημερώσεις: Διερεύνηση της απόδοσης σε δυναμικά σενάρια, με την ανάπτυξη διανυσματοποιημένων αλγορίθμων για τις λειτουργίες εισαγωγής και διαγραφής.

Συνοψίζοντας, η εργασία αυτή επιβεβαίωσε εμπράκτως ότι ο συνδυασμός φιλικών για χρήση SIMD διατάξεων μνήμης και διανυσματοποιημένων αλγορίθμων αποτελεί μια ισχυρή και αποτελεσματική στρατηγική για την επιτάχυνση των χωρικών ερωτημάτων σε σύγχρονες αρχιτεκτονικές υλικού.

Βιβλιογραφία

- [RSV01] Rigaux, P., Scholl, M., & Voisard, A. (2001). Spatial Databases: With Application to GIS. Morgan Kaufmann.
- [GG98] Gaede, V., & Günther, O. (1998). Multidimensional access methods. ACM Computing Surveys, 30(2), 170–231.
- [Ben75] Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. Communications of the ACM, 18(9), 509–517.
- [FB74] Finkel, R. A., & Bentley, J. L. (1974). Quad trees: A data structure for retrieval on composite keys. Acta Informatica, 4(1), 1–9.
- [Gut84] Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. In SIGMOD, 47–57.
- [BKS+90] Beckmann, N., Kriegel, H. P., Schneider, R., & Seeger, B. (1990). The R*-tree: An efficient and robust access method for points and rectangles. In SIGMOD, 322–331.
- [KF94] Kamel, I., & Faloutsos, C. (1994). Hilbert R-tree: An improved R-tree using fractals. In VLDB, 500–509.
- [BKK96] Berchtold, S., Keim, D. A., & Kriegel, H. P. (1996). The X-tree: An index structure for high-dimensional data. In VLDB, 28–39.
- [WPB+09] Willhalm, T., Popovici, N., Boshmaf, Y., Plattner, C., Zeier, A., & Müller, H. (2009). SIMD-Scan: Ultra-Fast in-Memory Table Scan using on-Chip Vector Processing Units. *Proceedings of VLDB Endowment (PVLDB)*, 2(1), 385–394.
- [LP13] Li, Y., & Patel, J. M. (2013). BitWeaving: Fast scans for main memory data processing. In *Proceedings of ACM SIGMOD*, 289–300.
- [BLP11] Blanas, S., Li, Y., & Patel, J. M. (2011). Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *Proceedings of ACM SIGMOD*, 37–48.
- [BTA+13] Balkesen, C., Teubner, J., Alonso, G., & Özsu, M. T. (2013). Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *Proceedings* of IEEE ICDE*, 362–373.*
- [KCS+10a] Satish, N., Kim, C., Chhugani, J., Nguyen, A. D., Lee, V. W., Kim, D., … Dubey, P. (2010). Fast sort on CPUs and GPUs: A case for bandwidth oblivious SIMD sort. In *Proceedings of ACM SIGMOD*, 351–362.

- [Ros04] Ross, K. A. (2004). Selection conditions in main memory. *ACM Transactions on Database Systems (TODS)*, 29(1), 132–161.
- [SWL11] Schlegel, B., Willhalm, T., & Lehner, W. (2011). Fast B-Tree Search using SIMD Instructions. In *Proceedings of DaMoN*, 1–8.
- [LKN13] Leis, V., Kemper, A., & Neumann, T. (2013). The Adaptive Radix Tree: ARTful indexing for main-memory databases. In *Proceedings of IEEE ICDE*, 38–49.
- [KCS+10] Kim, C., Chhugani, J., Satish, N., Nguyen, A. D., Lee, V. W., Kim, D., … Dubey, P. (2010). FAST: Fast architecture sensitive tree search on modern CPUs and GPUs. In *Proceedings of ACM SIGMOD*, 339–350.
- [YOHY12] Yamamuro, T., Onizuka, M., Hitaka, T., & Yamamuro, M. (2012). VAST-Tree: A vector-advanced and compressed structure for massive data tree traversal. In *Proceedings of EDBT*, 353–364.
- [RA23] Rayhan, Y., & Aref, W. G. (2023). SIMD-ified R-tree Query Processing and Optimization. In *Proceedings of ACM SIGSPATIAL*, 1–10.
- [SRF87] Sellis, T., Roussopoulos, N., & Faloutsos, C. (1987). The R^+-tree: A dynamic index for multi-dimensional objects. In *Proceedings of VLDB*, 507–518.
- [PrCL]https://www.promptcloud.com/blog/big-data-an-early-history-into-technology/
- [Xilinx]https://xilinx.github.io/Vitis_Libraries/data_analytics/2022.1/guide_L2/internals/geospatialJoin.html
- [SpHa] https://spatialhadoop.cs.umn.edu/datasets.html
- [GSDQ]https://www.linkedin.com/pulse/geo-spatial-data-queries-mongodb-how-works-r-tree-indexing-joshi/